

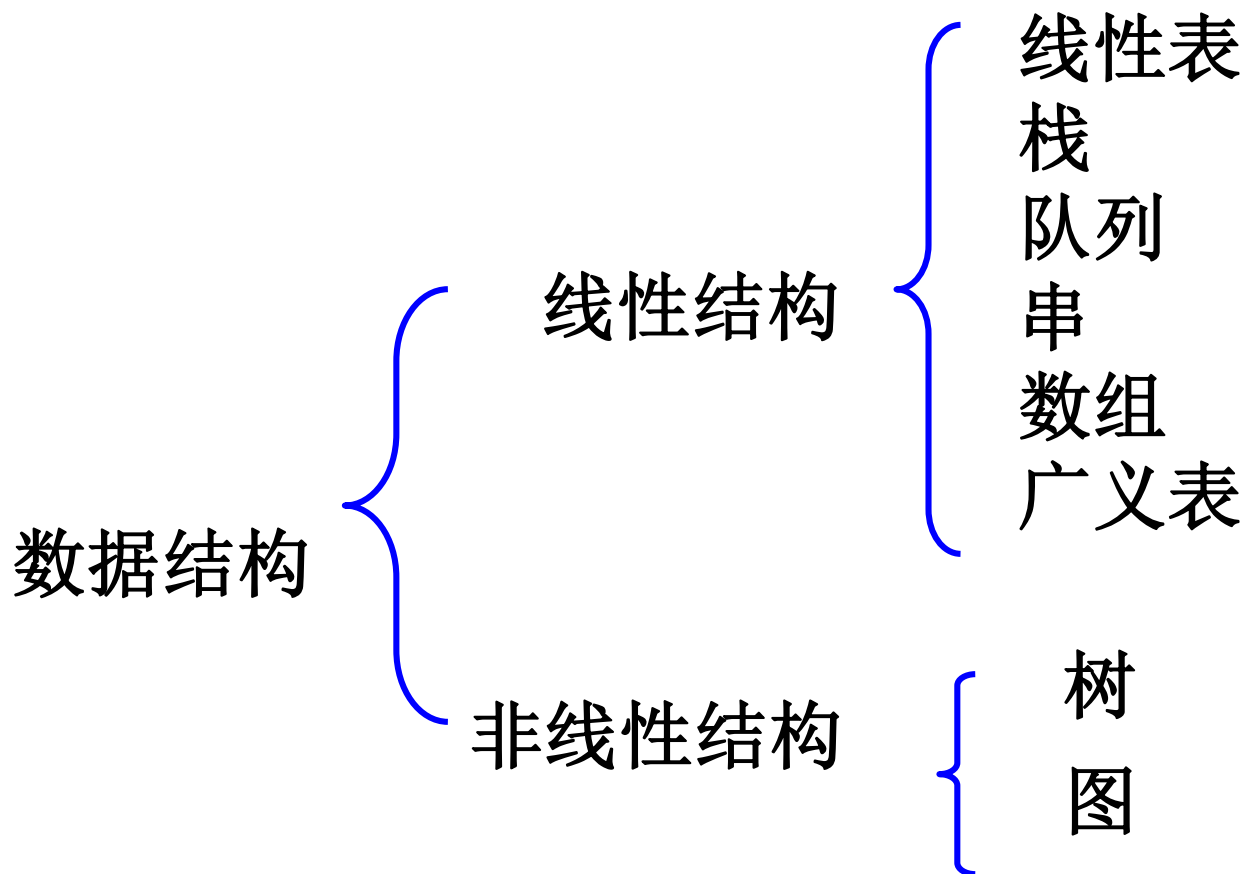


第5章 树





数据结构类型



本章是课程的重点



本章学习要点

树和二叉树

树的ADT

逻辑结构

存储结构

二叉树

逻辑结构

存储结构

基本性质

遍历二叉树

线索二叉树

树和森林

树的存储结构

树的遍历

树和森林与二叉树的转换

树的应用

Huffman树

判定过程



本章重点与难点

■ 重点：

- (1) 二叉树的定义、存储结构、遍历及应用；
- (2) 线索二叉树的定义、存储结构及相应的操作；
- (3) 树和森林与二叉树之间的相互转化方法；
- (4) 哈夫曼树的建立方法和哈夫曼编码。

■ 难点：

- (1) 二叉树的构造 、应用；
- (2) 线索二叉树的遍历和相应的操作。



第五章 树

5.1 树的有关概念

5.2 二叉树

5.3 二叉树的遍历

5.4 遍历的应用

5.5 线索二叉树

5.6 树和森林

5.7 树及应用



第五章 树

5.1 树的有关概念

5.2 二叉树

5.3 二叉树的遍历

5.4 遍历的应用

5.5 线索二叉树

5.6 树和森林

5.7 树及应用



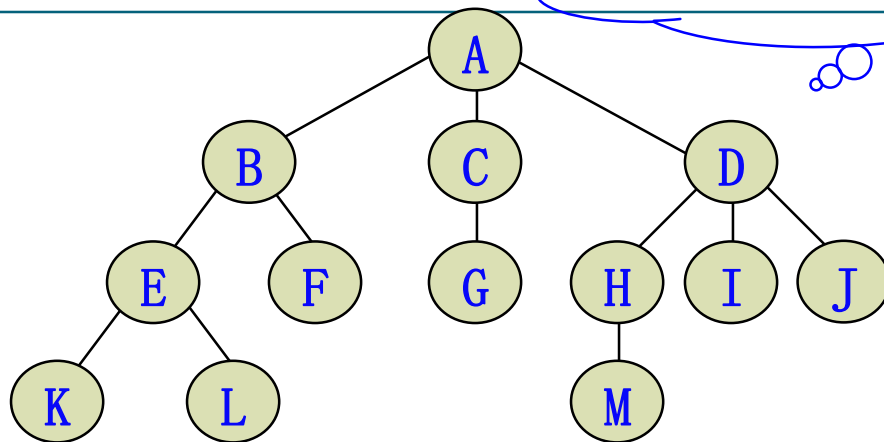
5.1 树的有关概念

• 树的定义:

树形结构是一种重要的非线性结构。树是 n 个结点的有限集合，在任一非空树中：

- (1) 有且仅有一个称为**根**的结点。
- (2) 其余结点可分为 m 个**互不相交的集合**，而且这些集合中的每一集合都本身又是一棵树，称**为根的子树**。

如何定义树？

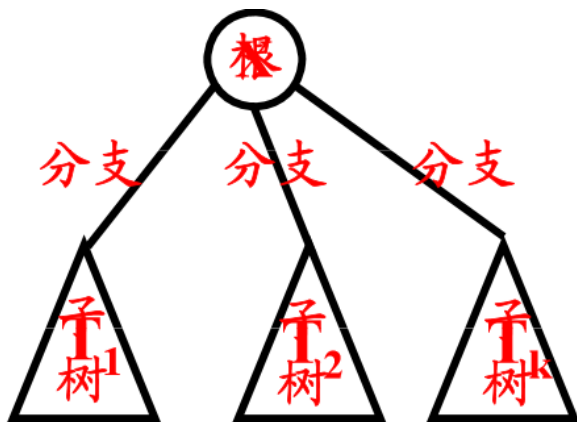


说明：树是递归结构，在树的定义中又用到了树的概念



5.1 树的有关概念

- 树的构造性递归定义：
 - 一个结点 x 组成的集合 $\{x\}$ 是**一棵树**，这个结点 x 称为**这棵树的根**（root）。
 - 假设 x 是一个结点， $T_1, T_2, \dots, T_k$ 是 k 棵互不相交的树，可以构造一棵新树：令 x 为根，并有 k 条边由 x 指向树 $T_1, T_2, \dots, T_k$ 。这些边也叫做**分支**， $T_1, T_2, \dots, T_k$ 称作根为 x 的树之**子树**（SubTree）。



说明：

- ✓ 递归定义，但不会产生循环定义；
- ✓ 构造性定义便于树型结构的建立；
- ✓ 一株树的每个结点都是这株树的某株子树的根。



5.1 树的有关概念

- 树的概念:

ADT Tree{

数据对象 D: D是具有相同特性的数据元素的集合。

数据关系 R:

若D为空集, 则称为空树。

否则:

(1) 在D中存在唯一的称为根的数据元素root;

(2) 当 $n > 1$ 时, 其余结点可分为 m ($m > 0$) 个互不相交的有限集 $T_1, T_2, \dots, T_m$, 其中每一棵子树本身又是一棵符合本定义棵树, 称为根root的子树。



5.1 树的有关概念

- 树的概念:

ADT Tree{

数据对象 D:

数据关系 R:

基本操作 P:

查 找 类

插 入 类

删 除 类

}



5.1 树的有关概念

- 树的概念:

查找类:

Root(T) // 求树的根结点

Value(T, cur_e) // 求当前结点的元素值

Parent(T, cur_e) // 求当前结点的双亲结点

LeftChild(T, cur_e) // 求当前结点的最左孩子

RightSibling(T, cur_e) // 求当前结点的右兄弟

TreeEmpty(T) // 判定树是否为空树

TreeDepth(T) // 求树的深度

TraverseTree(T, Visit()) // 遍历



5.1 树的有关概念

- 树的概念:

插入类:

`InitTree(&T)` // 初始化置空树

`CreateTree(&T, definition)` // 按定义构造树

`Assign(T, cur_e, value)` // 给当前结点赋值

`InsertChild(&T, &p, i, c)`

// 将以c为根的树插入为结点p的第i棵子树



5.1 树的有关概念

- 树的概念:

删除类:

`ClearTree(&T)` // 将树清空

`DestroyTree(&T)` // 销毁树的结构

`DeleteChild(&T, &p, i)`

// 删除结点p的第i棵子树



5.1 树的有关概念

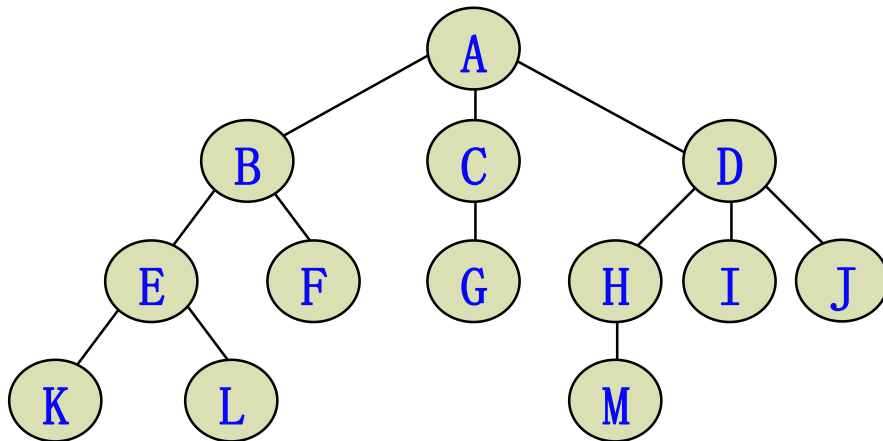
- 树的概念:

例如：集合 $T = \{A, B, C, D, E, F, G, H, I, J, K, L, M\}$

A是根，其余结点可以划分为3个互不相交的集合：

$T_1 = \{B, E, F, K, L\}$, $T_2 = \{C, G\}$, $T_3 = \{D, H, I, J, M\}$

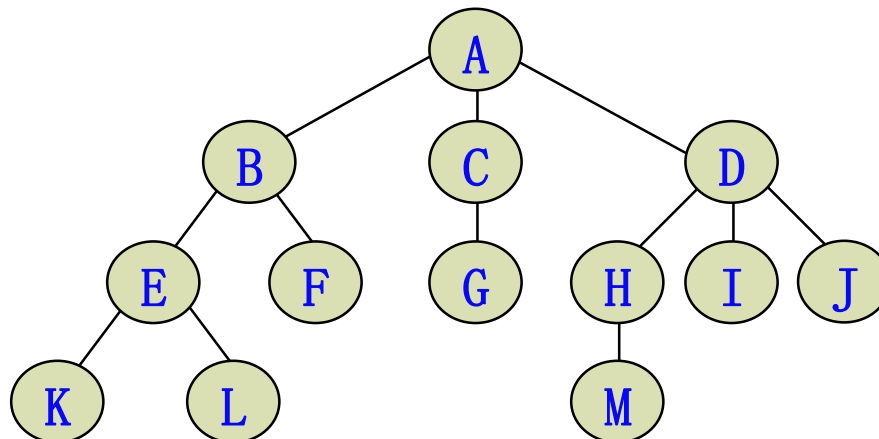
这些集合中的每一集合都本身又是一棵树，它们是A的子树。





5.1 树的有关概念

- 树的逻辑结构特点：
 - 树是一种分枝结构，树中只有根结点没有前驱；其余结点有零个或多个后继；
 - 除根外，其余结点都有且仅有一个前驱；都存在唯一一条从根到该结点的路径。





5.1 树的有关概念

线性结构

第一个数据元素
(无前驱)

最后一个数据元素
(无后继)

其它数据元素
(一个前驱、一个后继)

非线性结构——树

根结点
(无前驱)

多个叶子结点
(无后继)

其它数据元素
(一个前驱、多个后继)

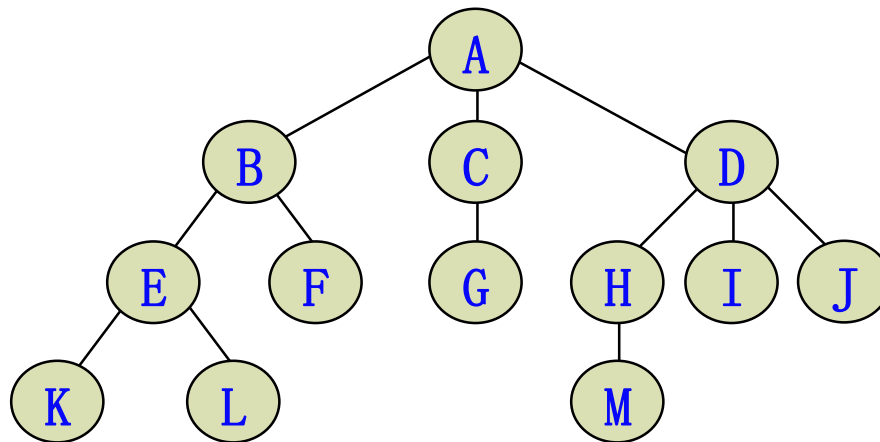


5.1 树的有关概念

- 树的应用：

1) 树可表示具有分枝结构关系的对象

例 单位行政机构的组织关系





5.1 树的有关概念

- 树的应用：

- 2) 树是常用的数据组织形式

- 有些应用中数据元素之间并不存在分支结构关系，但是为了便于管理和使用数据，将它们用树的形式来组织。

例 **计算机的文件系统：** 不论是DOS文件系统还是window文件系统，所有的文件是用树的形式来组织的。

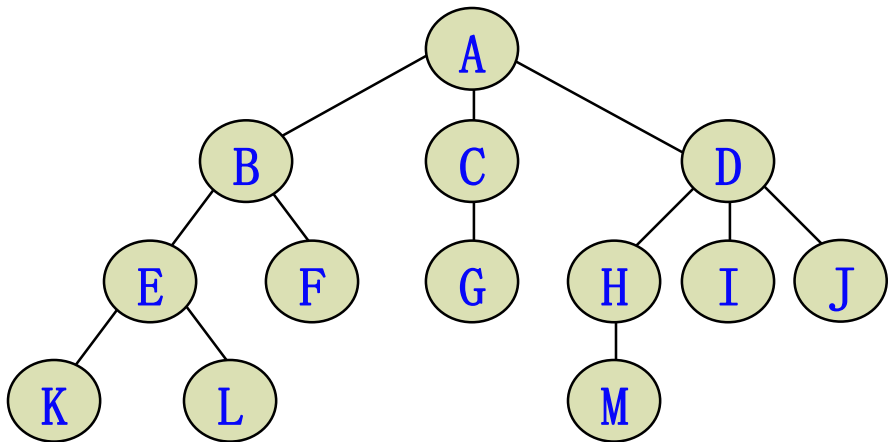




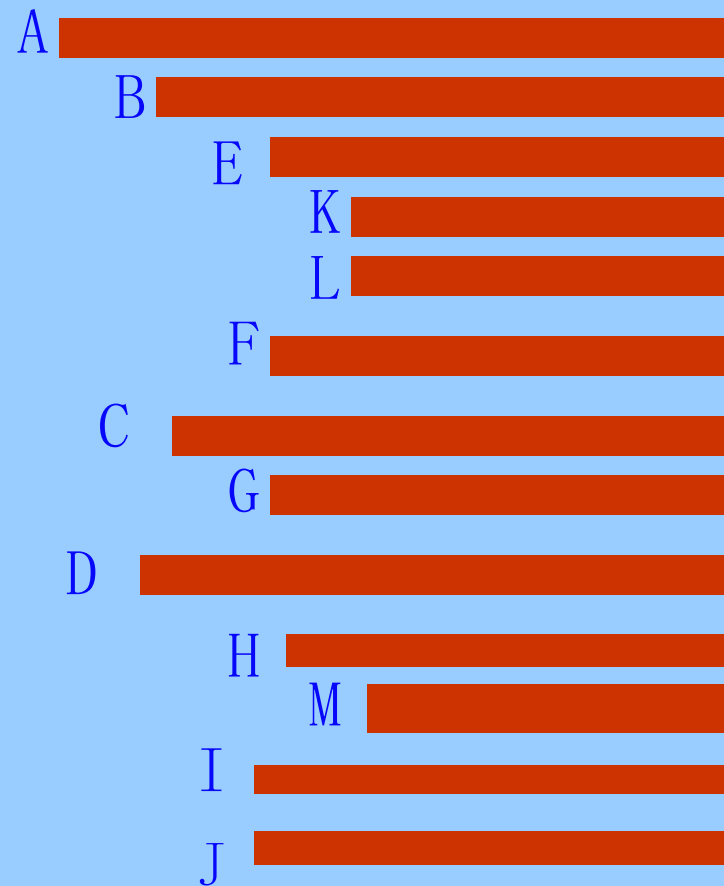
5.1 树的有关概念

- 树的表示:

- (1) 树形表示法



- (2) 凹入表示法

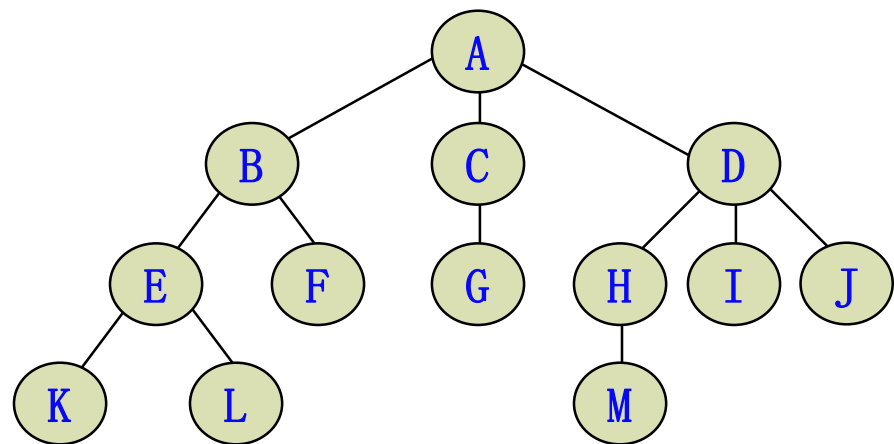
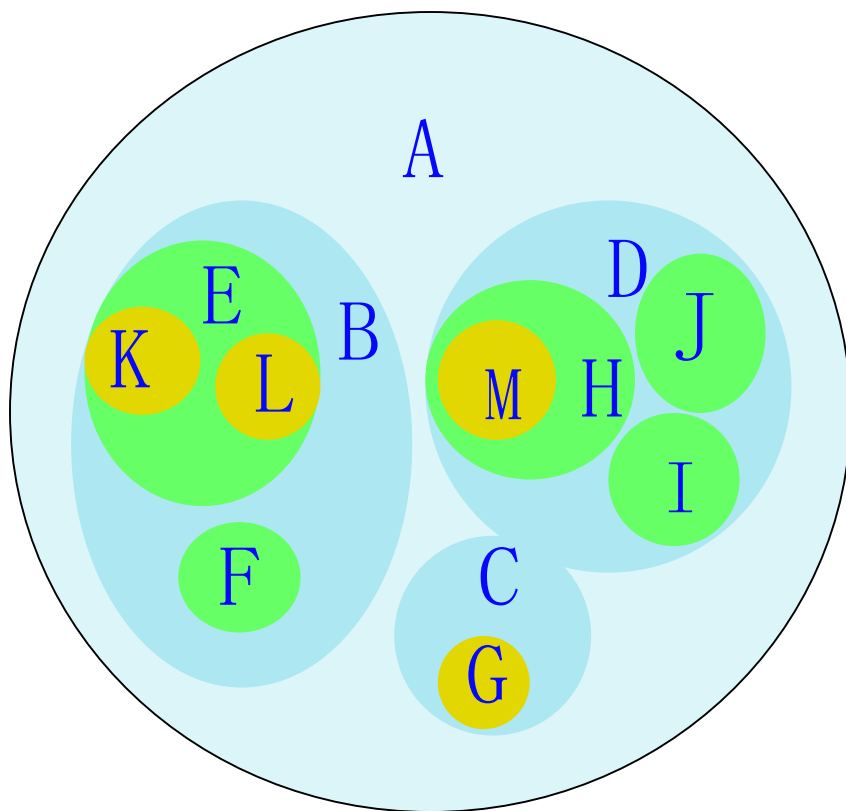




5.1 树的有关概念

- 树的表示:

- (3) 嵌套集合表示法 (文氏图)

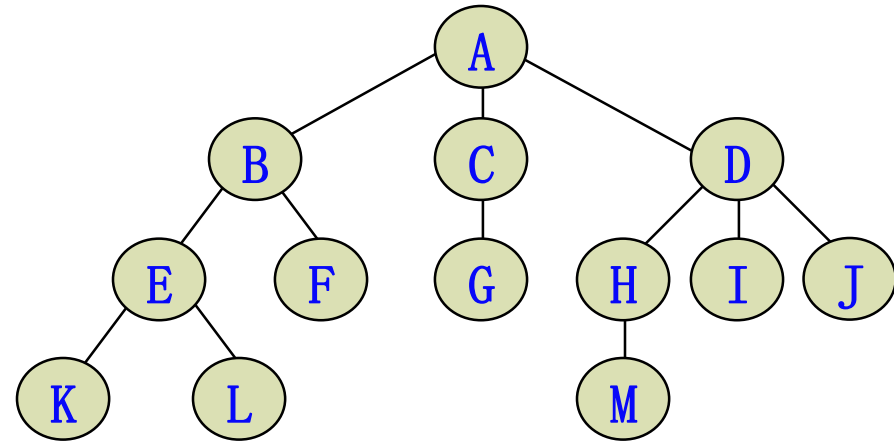




5.1 树的有关概念

- 树的表示:

(4) 广义表表示法



(A) 第一层

(A (B (E, F), C (G), D (H, I, J))) 第三层

(A (B (E (K, L), F), C (G), D (H (M), I, J))) 第四层

表示方法的多样化说明树结构应用的重要性



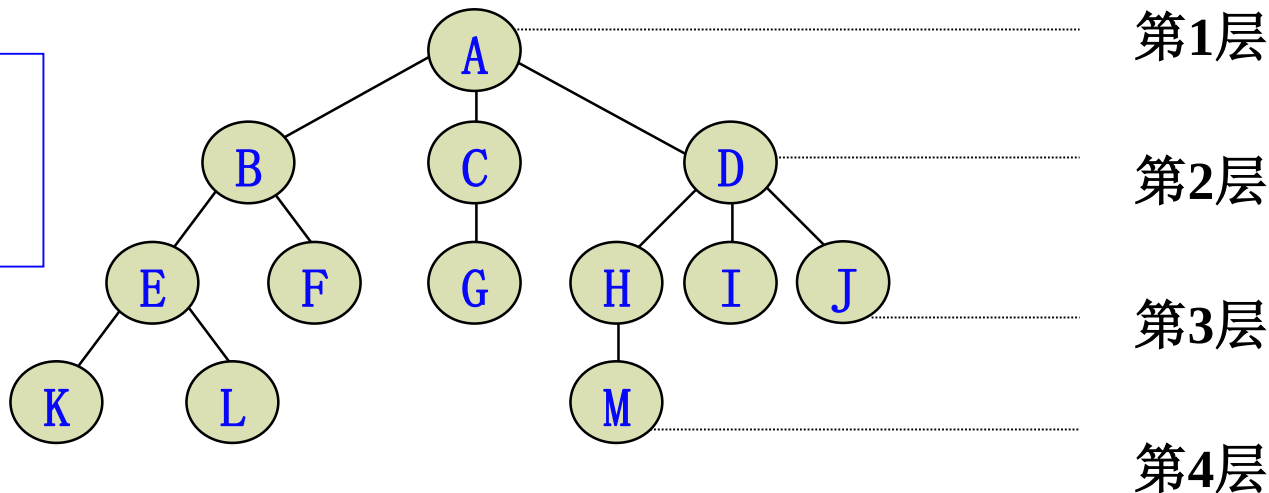
5.1 树的有关概念

• 树的有关术语:

- ✚ **结点层**: 根结点的层定义为1, 其它依此类推;
- ✚ **树的深度**: 树中最大的结点层;
- ✚ **结点的度**: 结点子树的个数;
- ✚ **树的度**: 树中最大的结点度;
- ✚ **叶子结点**: 也叫终端结点, 是度为 0 的结点;

树的度为3

树的深度为4

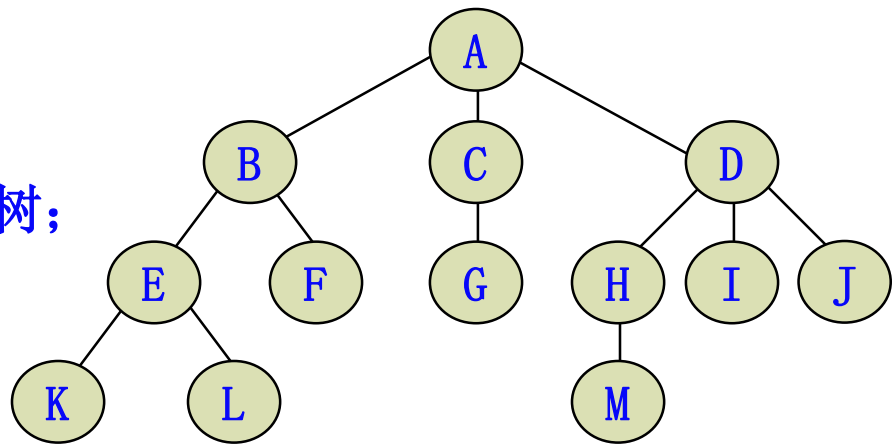




5.1 树的有关概念

• 树的有关术语:

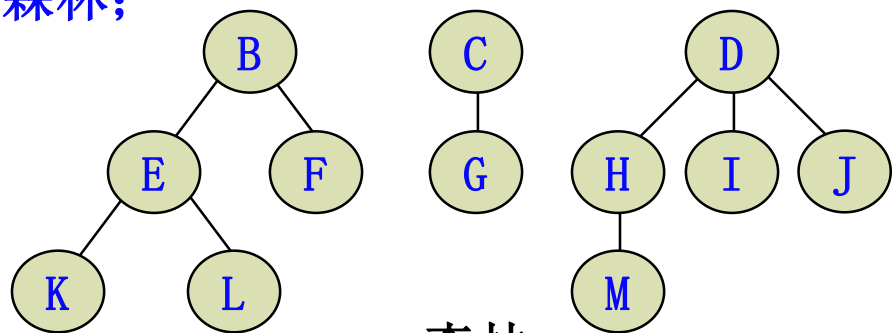
- 分枝结点: 度不为0的结点;
- 有序树: 子树有序的树, 如: 家族树;
- 无序树: 不考虑子树的顺序;
- 森林: 互不相交的树集合。



树

● 树和森林的关系:

- (1) 一棵树去掉根, 其子树构成一个森林;
- (2) 一个森林增加一个根结点成为树。

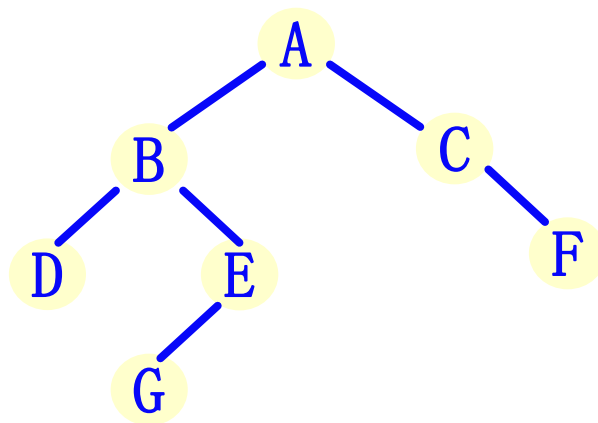


森林



5.2 二叉树

树是一种分枝结构的对象，在树的概念中，对每一个结点孩子的个数没有限制，因此树的形态多种多样，本节我们主要讨论另一种树型结构——**二叉树**。





5.2 二叉树

5.2.1 二叉树的概念

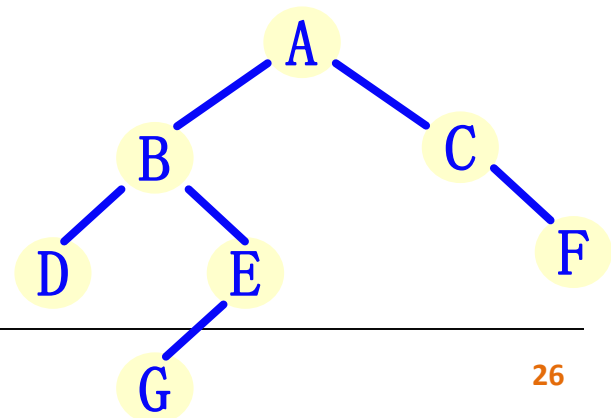
5.2.2 二叉树的性质

5.2.3 二叉树的存储结构



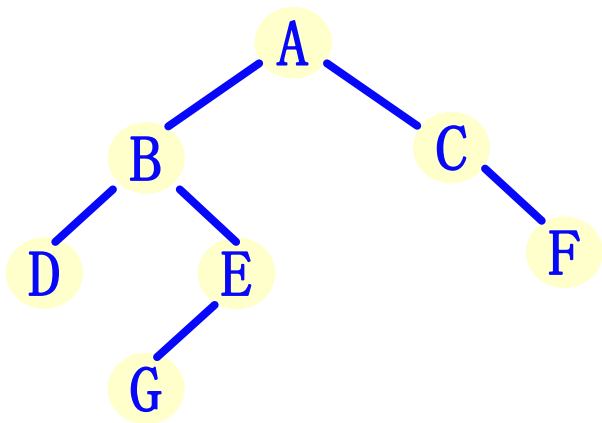
5.2.1 二叉树的概念

- **概念**：二叉树或为空树，或由根及两颗**不相交的左、右子树**构成，并且**左、右子树**本身也是二叉树。
- **特点**：
 - 二叉树中每个结点最多有两棵子树；即**二叉树每个结点的度小于等于2**；
 - 左、右子树不能颠倒——**有序树**；
 - 二叉树是**递归结构**，在二叉树的定义中又用到了二叉树的概念；

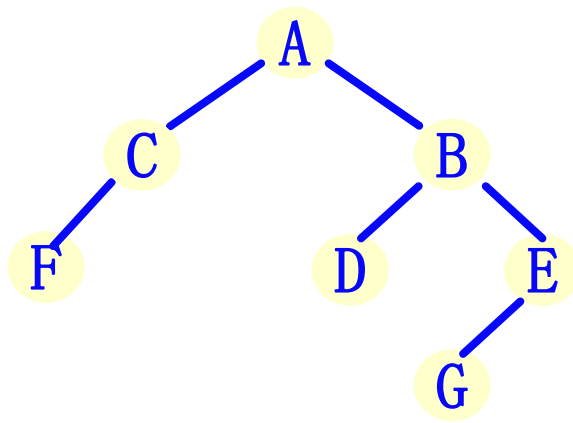




5.2.1 二叉树的概念



(a)



(b)

二叉树是有左右之分的



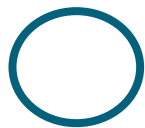


5.2.1 二叉树的概念

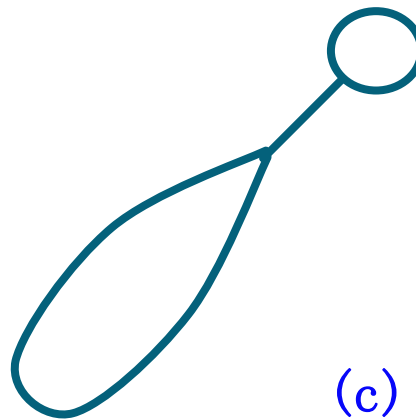
- 二叉树的基本形态

$\varnothing$

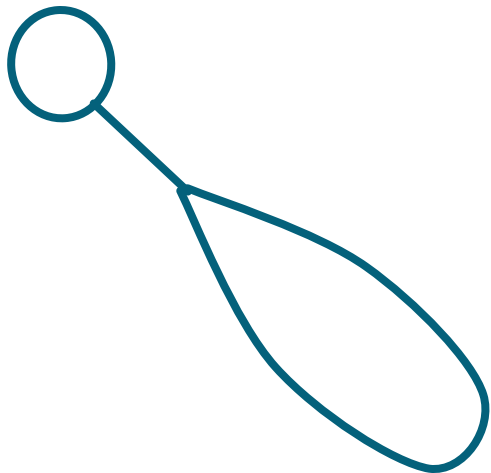
(a) 空树



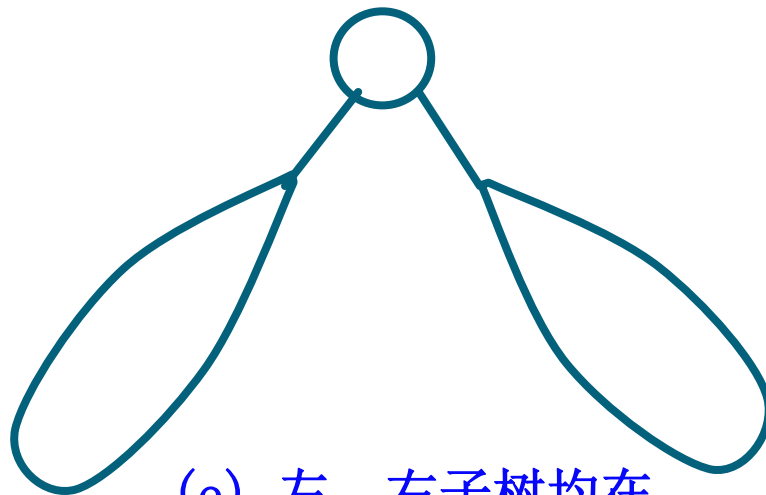
(b) 仅有根



(c) 右子树空



(d) 左子树空



(e) 左、右子树均在



5.2.2 二叉树的性质

性质1 在二叉树的第 i ($i \geq 1$) 层上**至多**有 2^{i-1} 个结点。

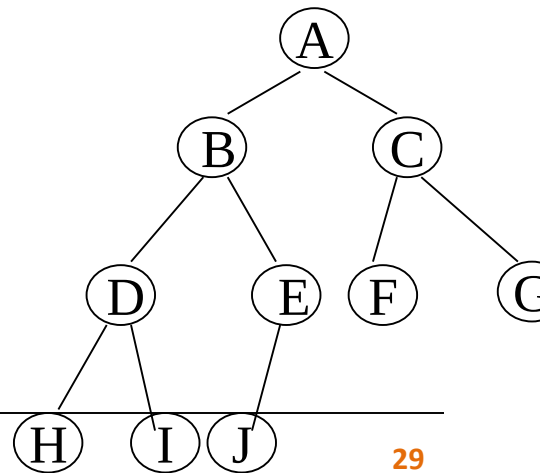
证明：用数学归纳法就可以证明。

性质2 深度为 k 的二叉树**最多**有 $2^k - 1$ 个结点。

证明：最多结点数为各层结点个数相加，即

$$1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1$$

k层二叉树



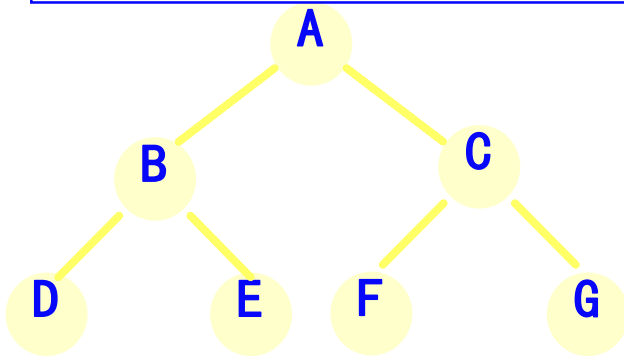


5.2.2 二叉树的性质

• 两种特殊的二叉树

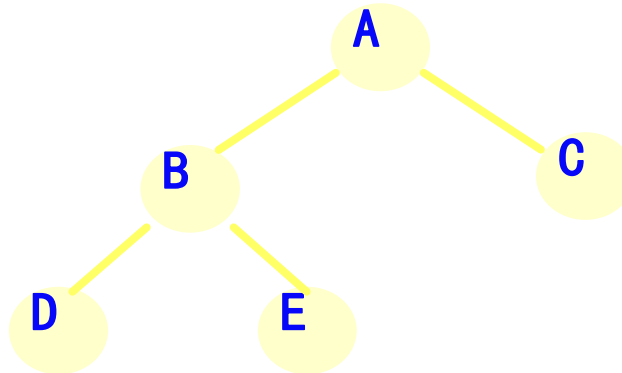
满二叉树：如果深度为 k 的二叉树，有 2^k-1 个结点则称为满二叉树；

完全二叉树：二叉树中所含的 n 个结点和满二叉树中编号为1至 n 的结点一一**对应**。



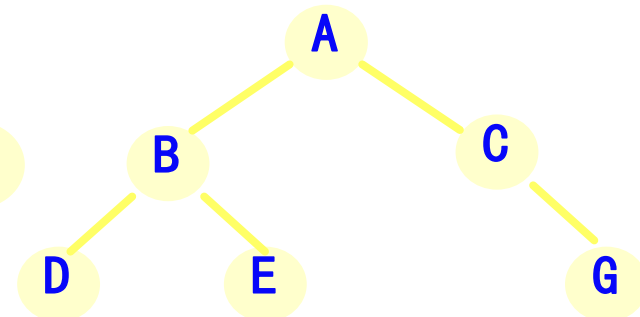
(a)

$K=3$ 的满二叉树



(b)

(b) 完全二叉树



(c)

(c) 不是完全二叉树

结论：满二叉树一定是完全二叉树，反之不一定



5.2.2 二叉树的性质

性质3 具有 n 个结点的**完全二叉树的深度**为 $\lfloor \log_2 n \rfloor + 1$

证明： 设所求完全二叉树的深度为 k

由性质2和完全二叉树的定义知：

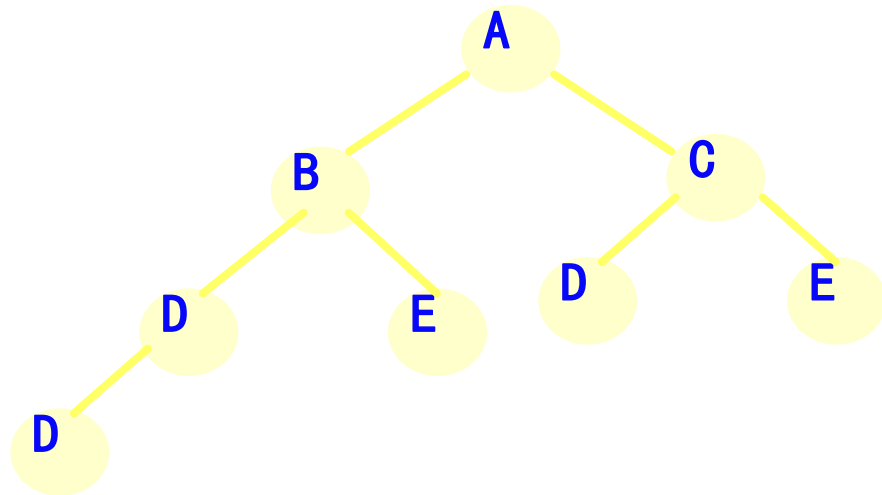
$k-1$ 层的最多结点数 $\longleftarrow 2^{k-1}-1 < n \leq 2^k-1 \longrightarrow k$ 层的最多结点数

由此可以推出： $2^{k-1} \leq n < 2^k$

取对数得： $k-1 \leq \log_2 n < k$

由于 k 为整数，故有 $k-1 = \lfloor \log_2 n \rfloor$

即： $k = \lfloor \log_2 n \rfloor + 1$



性质2:深度为 k 的二叉树最多有 2^k-1 个结点



5.2.2 二叉树的性质

性质4 对任意二叉树T，如果度数为0结点数为 n_0 ，度数为1结点数为 n_1 ，度数为2结点数为 n_2 ，则 $n_0 = n_2 + 1$ 。

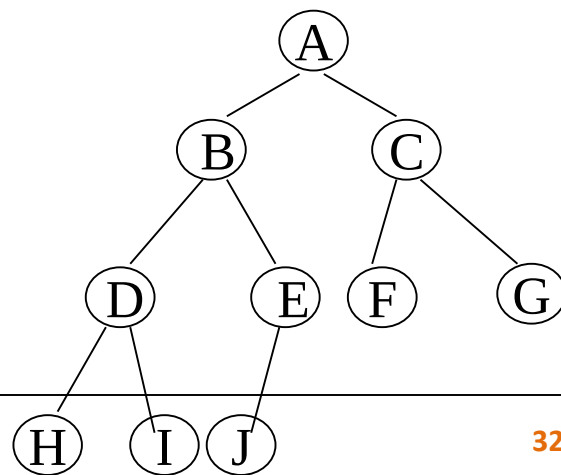
证明：二叉树T的**结点总数** $n = n_0 + n_1 + n_2$ (1)

二叉树的**分支结点总数** $b = n - 1$ (2)

由于分支结点是由度为1和度为2的结点生成的

注意： n_1 生成 $n_1 * 1$ 个节点， n_2 生成 $n_2 * 2$ 个节点，
即**分支结点总数** $b = n_1 + 2 * n_2$ (3)
根结点不由任何结点产生

由(1) (2) (3)得 求得： $n_0 = n_2 + 1$

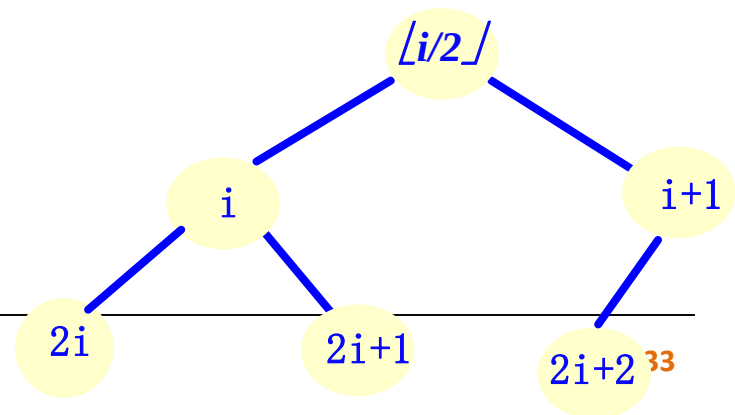




5.2.2 二叉树的性质

性质5: 若对含 n 个结点的完全二叉树从上到下且从左至右进行 1 至 n 的编号，则对完全二叉树中任意一个编号为 i 的结点：

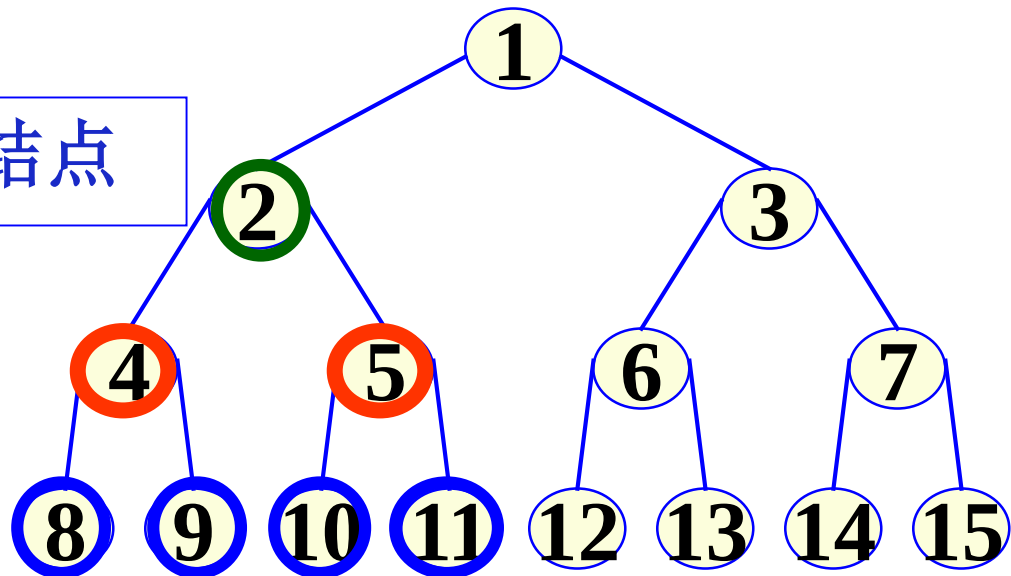
- (1) 若 $i=1$ ，则该结点是二叉树的根，无双亲，否则，编号为 $\lfloor i/2 \rfloor$ 的结点为其双亲结点；
- (2) 若 $2i > n$ ，则该结点无左孩子，否则，编号为 $2i$ 的结点为其左孩子结点；
- (3) 若 $2i+1 > n$ ，则该结点无右孩子结点，否则，编号为 $2i+1$ 的结点为其右孩子结点。





5.2.2 二叉树的性质

$i=1$ 只有根结点



编号 $i=4$
双亲为 $\lfloor i/2 \rfloor = 2$
左子树为 $2i=8$
右子树为 $2i+1=9$

编号 $i=5$
双亲为 $\lfloor i/2 \rfloor = 2$
左子树为 $2i=10$
右子树为 $2i+1=11$

$i=8, n=15$
 $2i > n$
无左子树

通过性质5把非线性结构转化成了线性结构



5.2.3 二叉树的存储结构

- 二叉树的顺序存储表示
- 二叉树的链式存储表示





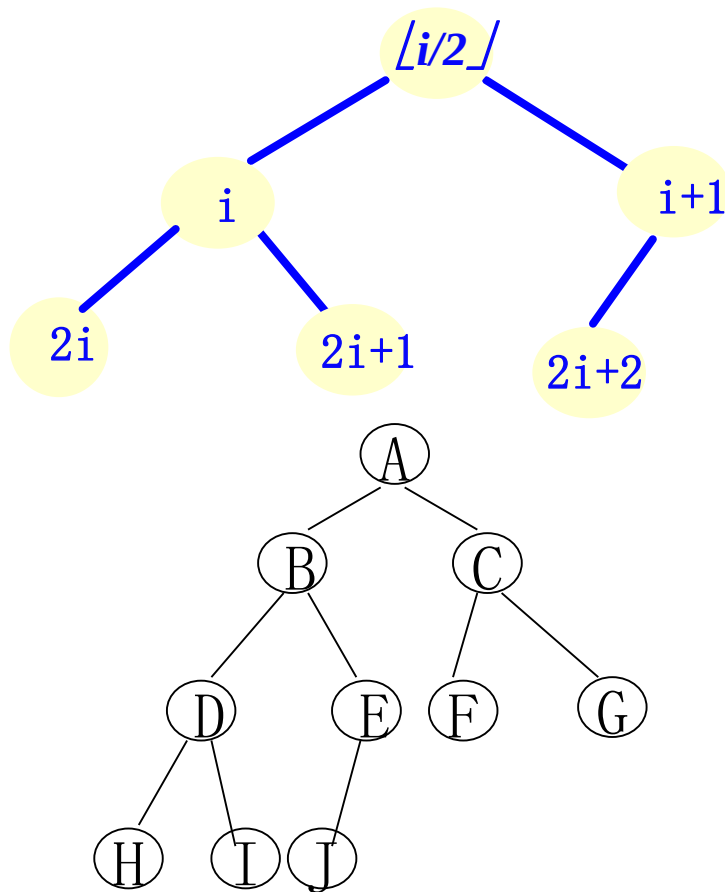
5.2.3 二叉树的存储结构

• 二叉树的顺序存储表示

(1) 完全（或满）二叉树

采用一维数组，按层序顺序依次存储二叉树的每一个结点。
如下图所示：

A	B	C	D	E	F	G	H	I	J
1	2	3	4	5	6	7	8	9	10



利用性质5实现线性结构和非线性结构的灵活转换。

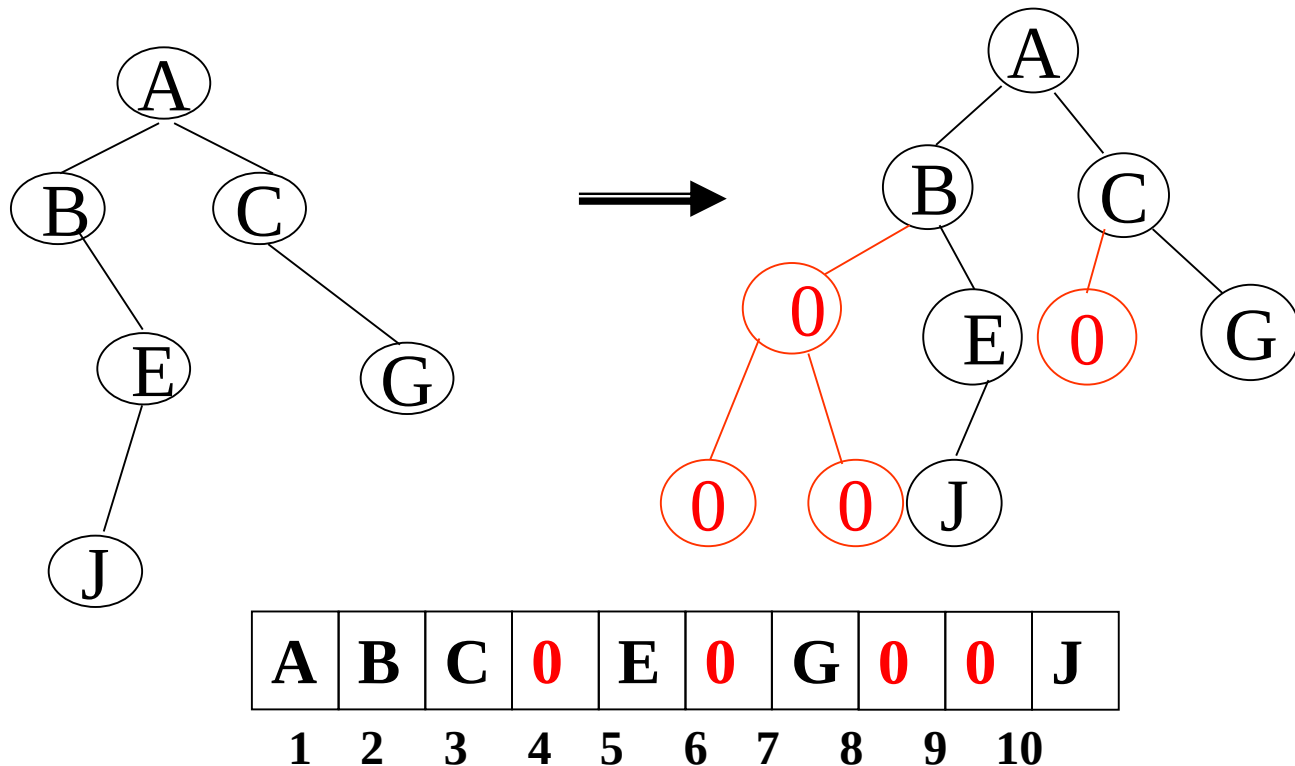


5.2.3 二叉树的存储结构

- 二叉树的顺序存储表示

(2) 一般二叉树

通过**虚设**部分结点，使其变成相应的**完全二叉树**。

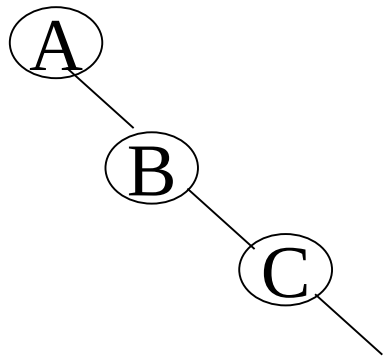




5.2.3 二叉树的存储结构

- 二叉树的顺序存储表示

(3)特殊的二叉树



J

说明：顺序存储方式对于畸形二叉树，浪费较大空间



5.2.3 二叉树的存储结构

- 二叉树的链式存储表示

二叉链表存储:

二叉链表中每个结点包含三个域: 数据域、左指针域、右指针域

lch	data	rch
-----	------	-----

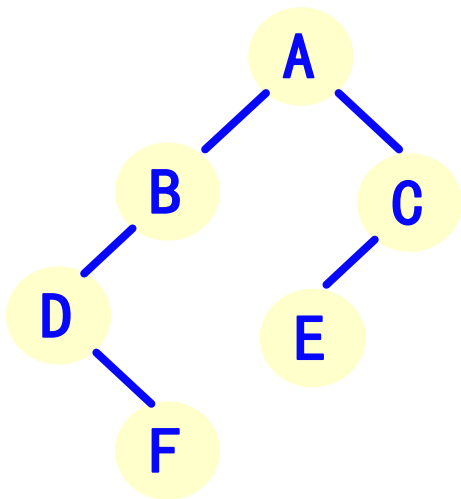
C 语言的类型描述如下:

```
typedef Struct node
{
    DataType data;
    Struct node *lch, *rch;
} BinTNode;
```

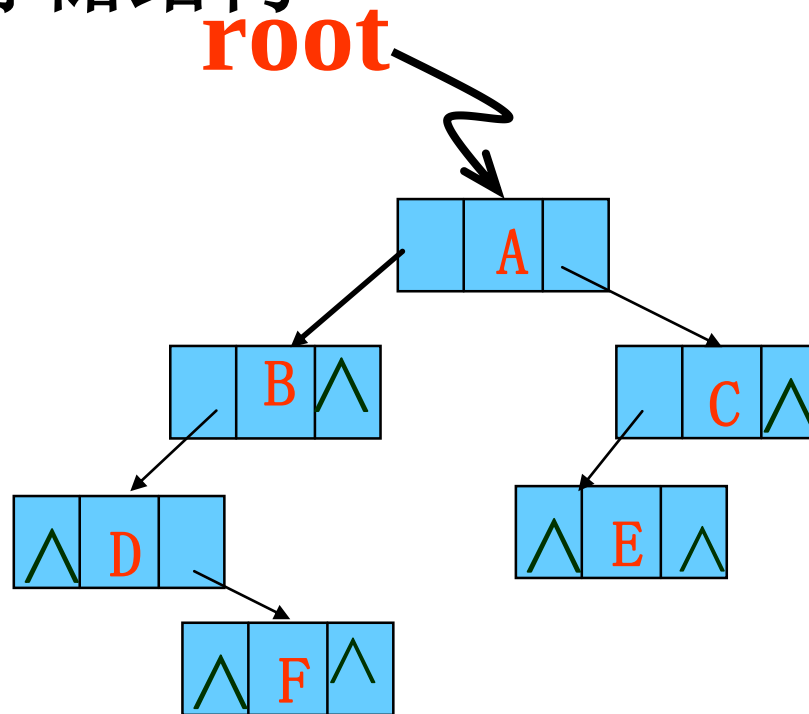


5.2.3 二叉树的存储结构

- 二叉树的链式存储表示



二叉树



二叉链表图示

n 个结点的二叉树中，
有多少个空链接域？

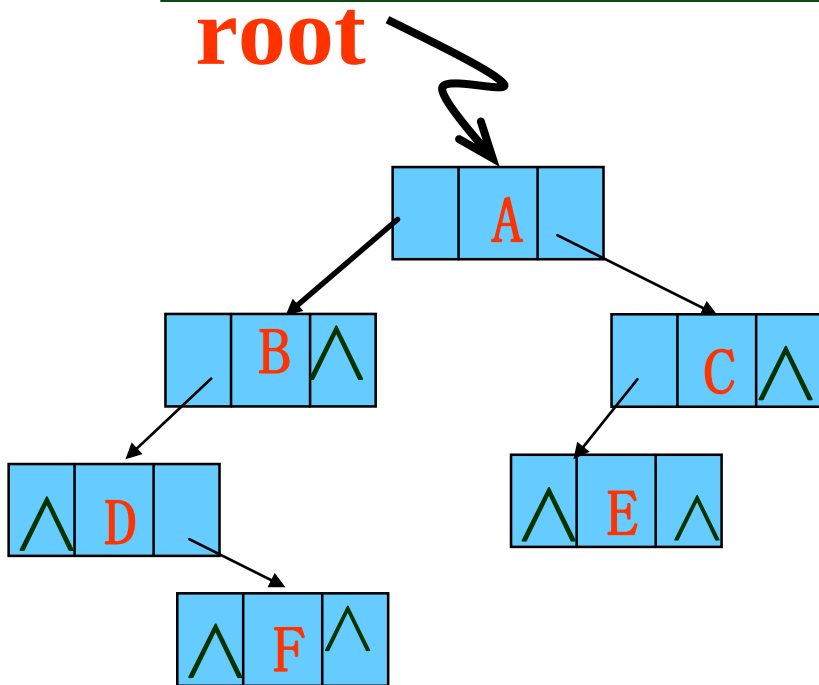


5.2.3 二叉树的存储结构

• 二叉树的链式存储表示

性质6: n 个结点的二叉树中，共有 $n+1$ 个空指针域。

root



二叉链表图示

证: n 个结点总的指针域数 $2n$

除了根结点外，其余 $n-1$ 个结点
都是由指针域指出的结点；

所以，剩余的结点数即
空指针域个数为：

$$2n - (n-1) = n+1$$

二叉链表的缺点是很难找到结点的双亲



5.2.3 二叉树的存储结构

- 二叉树的链式存储表示---三叉链表

三叉链表（带双亲指针的二叉链表）：三叉链表中每个结点包含四个域：数据域、左指针域、右指针域、双亲指针域

结点结构：

lch	data	rch	parent
-----	------	-----	--------

C 语言的类型描述如下：

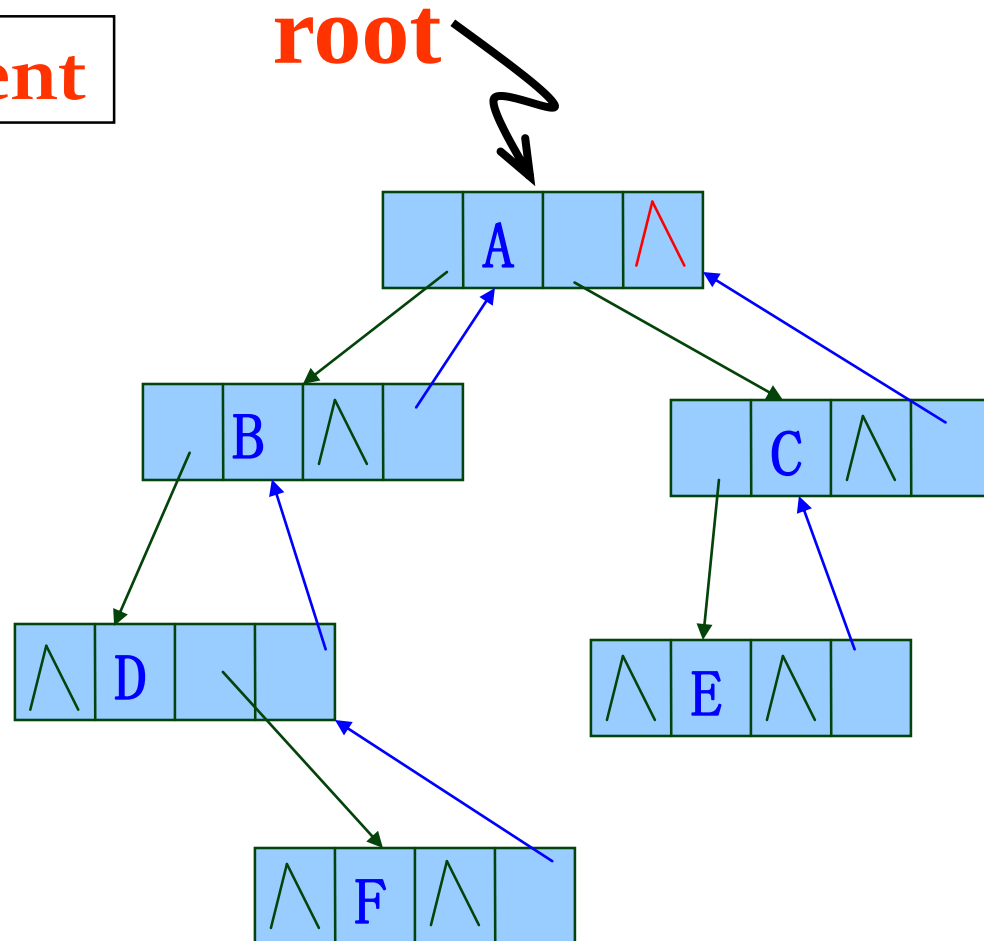
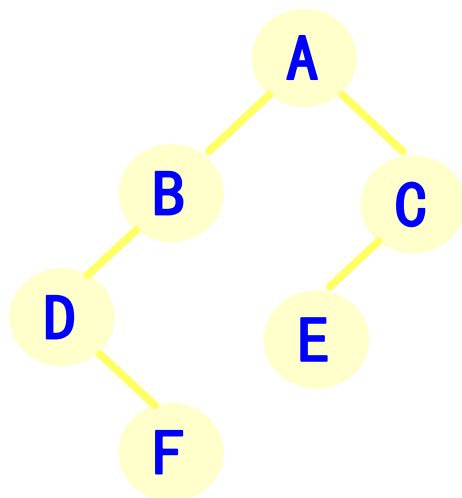
```
typedef Struct node
{
    DataType data;
    Struct node *lch, *rch, *parent;
};
```



5.2.3 二叉树的存储结构

- 二叉树的链式存储表示---**三叉链表**

lch	data	rch	parent
-----	------	-----	--------





5.3 二叉树的遍历

5.3.1 二叉树的遍历方法

5.3.2 遍历的递归算法

5.3.3 遍历的非递归算法





5.3.1 二叉树的遍历方法

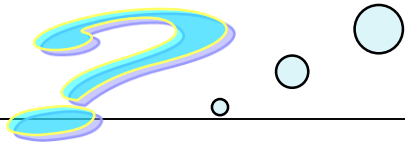
- **遍历：**
 - 按某种搜索路径**访问**二叉树的每个结点，而且每个结点仅被访问一次。
- **访问：**
 - 访问是指对结点进行各种操作的简称，包括输出、查找、修改等等操作。
- **遍历**是各种数据结构最基本的操作，许多其它的操作可以在遍历基础上实现。



5.3.1 二叉树的遍历方法

- “遍历” 是任何类型均有的操作：
 - 线性结构的遍历：只有一条搜索路径 (因为每个结点均只有一个后继)；
 - 非线性结构的遍历：二叉树是非线性结构，则存在如何遍历；即按什么样的搜索路径遍历的问题。

如何访问二叉树的每个结点，
而且每个结点仅被访问一次？





5.3.1 二叉树的遍历方法

- 对“二叉树”而言，可以有三条搜索路径：
 - 先上后下的按层次遍历；
 - 先左（子树）后右（子树）的遍历；
 - 先右（子树）后左（子树）的遍历。



5.3.1 二叉树的遍历方法

二叉树由根、左子树、右子树三部分组成

二叉树的遍历可以分解为：访问根，遍历左子树和遍历右子树

令：L：遍历左子树

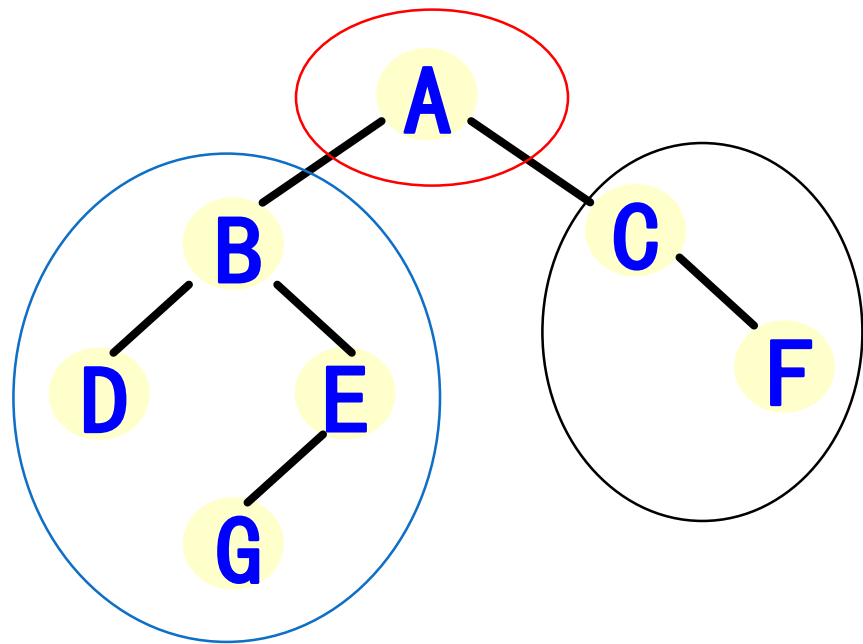
T：访问根结点

R：遍历右子树

有六种遍历方法：

T L R, L T R, L R T,

T R L, R T L, R L T



约定先左后右, 有三种遍历方法：T L R、L T R、L R T，分别称为先序（先根）遍历、中序（中根）遍历、后序（后根）遍历

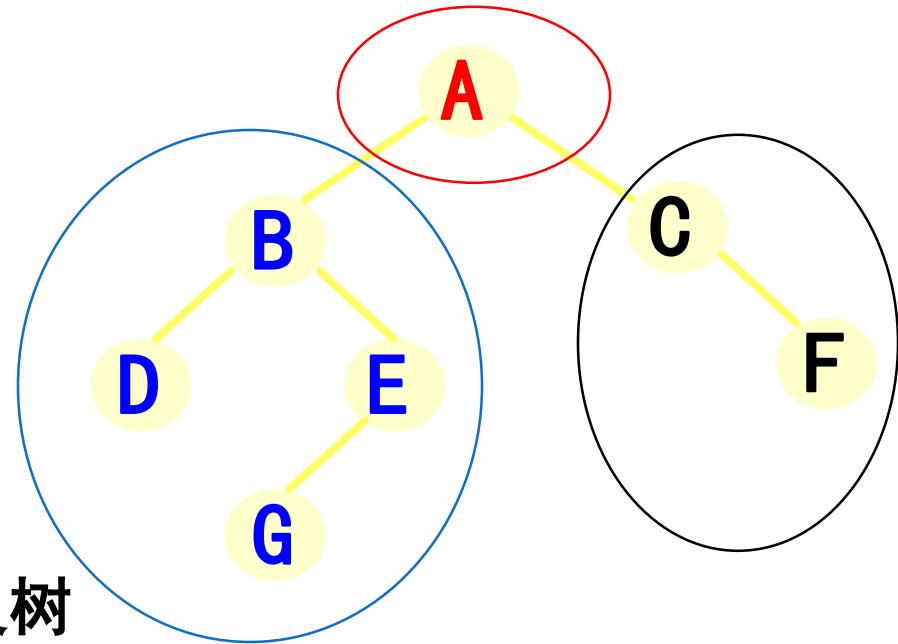


5.3.1 二叉树的遍历方法

- 中序遍历 (L T R)

若二叉树非空

- (1) 中序遍历左子树
- (2) 访问根结点
- (3) 中序遍历右子树



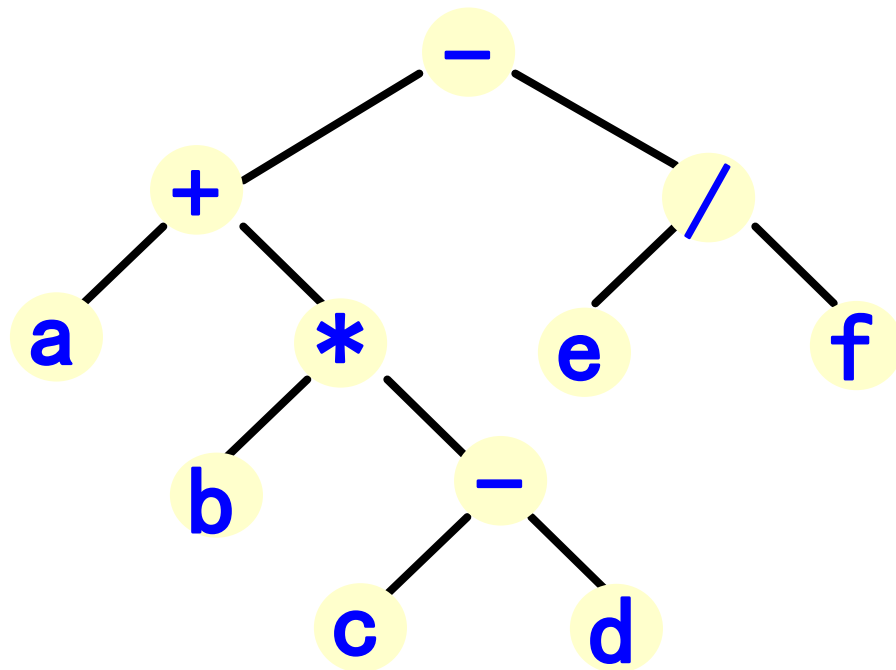
例 中序遍历右图所示的二叉树

中序遍历序列: D ,B ,G ,E ,A ,C ,F



5.3.1 二叉树的遍历方法

练习 表达式 $a+b*(c-d)-e/f$ 用二叉树表示如下：



中序 $a+b*c-d-e/f$

--- 中缀表示

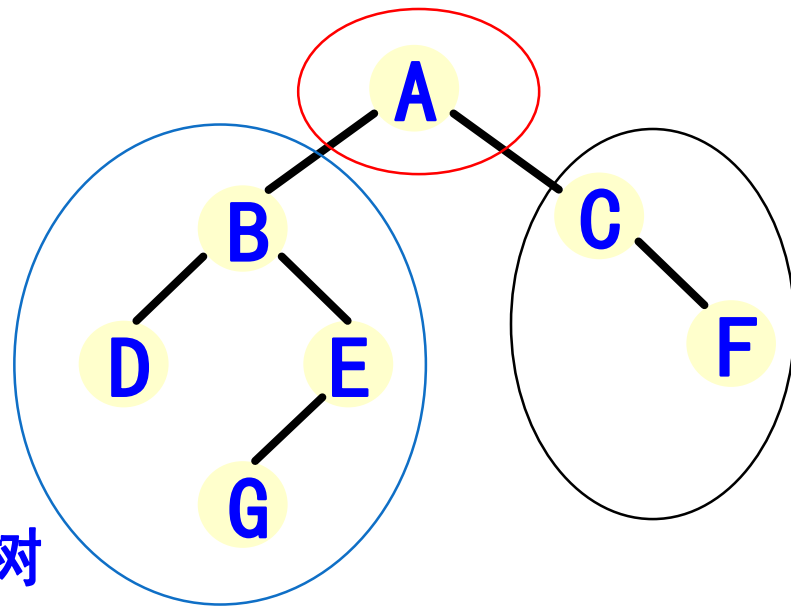


5.3.1 二叉树的遍历方法

• 先序遍历 (T L R)

若二叉树非空

- (1) 访问根结点;
- (2) 先序遍历左子树;
- (3) 先序遍历右子树;



例

先序遍历右图所示的二叉树

- (1) 访问根结点A
- (2) 先序遍历左子树：即按 T L R 的顺序遍历左子树
- (3) 先序遍历右子树：即按 T L R 的顺序遍历右子树

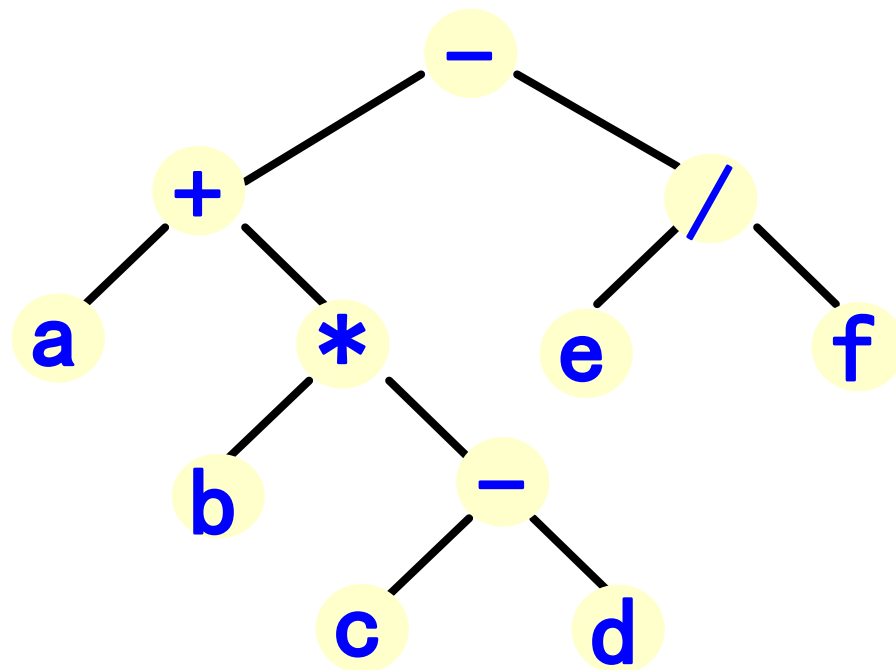
先序遍历序列：A, B, D, E, G, C, F

图示



5.3.1 二叉树的遍历方法

练习 先序遍历下图所示的二叉树



先序 $- + a * b - c d / e f$

前缀表示 (波兰式)

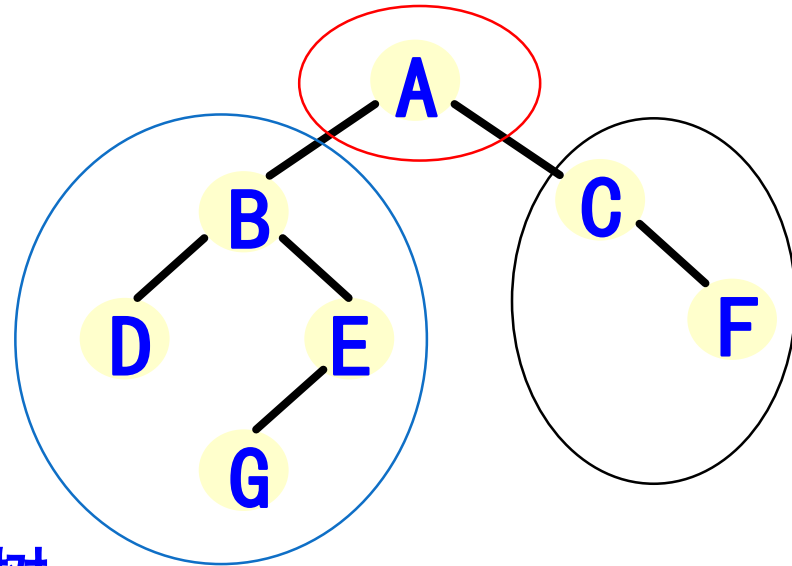


5.3.1 二叉树的遍历方法

• 后序遍历 (L T R)

若二叉树非空

- (1) 后序遍历左子树
- (2) 后序遍历右子树
- (3) 访问根结点



例 后序遍历右图所示的二叉树

- (1) 后序遍历左子树：即按 L R T 的顺序遍历左子树
- (2) 后序遍历右子树：即按 L R T 的顺序遍历右子树
- (3) 访问根结点A

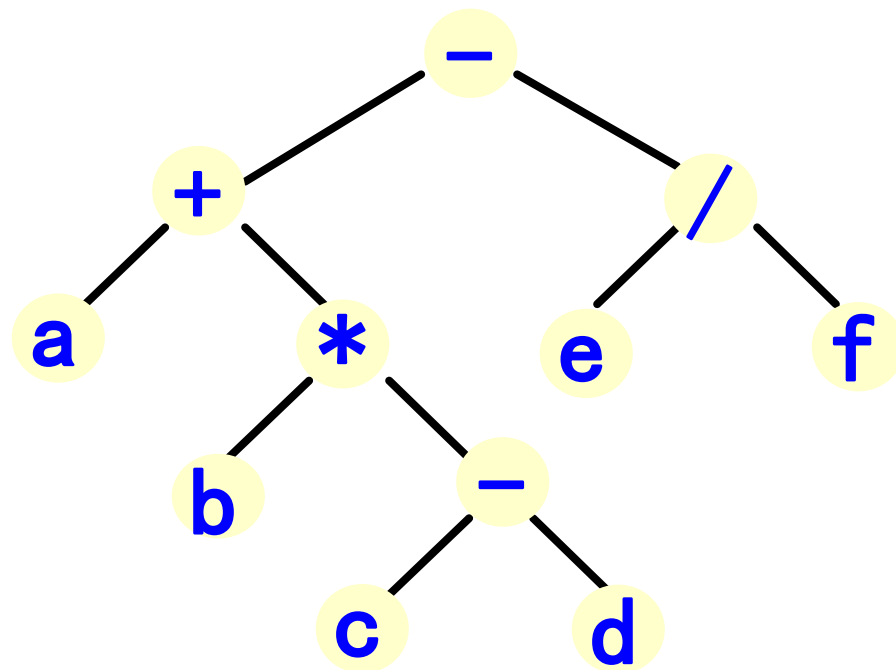
后序遍历序列： D, G, E, B, F, C, A





5.3.1 二叉树的遍历方法

练习 后序遍历下图所示的二叉树



后序 a b c d - * + e f / -

后缀表示 (逆波兰式)

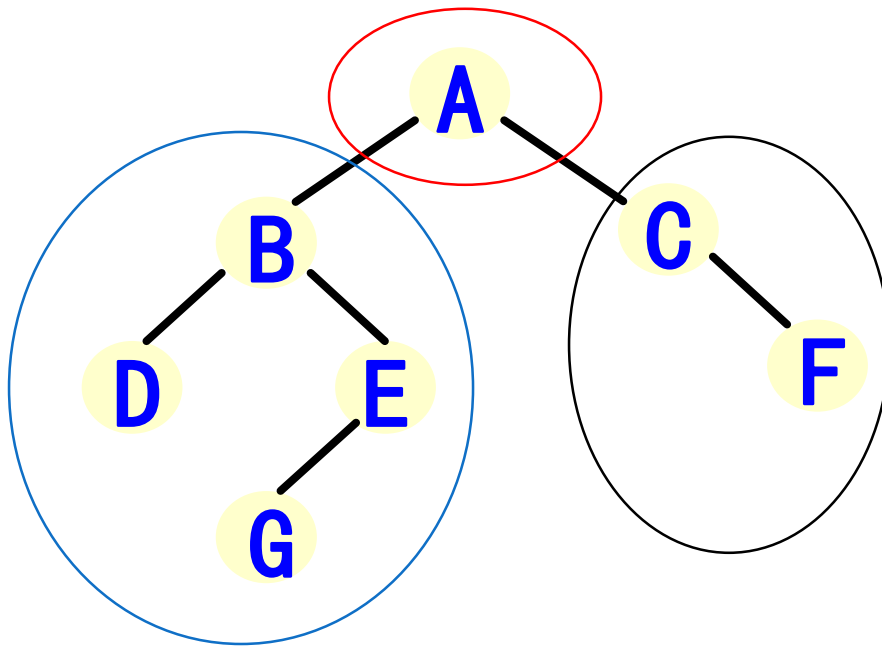


5.3.1 二叉树的遍历方法

- 按层遍历

层次遍历序列:

A, B, C, D, E, F, G





5.3.1 二叉树的遍历方法

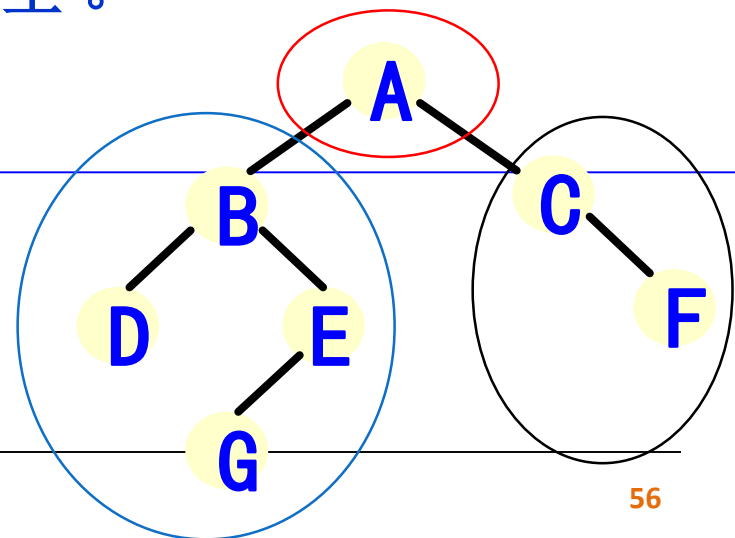
• 按层遍历

按层遍历引入了**队列**作为辅助工具。

算法思想为：

- (1) 将二叉树根入队列；
- (2) 将队头元素出队列，并判断此元素是否有左右孩子，若有，则将其左右孩子入列，否则转 (3)；
- (3) 重复步骤 (2)，直到队列为空。

课后思考：按层遍历算法





5.3 二叉树的遍历

5.3.1 二叉树的遍历方法

5.3.2 遍历的递归算法

5.3.3 遍历的非递归算法





5.3.2 遍历的递归算法

- 先序遍历 (TLR) 的定义

若二叉树非空

- (1) 访问根结点;
- (2) 先序遍历左子树
- (3) 先序遍历右子树;

上面先序遍历的定义等价于:

若二叉树为空, 结束 —— 基本项 (也叫终止项)

若二叉树非空 —— 递归项

- (1) 访问根结点;
- (2) 先序遍历左子树;
- (3) 先序遍历右子树;



5.3.2 遍历的递归算法

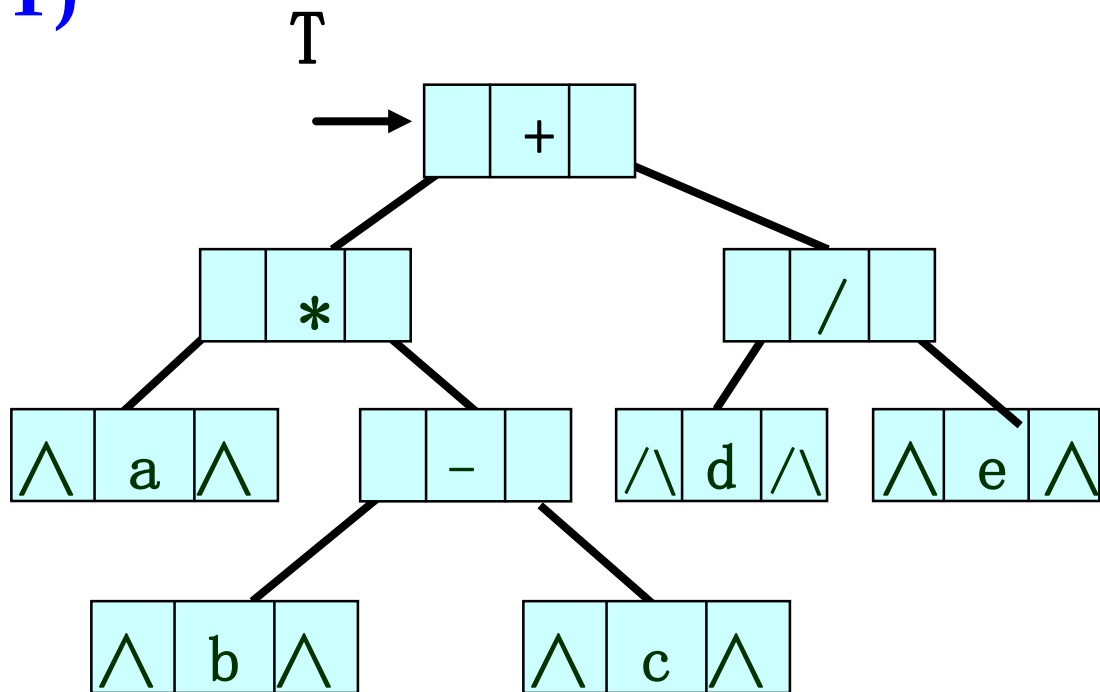
- 先序遍历递归算法

```
void prev (BinTree T)
{ if (T)
  { visit(T->data);
    prev(T->lch);
    prev(T->rch);
  }
}
```

先序序列为

$+ * a - b c / d e$

称为前缀表达式



$$a * (b - c) + d / e$$



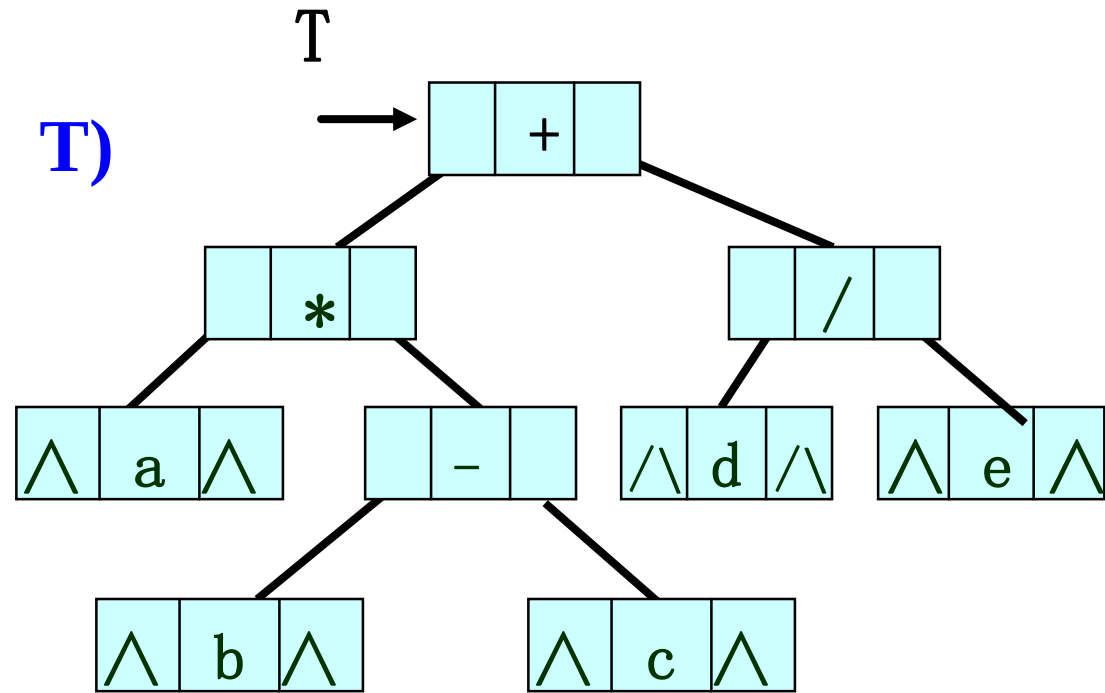
5.3.2 遍历的递归算法

• 中序遍历递归算法

```

void Mid (BinTree T)
{ if (T)
  { Mid(T->lch);
    visit( T->data);
    Mid(T->rch);
  }
}

```



中序序列为

$a * b - c + d / e$

称为中缀表达式

$a * (b - c) + d / e$

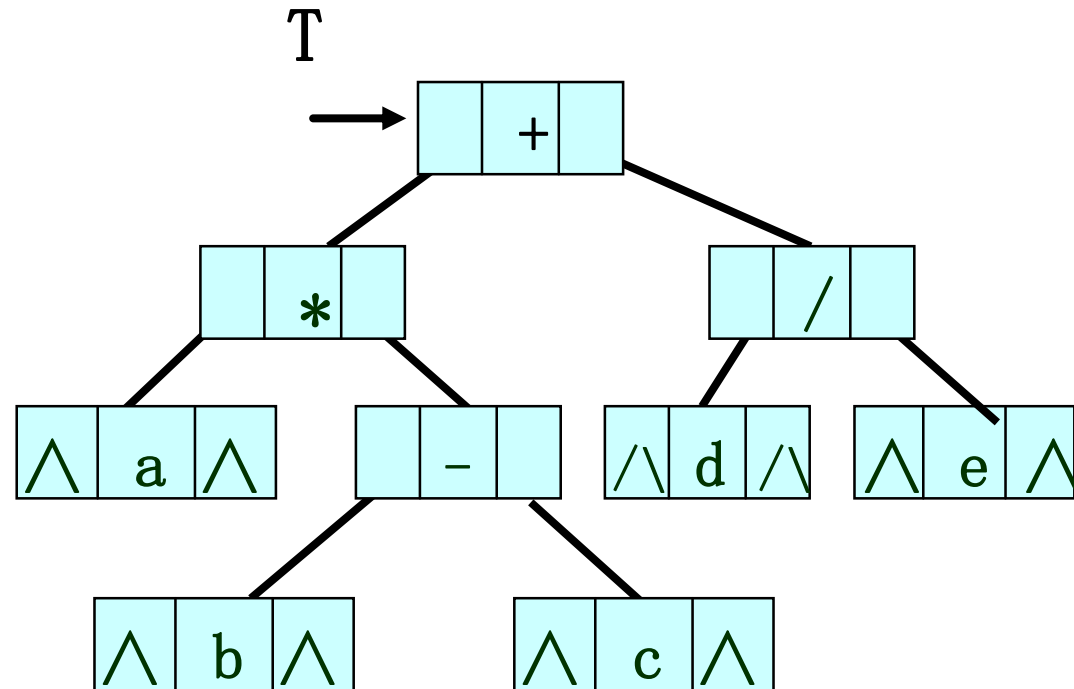


5.3.2 遍历的递归算法

- 后序遍历递归算法

```
void Post(BinTree T)
{ if (T)
  { Post(T->lch);
    Post(T->rch);
    visit(T->data);
  }
}
```

后序序列为 **a b c - * d e / +**
称为**后缀表达式**



$a * (b - c) + d / e$



5.3 二叉树的遍历

5.3.1 二叉树的遍历方法

5.3.2 遍历的递归算法

5.3.3 遍历的非递归算法





5.3.3 遍历的非递归算法

- 中序遍历的非递归算法

BiTNode *GoFarLeft(BiTree T, Stack *S)

{//找到左子树的最左下的结点

if (!T) return NULL;

while (T->lch) {

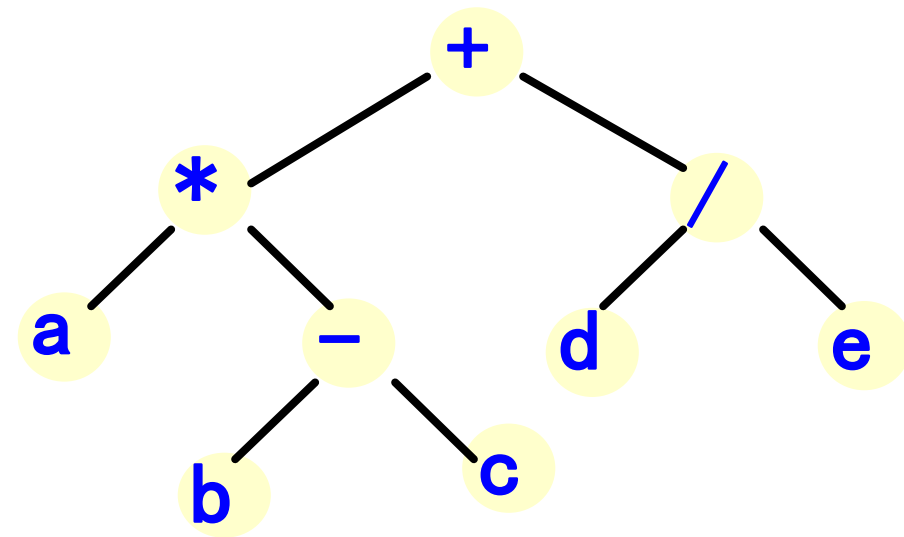
Push(S, T);

T = T->lch;

}

return T;

}



中序序列为: $a * b - c + d / e$



5.3.3 遍历的非递归算法

- 中序遍历的非递归算法

```
void Inorder_I(BiTree T)
{
    Stack *S;
    t = GoFarLeft(T, S); // 找到最左下的结点
    while(t) {
        visit(t->data);
        if (t->rch)
            t = GoFarLeft(t->rch, S); // 从右子树根结点出发, 继续往左走
        else if ( !StackEmpty(S) ) // 栈不空时退栈
            t = Pop(S);
        else t = NULL; // 栈空表明遍历结束
    } // while
} // Inorder_I
```



5.4 遍历的应用

遍历是二叉树各种操作的基础，可以在遍历过程中对结点进行各种操作：

- (1) 求结点的双亲、孩子结点、结点的层次；
- (2) 求孩子结点；
- (3) 求结点的层次；
- (4) 遍历过程中生成结点，建立二叉树；

遍历二叉树的过程实质是把二叉树的结点进行**线性排列**的过程。



5.4 遍历的应用

5.4.1 遍历的基本应用

5.4.2 二叉树的遍历与存储结构的应用

5.4.3 二叉树的相似与等价



5.4 遍历的应用

5.4.1 遍历的基本应用

5.4.2 二叉树的遍历与存储结构的应用

5.4.3 二叉树的相似与等价



5.4.1 遍历的基本应用

- 二叉树的生成

递归建立二叉树

我们按**先序**递归遍历的思想来建立二叉树。

其建立思想如下：

- (1) 建立二叉树的**根**结点；
- (2) 先序建立二叉树的**左子树**；
- (3) 先序建立二叉树的**右子树**。



5.4.1 遍历的基本应用

- 二叉树的生成

二叉树的生成的递归算法

```
bitree *creat()
{
    bitree *t;
    t=(bitree*)malloc(sizeof(bitree));
    t->data=x;
    t->lch=creat();
    t->rch=creat();
    return t;
}
```



5.4.1 遍历的基本应用

□ 求二叉树的叶子数。

算法思想：采用任何遍历方法，遍历时判断访问的结点是不是叶子，若是则叶子数加1。

```
int countleaf(bitree t,int num)
{ if(t!=NULL)
  {if((t->lch==NULL) &&(t->rch==NULL))
    num++;
   num=countleaf(t->lch,num);
   num=countleaf(t->rch,num);
  }
return num;
}
```



5.4.1 遍历的基本应用

□ 求二叉树的深度

算法思想：从第一层的根结点开始往下递推可得到。

求二叉树深度的递归算法(后序遍历)

```
Int treedepth(bitree *t)
{int h,lh,rh;
  if(t==NULL)  h=0;
  else  { lh=treedepth(t->lch);
         rh=treedepth(t->rch);
         if(lh>=rh) h=lh+1;
         else  h=rh+1; }
  return h;
}
```



5.4 遍历的应用

5.4.1 遍历的基本应用

5.4.2 二叉树的遍历与存储结构的应用

5.4.3 二叉树的相似与等价

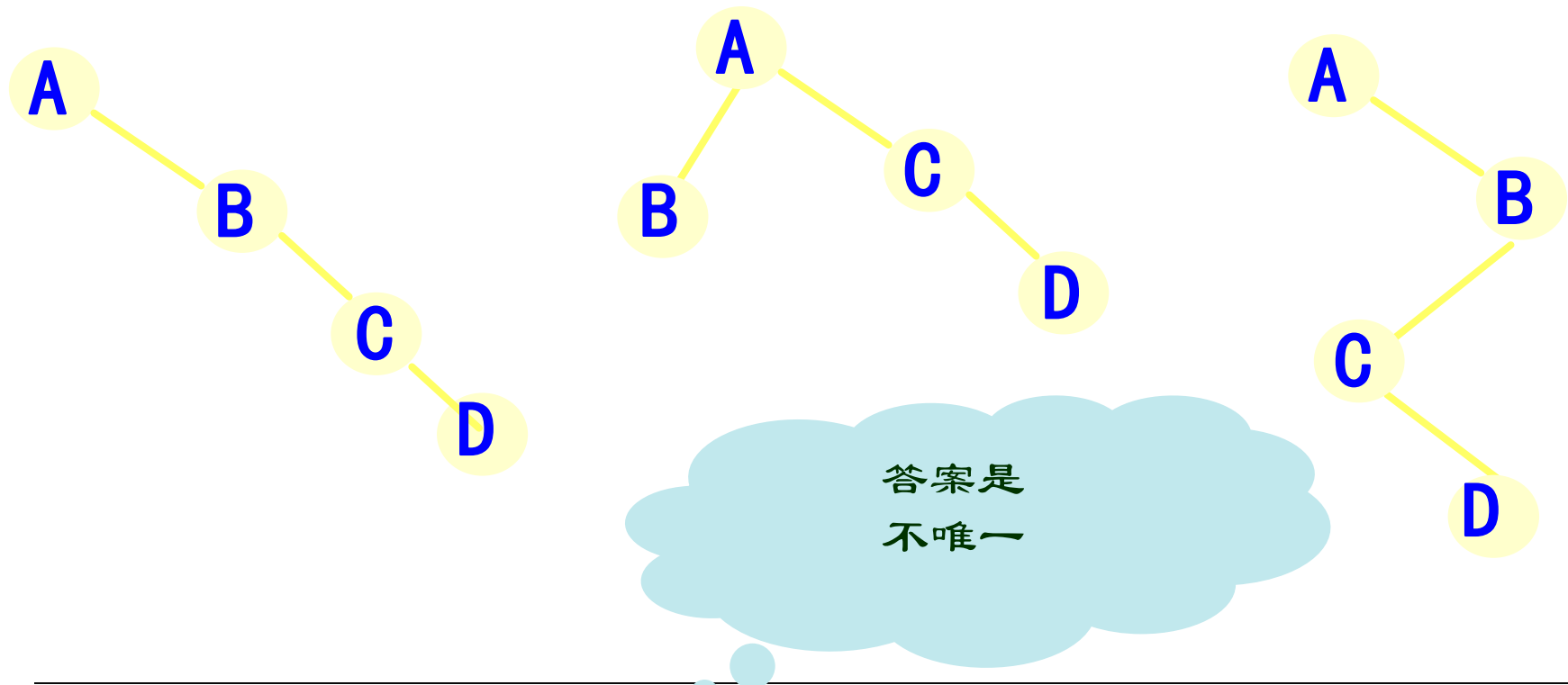


5.4.2 二叉树的遍历与存储结构的应用

□ 二叉树的遍历与存储结构之间的转化

问题： 给定一个遍历序列，能否唯一确定一棵二叉树？

例如：先序序列为ABCD, 其二叉树的结构是什么？





5.4.2 二叉树的遍历与存储结构的应用

□ 构造二叉树

关键 (1) 确定二叉树的根结点;
(2) 结点的左右次序。

给定某两种遍历序列能否唯一确定一棵二叉树?

给定中序和后序 $\longrightarrow$ 唯一确定一棵二叉树

给定中序和前序 $\longrightarrow$ 唯一确定一棵二叉树

给定先序和后序 **不能** 唯一确定一棵二叉树



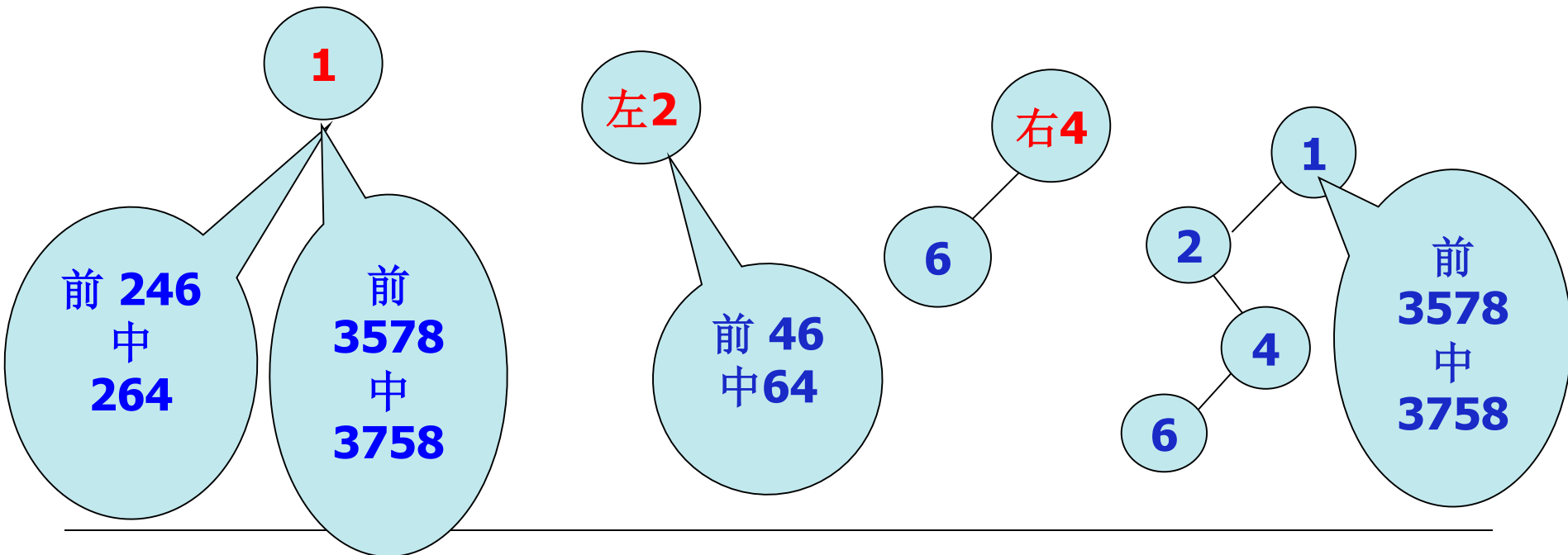
5.4.2 二叉树的遍历与存储结构的应用

□ 构造二叉树

例 给定二叉树先序和中序遍历序列，如何构造二叉树？

先序：1 2 4 6 3 5 7 8

中序：2 6 4 1 3 7 5 8





5.4.2 二叉树的遍历与存储结构的应用

结论:

- “遍历”是二叉树各种操作的基础;
- 可以在遍历过程中对结点进行各种操作,
 - 对于一棵已知树可求结点的双亲;
 - 求结点的孩子结点;
 - 判定结点所在层次;
 - 树的深度;
 - 生成二叉树
 -



5.4 遍历的应用

5.4.1 遍历的基本应用

5.4.2 二叉树的遍历与存储结构的应用

5.4.3 二叉树的相似与等价



5.4.2 二叉树的相似与等价

- 二叉树的相似与等价的含义

两株二叉树具有**相同结构**指：

“形状”相同

(1) 它们都是空的；

(2) 它们都是非空的，且左右子树分别具有相同结构。

■ 定义具有相同结构的二叉树为**相似**二叉树。

■ 相似且相应结点包含相同信息的二叉树称为

等价二叉树。



5.4.2 二叉树的相似与等价

- 判断两株二叉树是否等价

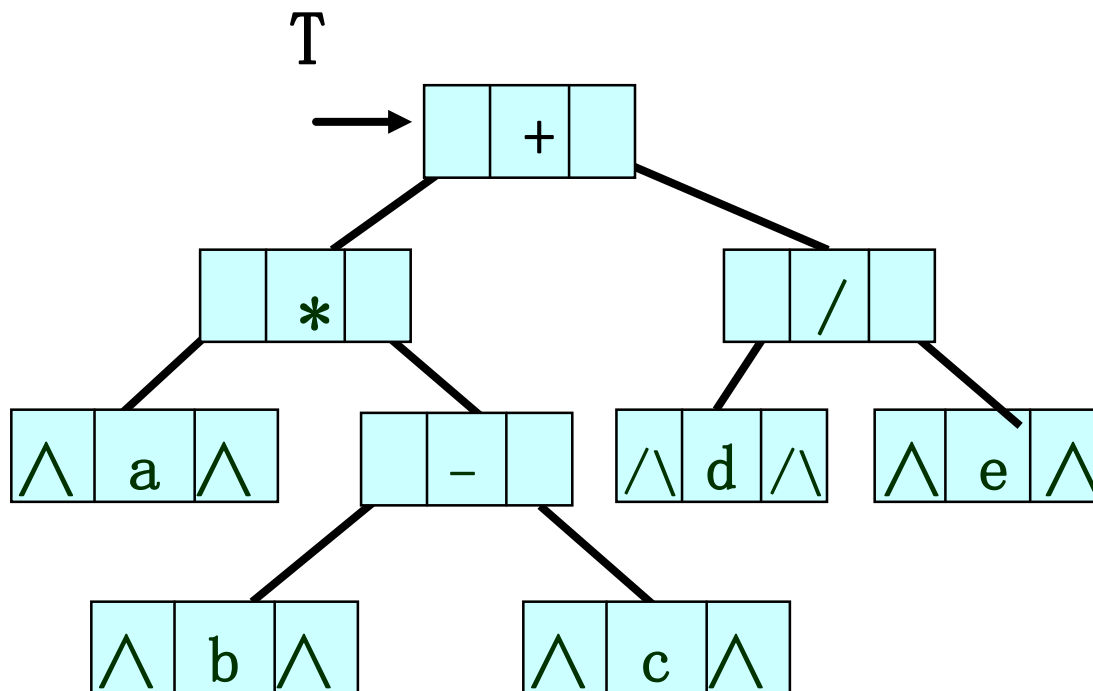
```
int EQUAL( t1 , t2 )
BTREE t1 , t2 ;
{ int x ;
  x = 0 ;
  if ( ISEMPY(t1) && ISEMPY(t2) )//二叉树空
    x = 1 ;
  else if ( !ISEMPY( t1 ) && ! ISEMPY( t2 ) ) //二叉树不空
    if ( DATA( t1 ) == DATA( t2 ) )
      if ( EQUAL( LCHILD( t1 ) , LCHILD( t2 ) ) )
        x= EQUAL( RCHILD( t1) , RCHILD( t2) )
    return( x ) ;
} /* EQUAL */
```



5.4.2 二叉树的相似与等价

- 二叉树的复制

```
BTREE COPY( BTREE oldtree )
{
    BTREE temp ;
    if ( oldtree != NULL )
    {
        temp = new Node ;
        temp -> lch = COPY( oldtree->lch ) ;
        temp -> rch = COPY( oldtree->rch ) ;
        temp -> data = oldtree->data ;
        return ( temp ) ;
    }
    return ( NULL ) ;
}
```



如何利用二叉链表的空指针域？

线性链表:从单向链表→双向链表



知识回顾

- 正像线性链表可以从单向链表发展到双向链表一样，**二叉链表也可以采用双向链表。**
- **二叉链表是一种单向链接结构**，从一个结点出发，沿着指针走向只能到达其子孙结点，却无法返回其祖先结点。
- 按某种遍历方式对二叉树进行遍历，可以将二叉树中的结点排列成一个线性序列。但是，**二叉树的存储结构不能反映结点的前驱和后继关系**，只能在对二叉树遍历的动态过程中得到这些信息。

线索二叉树



5.5 线索二叉树

5.5.1 线索二叉树的表示

5.5.2 二叉树的线索化

5.5.3 线索二叉树的遍历



5.5 线索二叉树

5.5.1 线索二叉树的表示

5.5.2 二叉树的线索化

5.5.3 线索二叉树的遍历



5.5.1 线索二叉树的表示

- 线索二叉树的概念

➤考虑利用这些空链域来存放遍历后结点的前驱和后继信息，这就是线索二叉树构成的思想。

➤采用既可以指示其前驱又可以指示后继的双向链接结构的二叉树被称为线索二叉树。

利用空链域存储信息



链式存储结构——线索链表



5.5.1 线索二叉树的表示

- 线索链表的结点结构

lch	ltag	data	rtag	rch
-----	------	------	------	-----

```
typedef struct node
```

```
{datatype data;
```

```
struct node *lch,*rch;
```

```
int ltag,rtag;
```

```
}threadbithptr;
```

其中：ltag, rtag为两个标志域

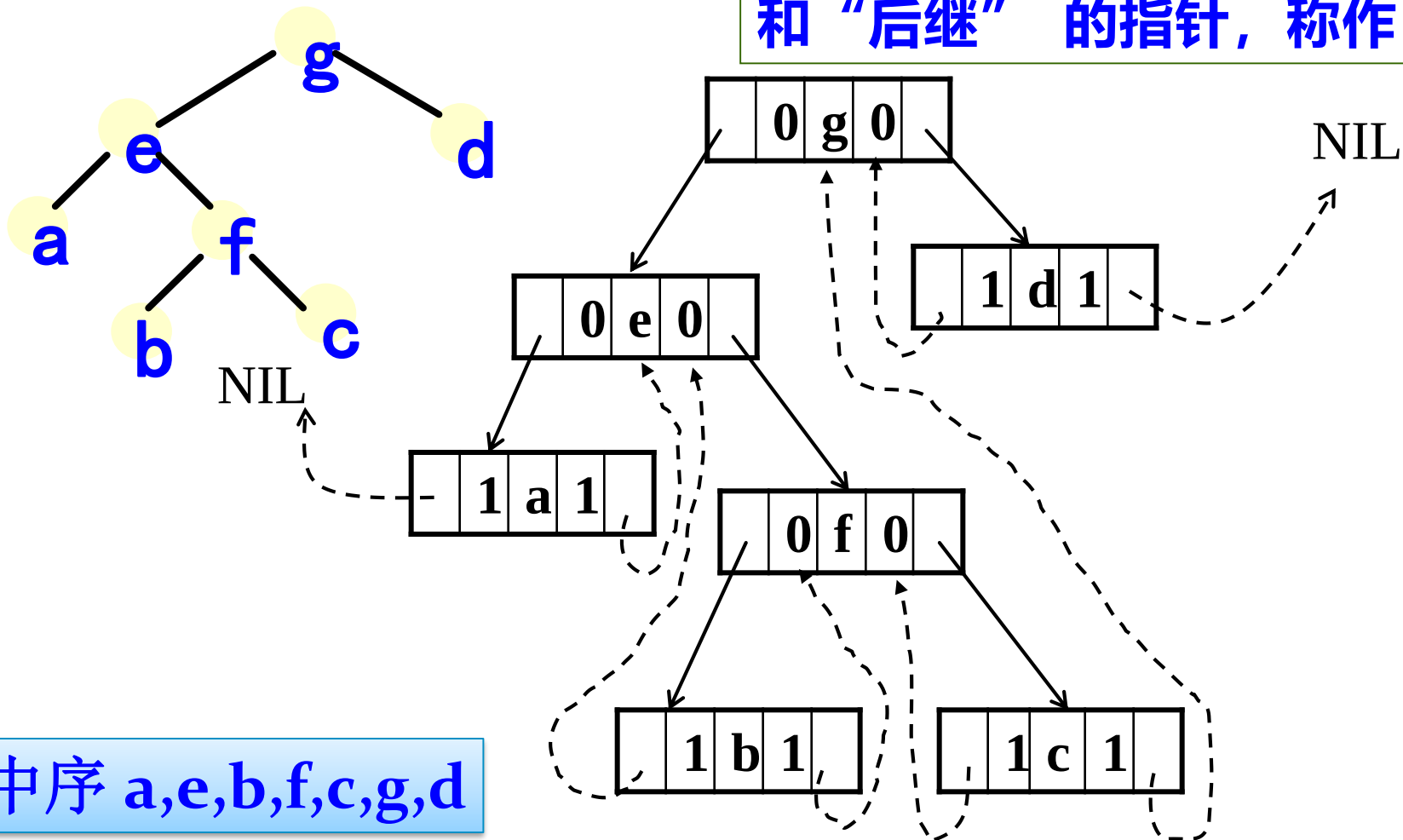
ltag= $\begin{cases} 0 & \text{lch域指示结点的左孩子} \\ 1 & \text{lch域指示结点的前驱} \end{cases}$

rtag= $\begin{cases} 0 & \text{rch域指示结点的右孩子} \\ 1 & \text{rch域指示结点的后继} \end{cases}$



5.5.1 线索二叉树的表示

- 二叉树存储结构--线索链表
- 遍历指向该线性序列中的“前驱”和“后继”的指针，称作“线索”





5.5.1 线索二叉树的表示

- 线索二叉树的相关概念

- **线索链表**：以上述结点结构构成的二叉链表作为二叉树的存储结构，叫线索链表。
- **线索**：指向前驱和后继的指针。
- **线索二叉树**：采用双向链接结构表示的二叉树。
- **线索化**：对二叉树以某种次序遍历使其变为线索二叉树的过程。



5.5 线索二叉树

5.5.1 线索二叉树的表示

5.5.2 二叉树的线索化

5.5.3 线索二叉树的遍历



5.5.2 二叉树的线索化

- 关于线索二叉链表的头结点的设定方法1

线性链表:从单向链表→双向链表

二叉树: 二叉链表→线索二叉链表

如何设定
头结点?

线索二叉树的头结点: 在某种遍历下的第一个结点的前驱线索和最后一个结点的后继线索均指向头结点。



5.5.2 二叉树的线索化

- 关于线索二叉链表的头结点的设定方法1

申请头结点：在存储线索二叉树时增设一个头结点，令其lchild指向二叉树的根结点，其rchild指向自身。

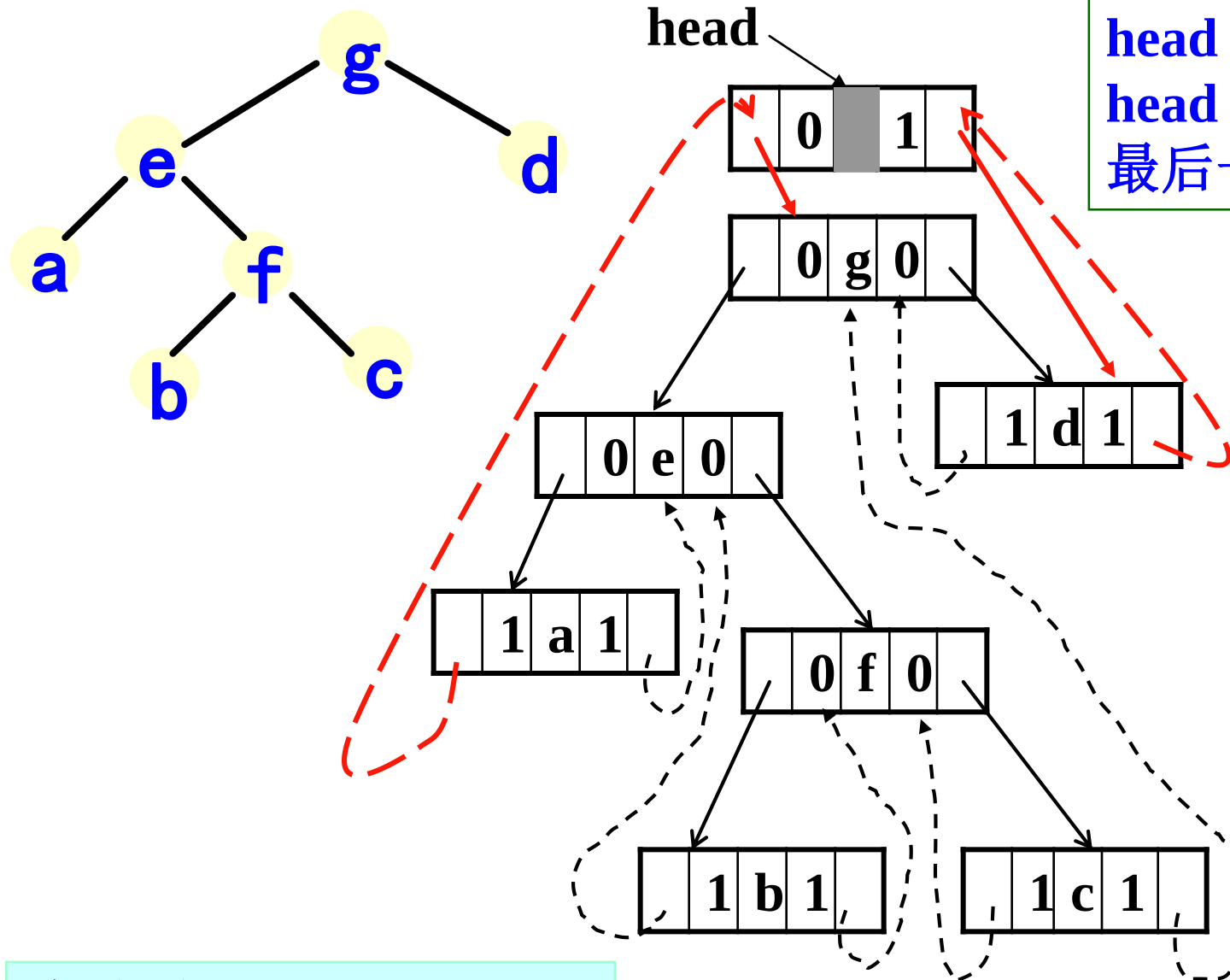
```
head->lch = T (二叉树的根);  
head->rch = head;  
head->ltag = 0;  
head->rtag = 0;
```

当线索树为空时：

```
head->lch = head;  
head->rch = head;  
head->ltag = 1;  
head->rtag = 0;
```



关于线索二叉链表的头结点的设定方法2



head ->lch指向根结点
head ->rch指向中序
最后一个结点

中序遍历的
第一个结点和
最后一个结点
都指向头结点

构成双向
循环线索链表

中序序列 a,e,b,f,c,t,d

哈工大《深圳》计算机学院 中序线索链表



5.5.2 二叉树的线索化

- 查找---在线索树中找结点---**中序后继**

在中序线索树中查找中序后继的方法

(1) 当结点没有右子树时，即：

当 $p \rightarrow rtag = 1$ 时， $p \rightarrow rch$ 既为所求后继结点（线索）。

(2) 当结点有右子树时，即：

当 $p \rightarrow rtag = 0$ 时， p 的后继结点为 p 的**右子树的最左下结点**。

如何在
中序线索树找指
定结点的后继？



5.5.2 二叉树的线索化

- 在中序线索树中找结点*p的**中序后继**

```
threadbithptr * Inordernext(bithptr *p)
```

```
{
```

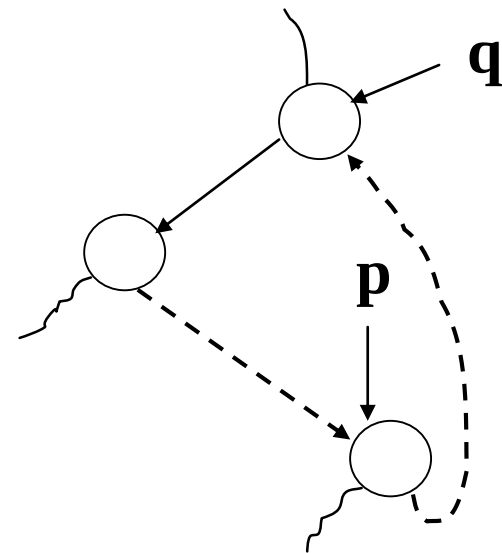
```
threadbithptr *q;
```

```
if (p->rtag==1) // 右子树空
```

```
return(p->rch);
```

```
else // 右子树非空
```

```
}
```



(a)

p的右子树空



5.5.2 二叉树的线索化

- 在中序线索树中找结点*p的**中序后继**

```
threadbithptr * Inordernext(bithptr *p)
```

```
{
```

```
    else                // *p右子树非空
```

```
    { q=p->rch; //从*p的右子树开始找
```

```
      while(q->ltag==0)
```

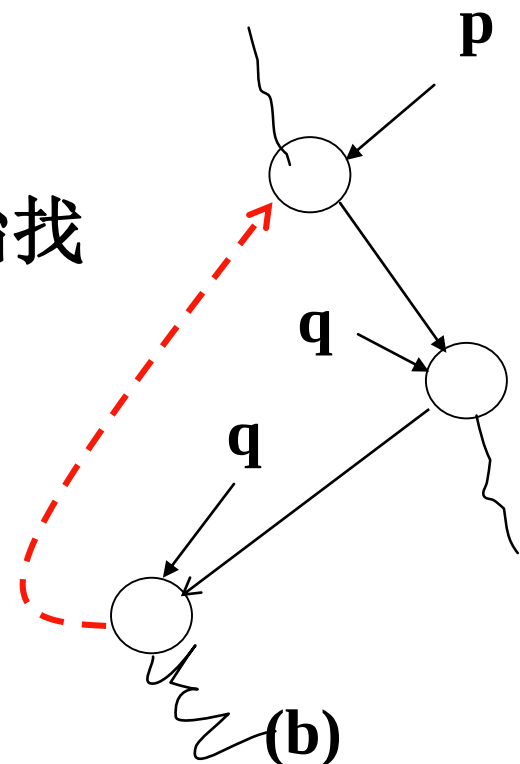
```
        q=q->lch;
```

```
    // 找右子树的“最左下”结点
```

```
    return(q);
```

```
}
```

```
}
```



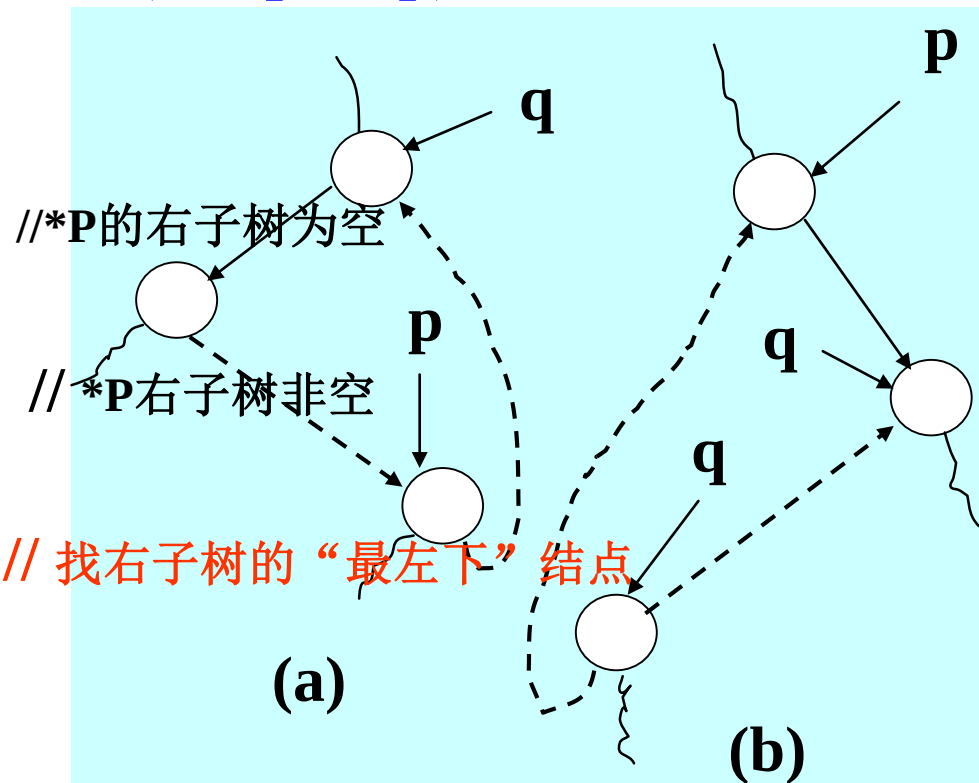
p的右子树不空



5.5.2 二叉树的线索化

- 在中序线索树中找结点*p的**中序后继**
`threadbithptr * Inordernext(bithptr *p)`

```
{
threadbithptr *q;
if (p->rtag==1)
    return(p->rch);
else
{   q=p->rch;
    while(q->ltag==0) // 找右子树的“最左下”结点
        q=q->lch;
    return(q);
}
```



//查找结点*p的中序后继



5.5.2 二叉树的线索化

- 查找--在线索二叉树中找结点--**中序前驱**

在线索二叉树中查找中序前驱的方法

(1) 当结点没有左子树时，即：

当 $p \rightarrow ltag = 1$ 时， $p \rightarrow lch$ 既为所求前驱结点（线索）。

(2) 当结点有左子树时，即：

当 $p \rightarrow ltag = 0$ 时， p 的前驱结点为 p 的**左子树的最右下结点**。

如何在**中序线索树**找

指定结点的前驱？

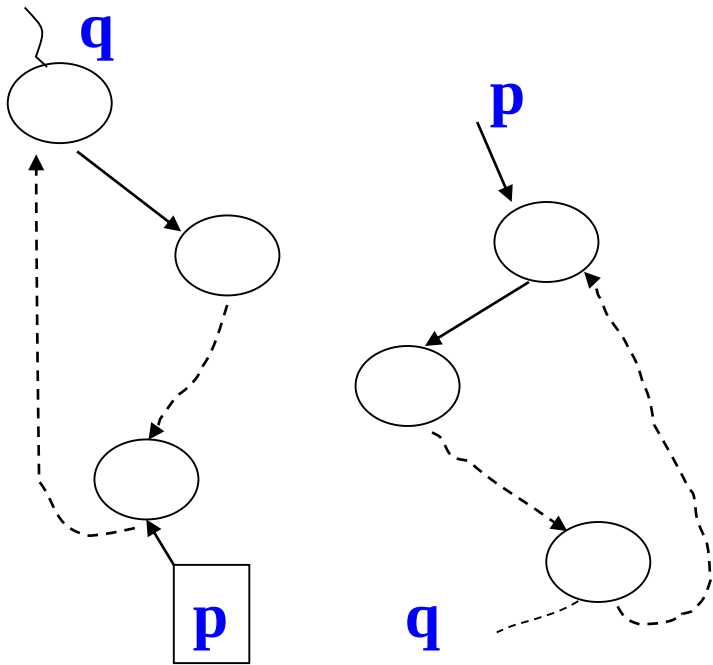


5.5.2 二叉树的线索化

- 在中序线索树中找结点*p的**中序前驱**

分析：(1) 当 $p \rightarrow ltag = 1$ 时， $p \rightarrow lch$ 既为所求（线索）。

(2) 当 $p \rightarrow ltag = 0$ 时， q 为 p 的**左子树的最右下结点**。



p 的左孩子为空 p 的左孩子不空

```

threadbithptr inpre(threadbithptr p)
threadbithptr p ;
{threadbithptr q ;
  q = p->lch ;

  if (p->ltag == 0 )
    while(q->rtag == 0)
      q = q->rch ;
}
  
```



5.5 线索二叉树

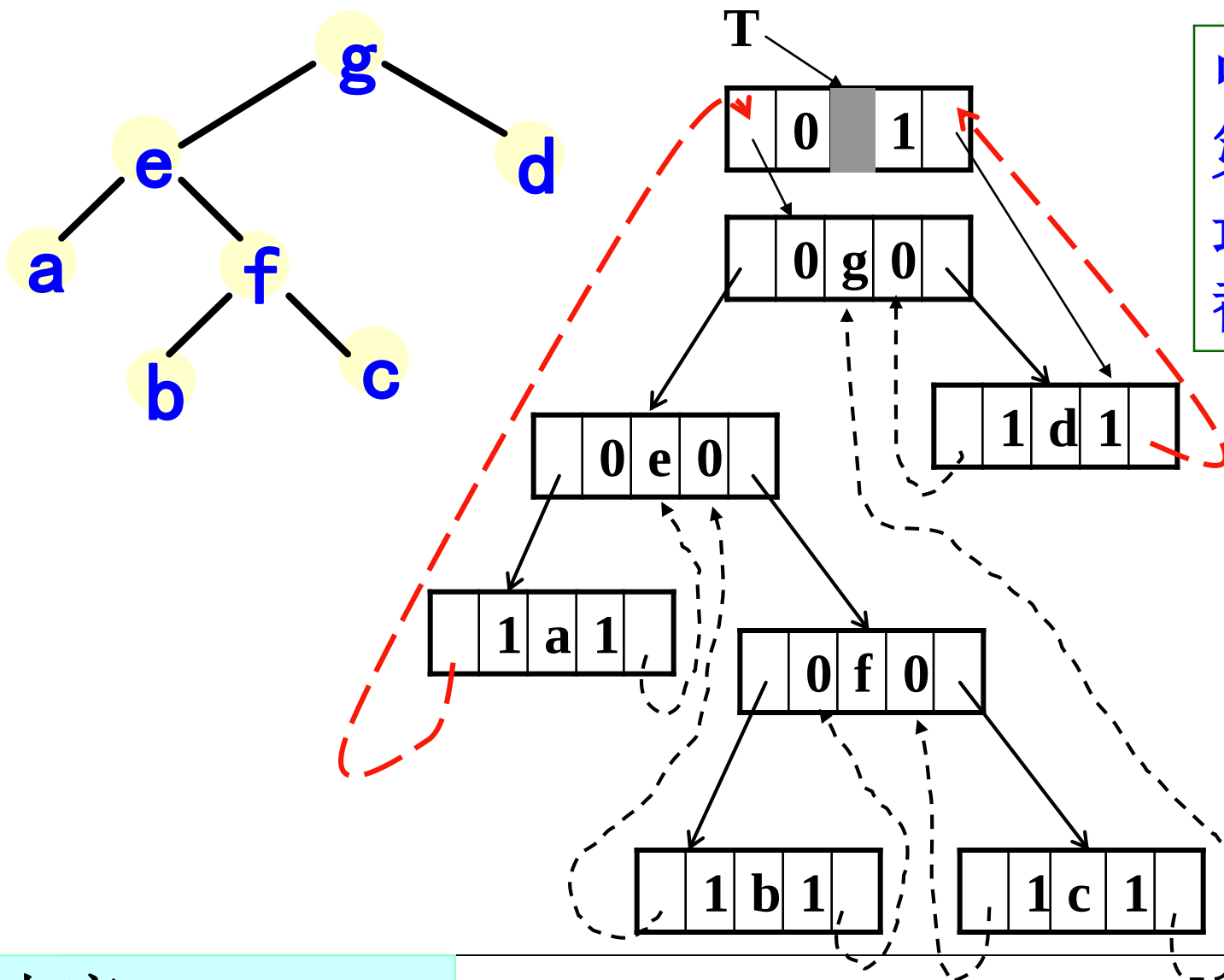
5.5.1 线索二叉树的表示

5.5.2 二叉树的线索化

5.5.3 线索二叉树的遍历



5.5.3 线索二叉树的遍历



中序遍历的
第一个结点和
最后一个结点
都指向根结点

中序 a,e,b,f,c,t,d



5.5.3 线索二叉树的遍历

- 遍历中序线索二叉树的递归算法

```
TraverseInthread(threadbithptr *p)
```

```
{if (p!=Null)
```

```
{
```

```
    while (p->ltag==0)//找中序序列的开始结点
```

```
        p=p->lch;
```

```
do
```

```
{    visit(p->data)
```

```
    p=Inordernext(p);//找*p的中序后继结点
```

```
} while (p!=Null);
```

```
}
```

```
}
```



5.5.3 线索二叉树的遍历

• 遍历中序线索二叉树的非递归算法

Status InthreadTraverse (BiThrTree T, Status(*visit)){

//中序遍历线索二叉树T的非递归算法

T指向线索二叉树的头结点，头结点的lch指向根结点，中序的最后一个结点指向根结点。每次判定结点的ltag和rtag

p=T->lch;

//p指向二叉树真正的根结点

while(p!=T){

//空树或遍历结束时，p=T

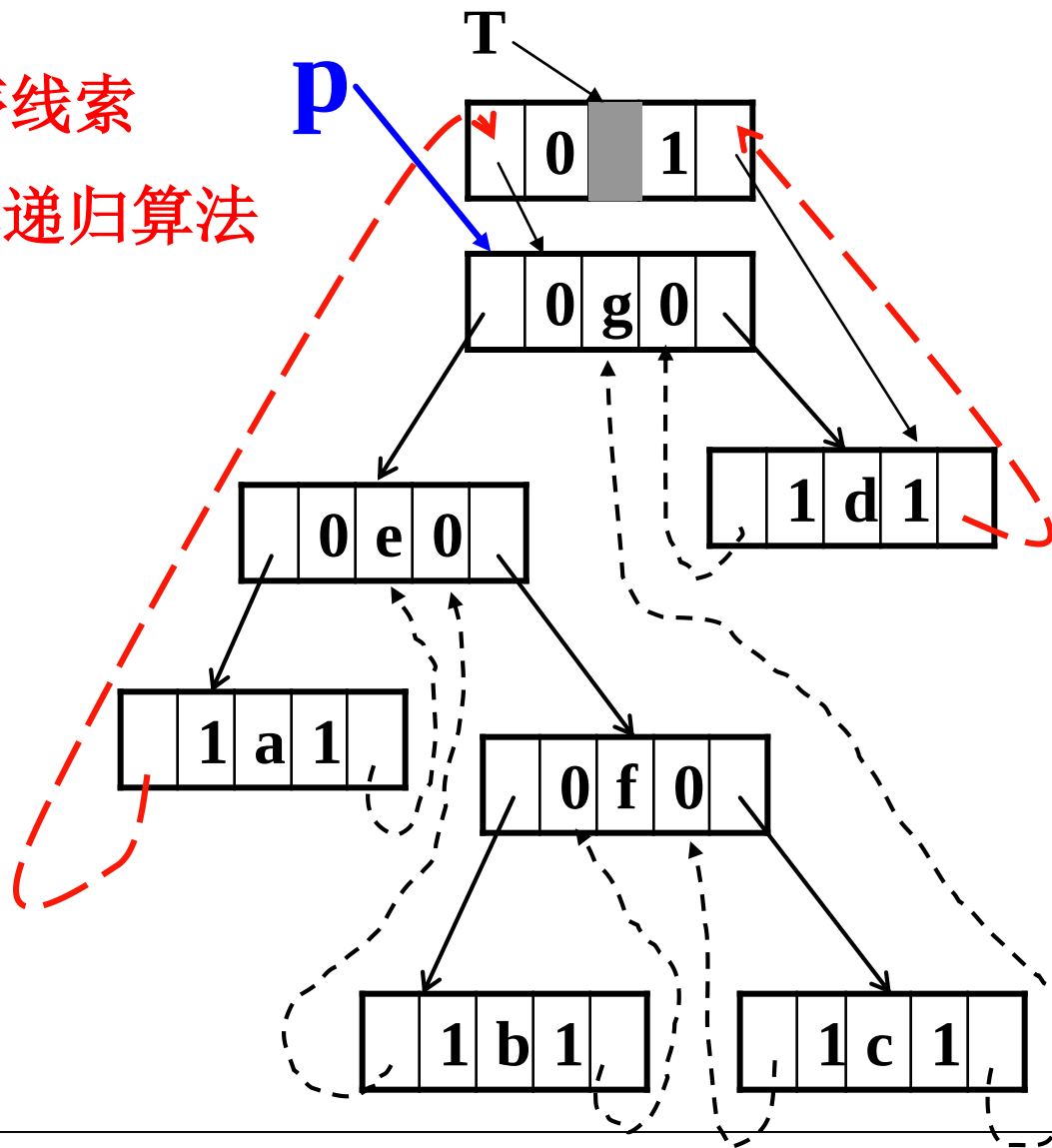
// 重复以上过程，直到有线索指向根为止

} return(ok);



5.5.3 线索二叉树的遍历

- 遍历中序线索
- 二叉树的非递归算法





Status InthreadTraverse (BiThrTree T, Status(*visit)){

//中序遍历线索二叉树T的非递归算法

T指向线索二叉树的头结点，头结点的lch指向根结点，中序的最后一个结点指向根结点。每次判定结点的ltag和rtag

```
p=T->lch;                //p指向二叉树真正的根结点
while(p!=T){              //空树或遍历结束时，p=T
    while (p->ltag==0)      p=p->lch;  //找中序序列的开始结点
        visit(p->data)        //访问图中的结点a
    while (p->rtag==1&& p->rch!=T ) //p不是中序遍历的最后一个结点
    {   p=p->rch;
        visit(p->data)    }          //访问后继结点，当前访问图中e
        p=p->rch;          //右线索指向后继结点
        // 重复以上过程，直到有线索指向根为止
    }   return(ok);
}
```



5.5.3 线索二叉树的遍历

- 中序线索化二叉树

线索化的实质：

是将二叉链表中的空指针改为指向前驱或后继的线索，而前驱或后继信息只有在遍历时才能得到，因此线索化的过程即为在遍历过程中修改空指针的过程。



5.5.3 线索二叉树的遍历

• 中序线索化二叉树

如何建立线索链表？

1. 在中序遍历过程中修改结点的左、右指针域
2. 保存当前访问结点的“前驱”和“后继”信息。
3. 遍历过程中，附设指针pre，pre指向刚访问过的结点。P指向当前访问结点。即pre是p的前驱

类似线性表的两个相邻的结点，
修改相应的pre域和next域

图示



5.5.3 线索二叉树的遍历

- 构建中序线索化二叉树//有头结点

```
Status InOrderThreading(BiThrTree &head, BiThrTree T) {  
    // 中序遍历二叉树，并将其线索化，head指向头结点  
    if (!(head = (BiThrTree)malloc(sizeof( BiThrNode))))  
        exit(OVERFLOW);  
    head->ltag = 0;  
    head->rtag = 1;  
    head->rchild = head;    // 添加头结点，指向自身
```



5.5.3 线索二叉树的遍历

- 中序线索化二叉树

```
if (!T)  head->lchild = head; //二叉树空，指向自身
else {  // 修改头结点和遍历的最后一个结点
    head->lchild = T;  pre = head;
                                //初始pre指向头结点head
    InThreading(T);      //对二叉树进行中序线索化
    pre->rtag = 1; pre->rchild = head;
    head->rchild = pre;   //修改头结点和最后一个结点
                        //头结点head与最后一个结点pre 互链接 }

return OK;
} // InOrderThreading
```



5.5.3 线索二叉树的遍历

- 中序线索化二叉树

```
void InThreading(BiThrTree p) {  
    if (p) {          // 对非空二叉树进行中序线索化  
        InThreading(p->lchild);          // 左子树线索化  
        if (!p->lchild)                  // 结点无左子树，建前驱线索  
            { p->ltag = 1;  p->lchild = pre; }  
        if (!pre->rchild)                  // 结点无右子树，建后继线索  
            { pre->rtag = 1;  pre->rchild = p; }  
        pre = p;                          // 保持 pre 指向 p 的前驱  
        InThreading(p->rchild);          // 右子树线索化  
    } // if  
}
```



5.5.3 线索二叉树的遍历

- 在二叉树中一般不讨论结点的插入与删除，原因是其插入与删除的操作与线性表相同，所不同的是需要详细说明操作的具体要求。
- 而在线索树中结点的插入与删除则不同，因为要同时考虑修正线索的操作。

线索树的缺点：

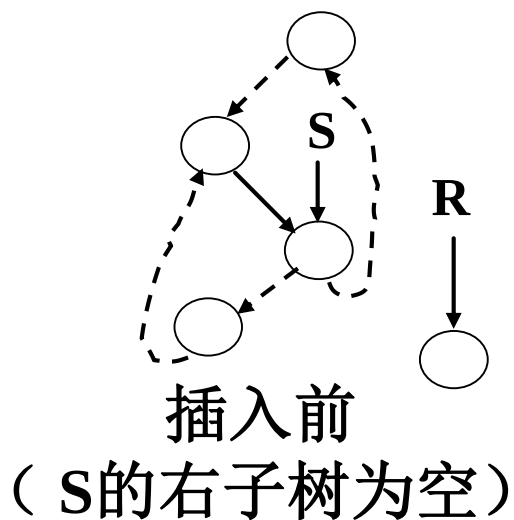
在插入和删除时，除了修改指针外，还要相应地修改线索。



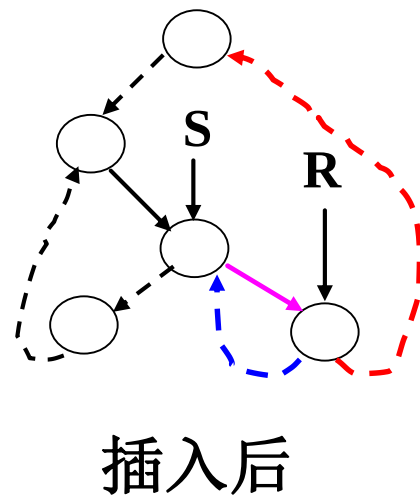
5.5.3 线索二叉树的遍历

例

将结点 R 插入作为结点 S 的右孩子结点：
(1) 若S的右子树为空，则直接插入；如图所示



(a)

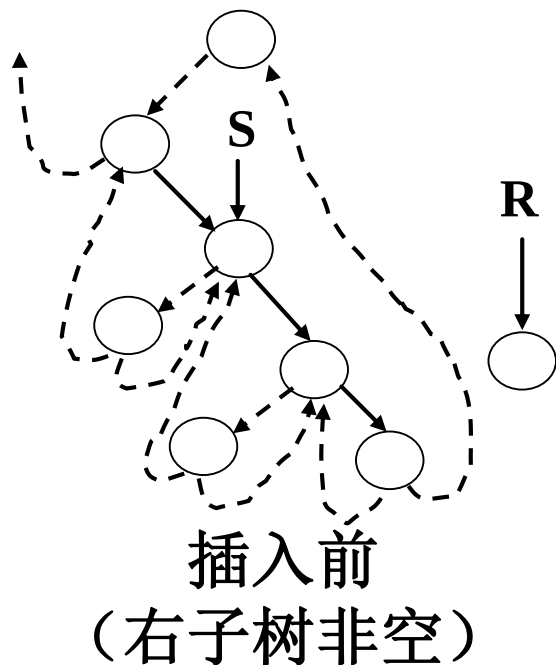




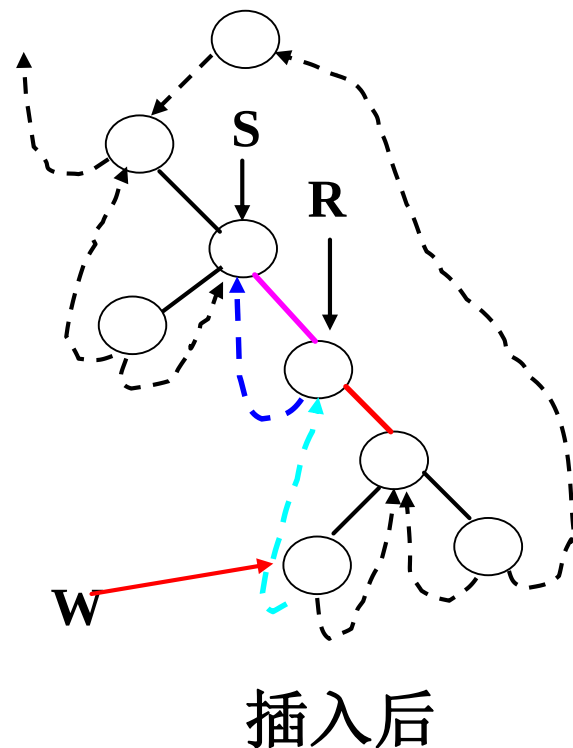
5.5.3 线索二叉树的遍历

例

(2) 若S的右子树非空，则R插入后原来S的右子树作为R的右子树。



(b)



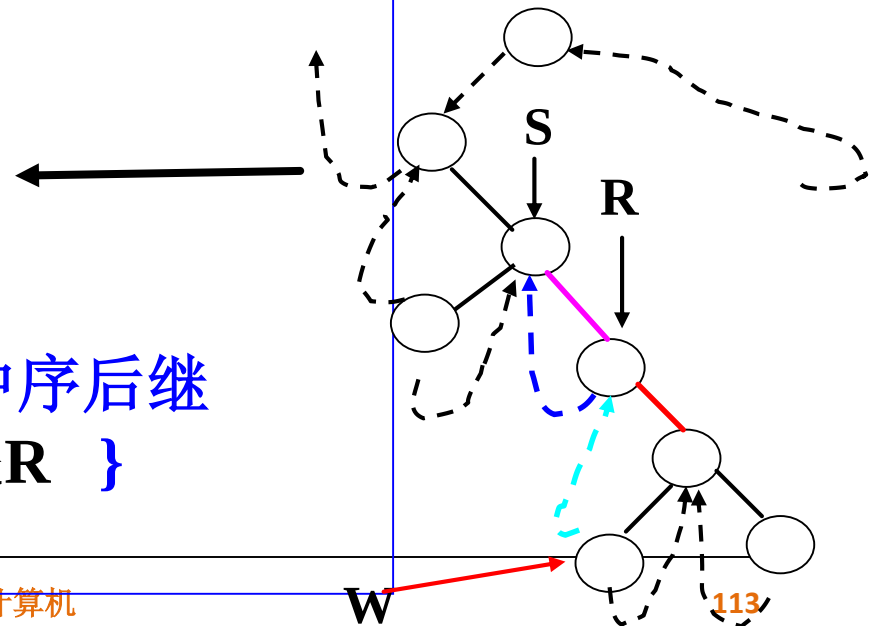
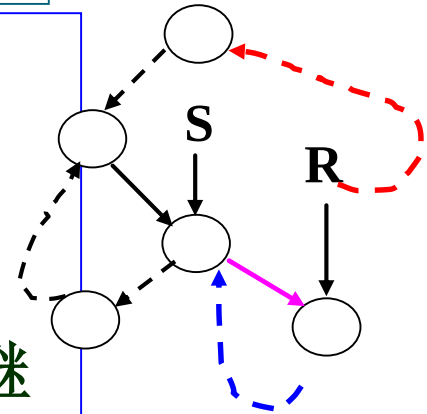


例

中序线索二叉树的右插入算法：
将结点 R 插入作为结点 S 的右孩子

```

Void RINSERT (threadbithptr S, R)
{ if ( S->rtag = 1)           // 右子树空
{ R->ltag = 1; R->rtag = 1;
  R->rch = S->rch;
  //R的中序后继是原来S的后继
  R->lch = S; //R的中序前驱是S
  S->rtag = 0; //修改s的右标识
  S->rch = R; //S的右孩子是R
}
else //右子树非空
{ .....
  w = Inordernext ( S ); //找S的中序后继
  w->lch = R; //w的中序前驱是R }
}
  
```





5.5.3 线索二叉树的遍历

思考题：

线索二叉树的左插入算法：将结点 R 插入作为结点 S 的左孩子



5.5.3 线索二叉树的遍历

应用举例

按给定先缀的表达式建立相应的二叉树

已知表达式的先缀表示式 $- \times + a b c / d e$

前缀式的运算规则为：

- 连续出现的两个操作数和它们在之前且紧靠它们的运算符构成一个最小表达式；
- 前缀式唯一确定了运算顺序；

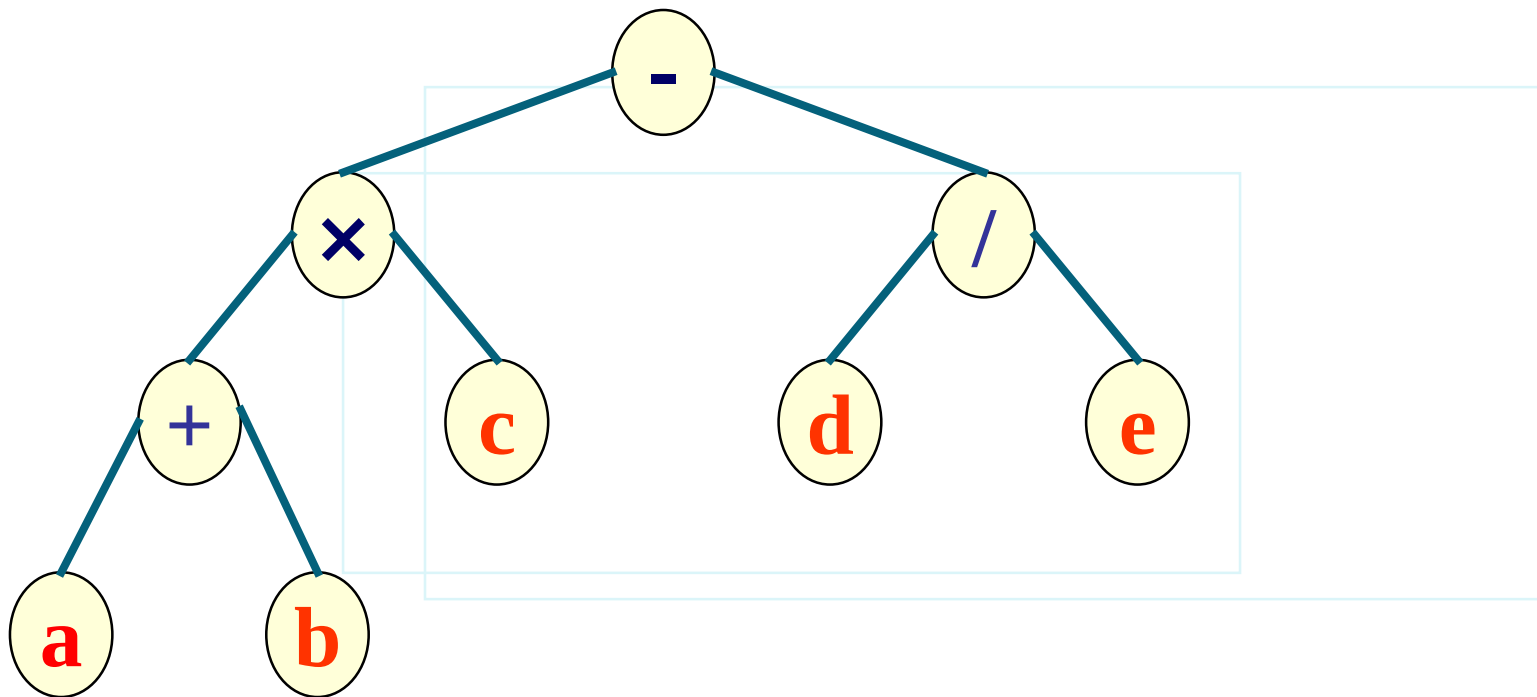


5.5.3 线索二叉树的遍历

应用举例

按给定先缀的表达式建立相应的二叉树

已知表达式的先缀表示式 $- \times + a b c / d e$



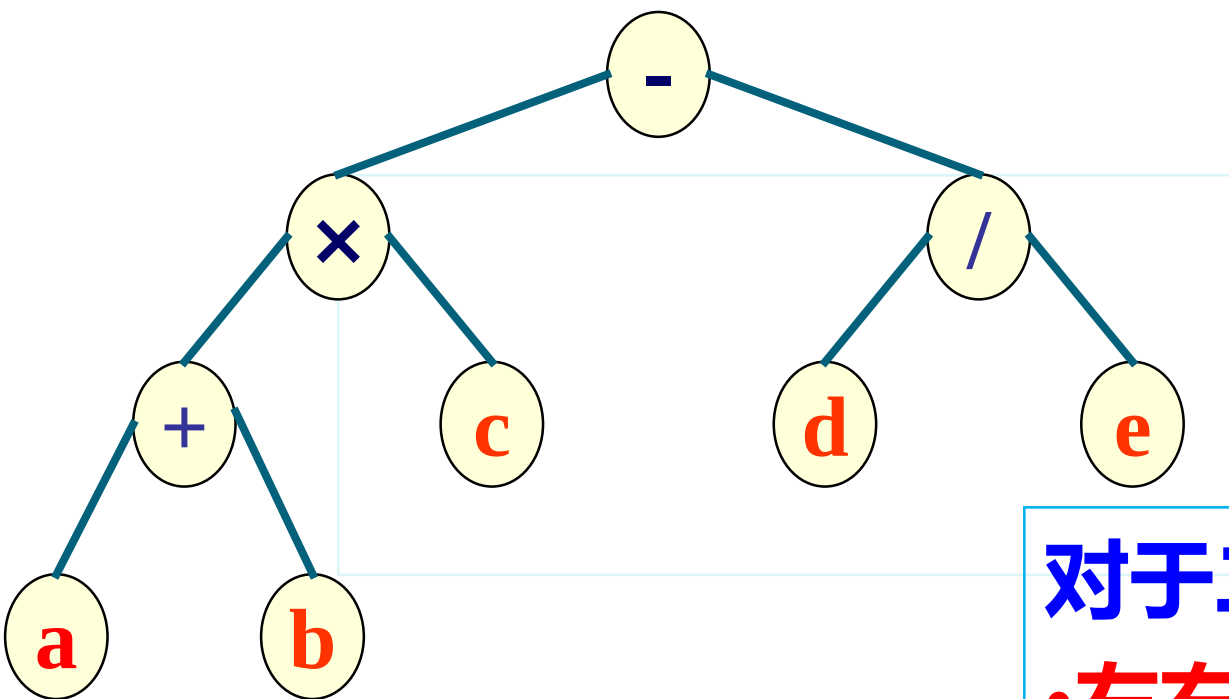
表达式 = (第一操作数) (运算符) (第二操作数)



5.5.3 线索二叉树的遍历

应用举例

按给定的表达式建立相应的二叉树



对于二元运算符：

- 左右子树不空；
- 操作数为叶子结点；
- 运算符为分支结点；



5.5.3 线索二叉树的遍历

应用举例

按给定后缀的表达式建立相应的二叉树

后缀式的运算规则为

- 运算符在式中出现顺序恰为表达式的运算顺序；
- 每个运算符和在它之前出现且紧靠它的两个操作数构成一个最小表达式。
- 先找运算符，再找操作数。
- 操作数的顺序不变。



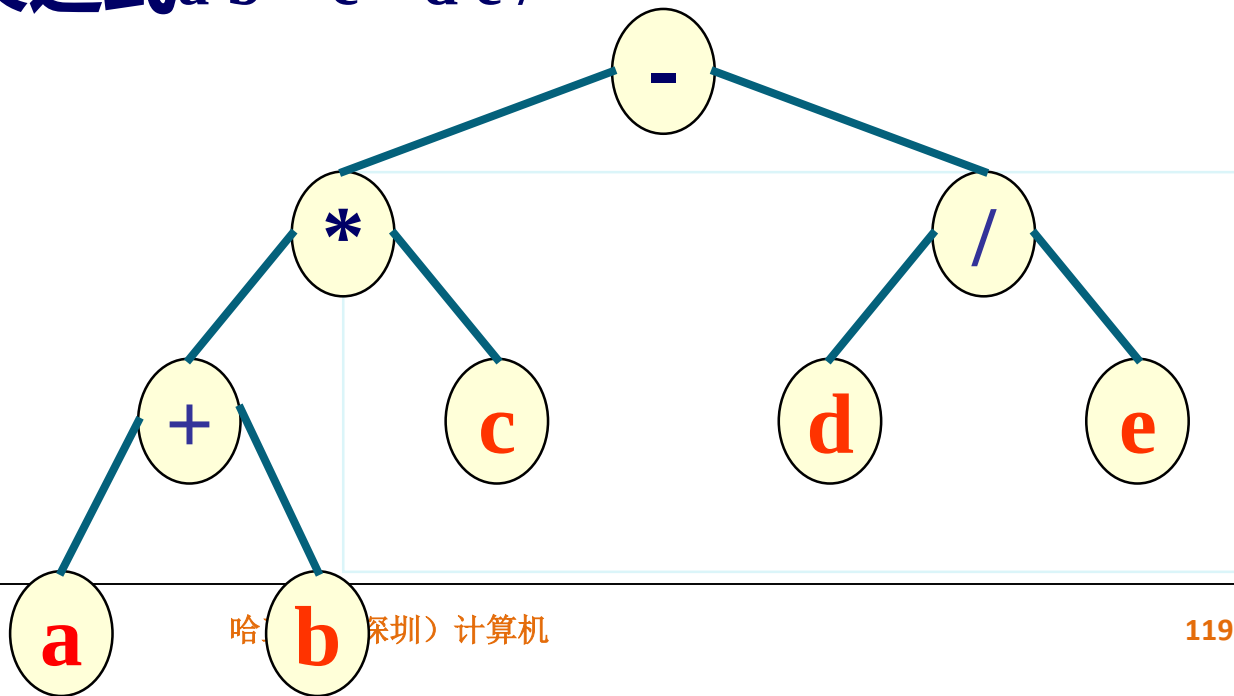
5.5.3 线索二叉树的遍历

应用举例

按给定后缀的表达式建立相应的二叉树

方法：从左到右扫描后缀表达式，遇到操作符则对前面的操作数建立二叉树，以此类推。

例如：后缀表达式 $a\ b\ +\ c\ *\ d\ e\ /\ -$





研究树的目的：

使树的操作都是通过二叉树来完成的



5.6 树和森林

5.5.1 树的存储结构

5.5.2 树、森林与二叉树的转换

5.5.3 树和森林的遍历

5.5.4 huffman树及其应用



5.6 树和森林

5.5.1 树的存储结构

5.5.2 树、森林与二叉树的转换

5.5.3 树和森林的遍历

5.5.4 huffman树及其应用

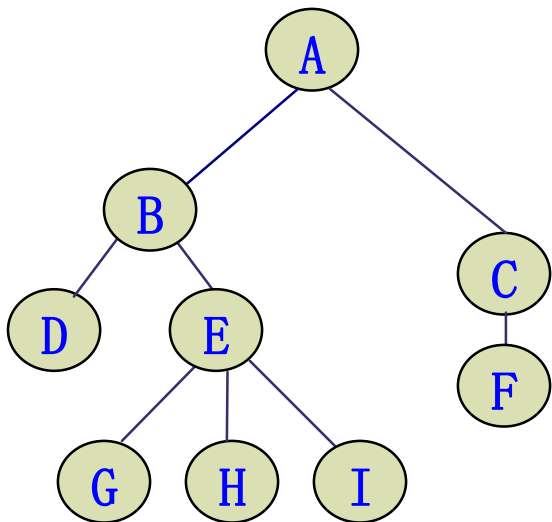


5.5.1 树的存储结构

• 双亲表示法

采用一组连续空间存储树的结点, 通过保存每个结点的
双亲结点的位置, 表示树中结点之间的结构关系。

结点数



data parent

0		9
1	A	0
2	B	1
3	C	1
4	D	2
5	E	2
6	F	3
7	G	5
8	H	5
9	I	5



5.5.1 树的存储结构

- 双亲表示法

树结构:

data	parent
------	--------

双亲表示类型定义

```
#define MAX 100
Typedef struct PTnode{
    //结点结构
    Elem data;
    int parent; //双亲位置域
} PTnode
```



5.5.1 树的存储结构

- 双亲表示法

树结构:

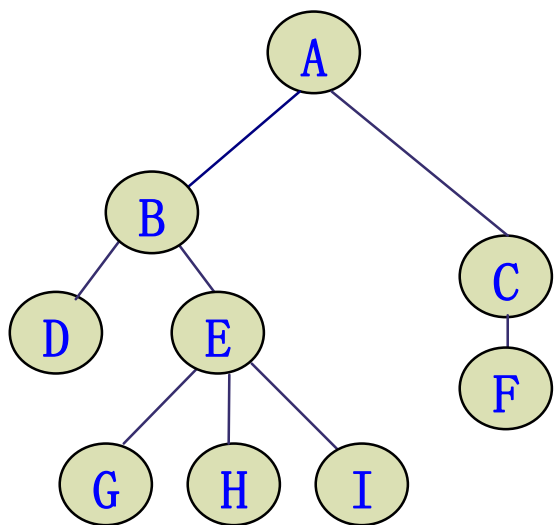
```
typedef struct {  
    PTNode nodes  
        [MAX_TREE_SIZE];  
    int    r, n;  
        // 根结点的位置和结点个数  
} PTree;
```



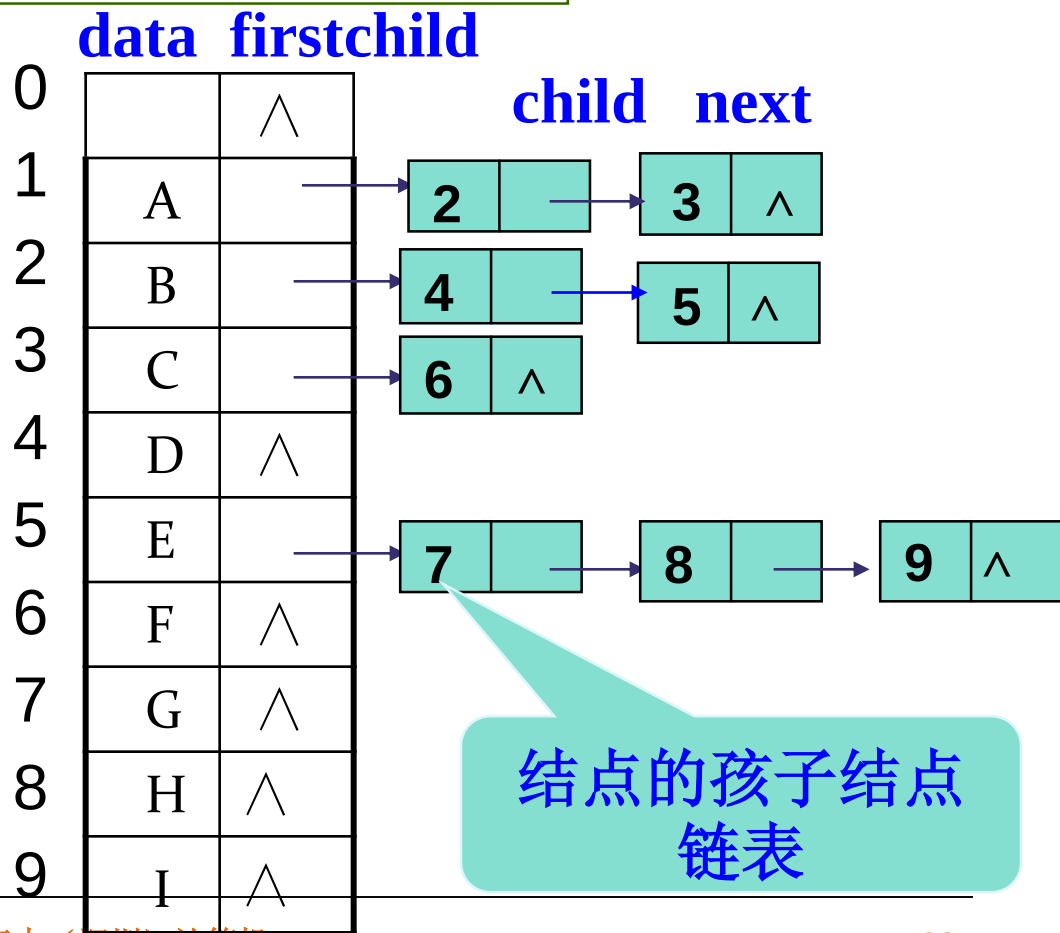
5.5.1 树的存储结构

• 孩子链表表示法

对树的每个结点用线性链表存贮它的孩子结点



树的孩子链表图示





5.5.1 树的存储结构

• 孩子链表表示法

对树的每个结点用线性链表存贮它的孩子结点

孩子结点结构:

child	next
-------	------

```
typedef struct CTNode {  
    int      child;  
    struct CTNode *next;  
} *ChildPtr;
```

双亲结点结构:

data	firstchild
------	------------

```
typedef struct {  
    Elem  data;  
    ChildPtr firstchild;  
        // 孩子链的头指针  
} CTBox;
```

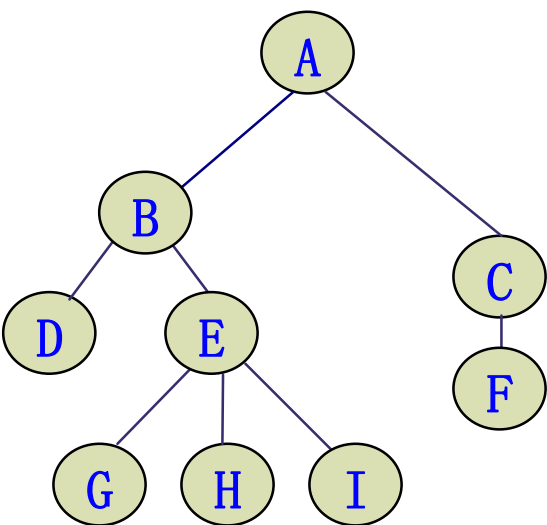
找一个结点的孩子十分方便，但要找一个结点的双亲则要遍历整个结构



5.5.1 树的存储结构

• 双亲孩子表示法

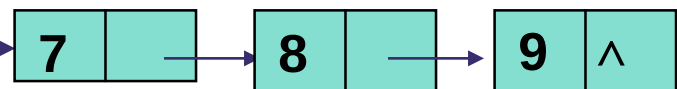
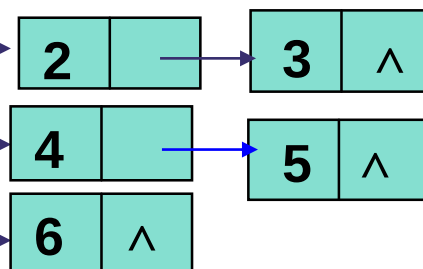
结合双亲表示法
和孩子表示法



data parent link

r=0
n=9

0		9	^
1	A	0	
2	B	1	
3	C	1	
4	D	2	^
5	E	2	
6	F	3	^
7	G	5	^
8	H	5	^
9	I	5	^



结点的孩子结点
链表



5.5.1 树的存储结构

- 树的二叉链表---孩子兄弟表示法

如何把树与二叉树联系起来？



5.5.1 树的存储结构

- 树的二叉链表---孩子兄弟表示法

用二叉链表作为树的存贮结构。链表的两个指针域分别指向该结点的第一个孩子结点和右边下一个兄弟结点。

结点结构:

firstchild	data	firstbrother
------------	------	--------------

```
typedef struct CSNode{  
    Elem      data;  
    struct CSNode  
        *firstchild, *firstbrother;  
} CSNode, *CSTree;
```

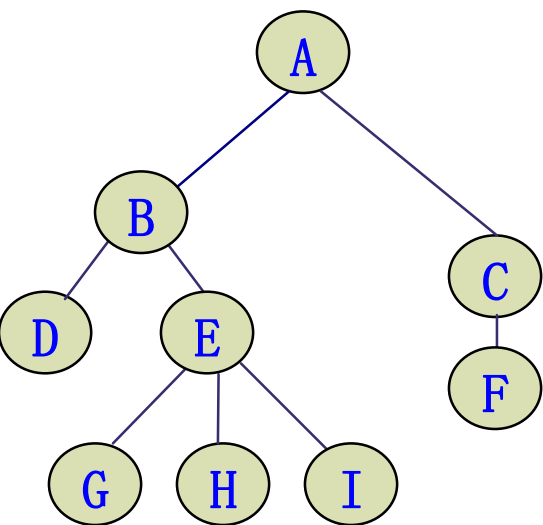


5.5.1 树的存储结构

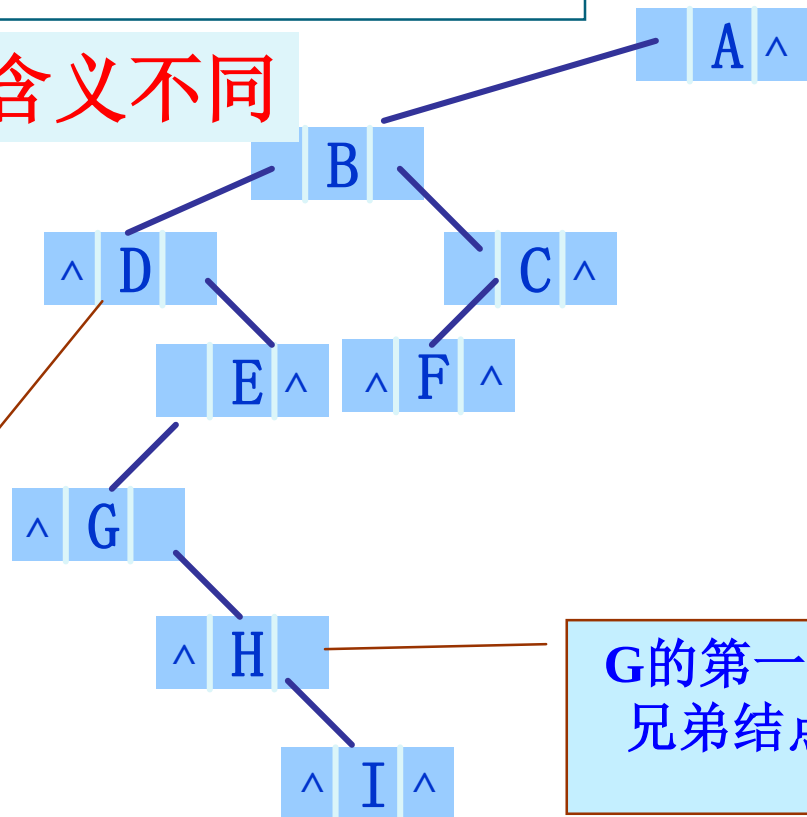
• 树的二叉链表---孩子兄弟表示法

用二叉链表作为树的存贮结构。链表的两个指针域分别指向该结点的第一个孩子结点和右边第一个兄弟结点。

与二叉树的存储表示一样，但含义不同



B的第一个
孩子结点

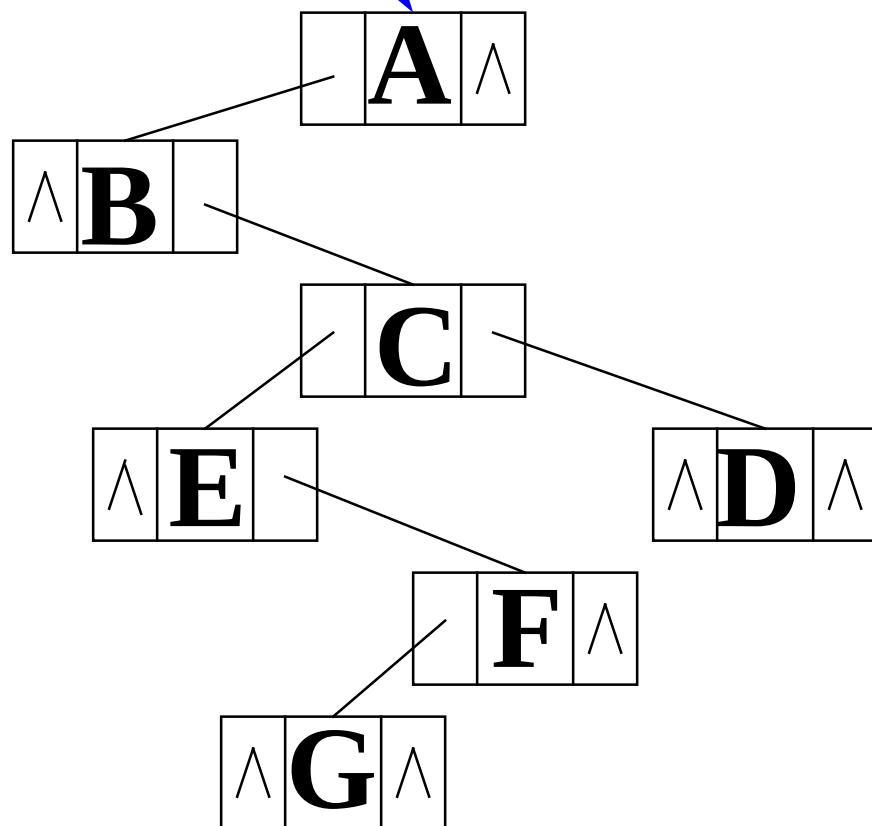
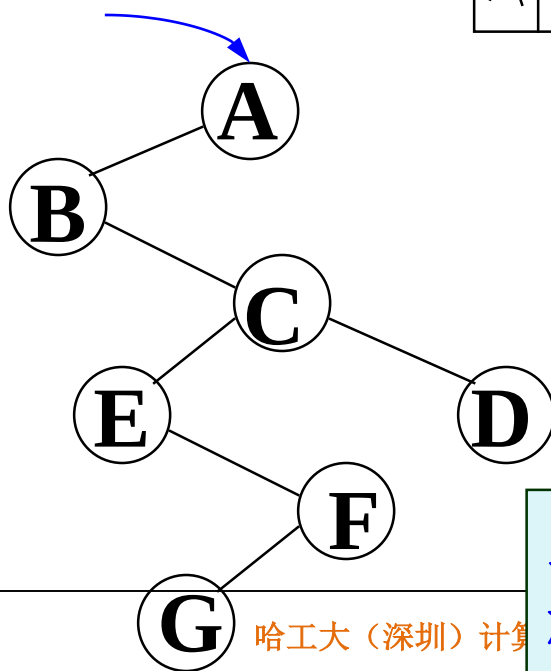
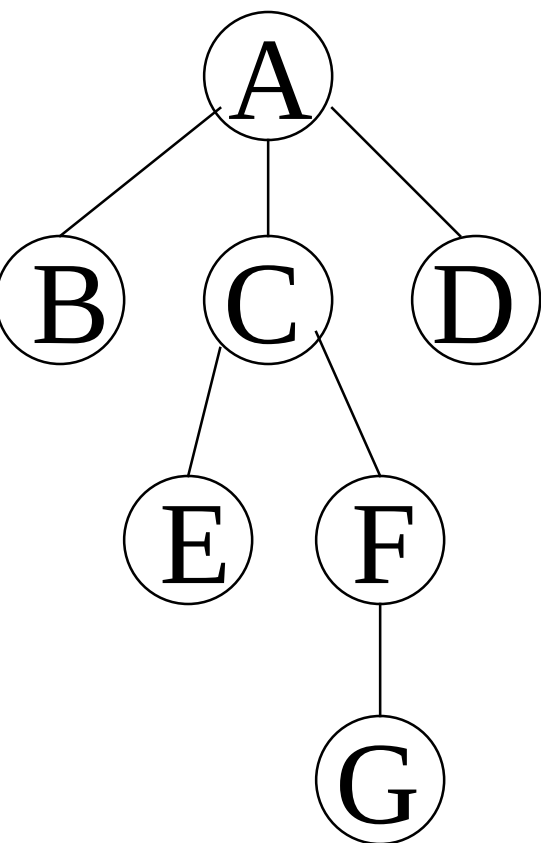


G的第一个
兄弟结点



树的二叉链表---孩子兄弟表示法

root



二叉树：左、右子树
树：左是孩子，右是兄弟



5.5.1 树的存储结构

• 树的二叉链表---孩子兄弟表示法

树和二叉树的存储表示方式是一样的，只是左右孩子表达的逻辑关系不同：

- ✚ 二叉树：左右孩子；
- ✚ 树的二叉链表：第一个孩子结点和右边第一个兄弟结点。

把树和二叉树对应起来

如何把树转化成二叉树？



5.6 树和森林

5.5.1 树的存储结构

5.5.2 树、森林与二叉树的转换

5.5.3 树和森林的遍历

5.5.4 huffman树及其应用



5.5.2 树、森林与二叉树的转换

- 树转换为二叉树的方法

树转换为二叉树的方法

- (1) 在所有兄弟结点之间加一条连线;
- (2) 对每个结点, 除了保留与其长子的连线外, 去掉该结点与其它孩子的连线;

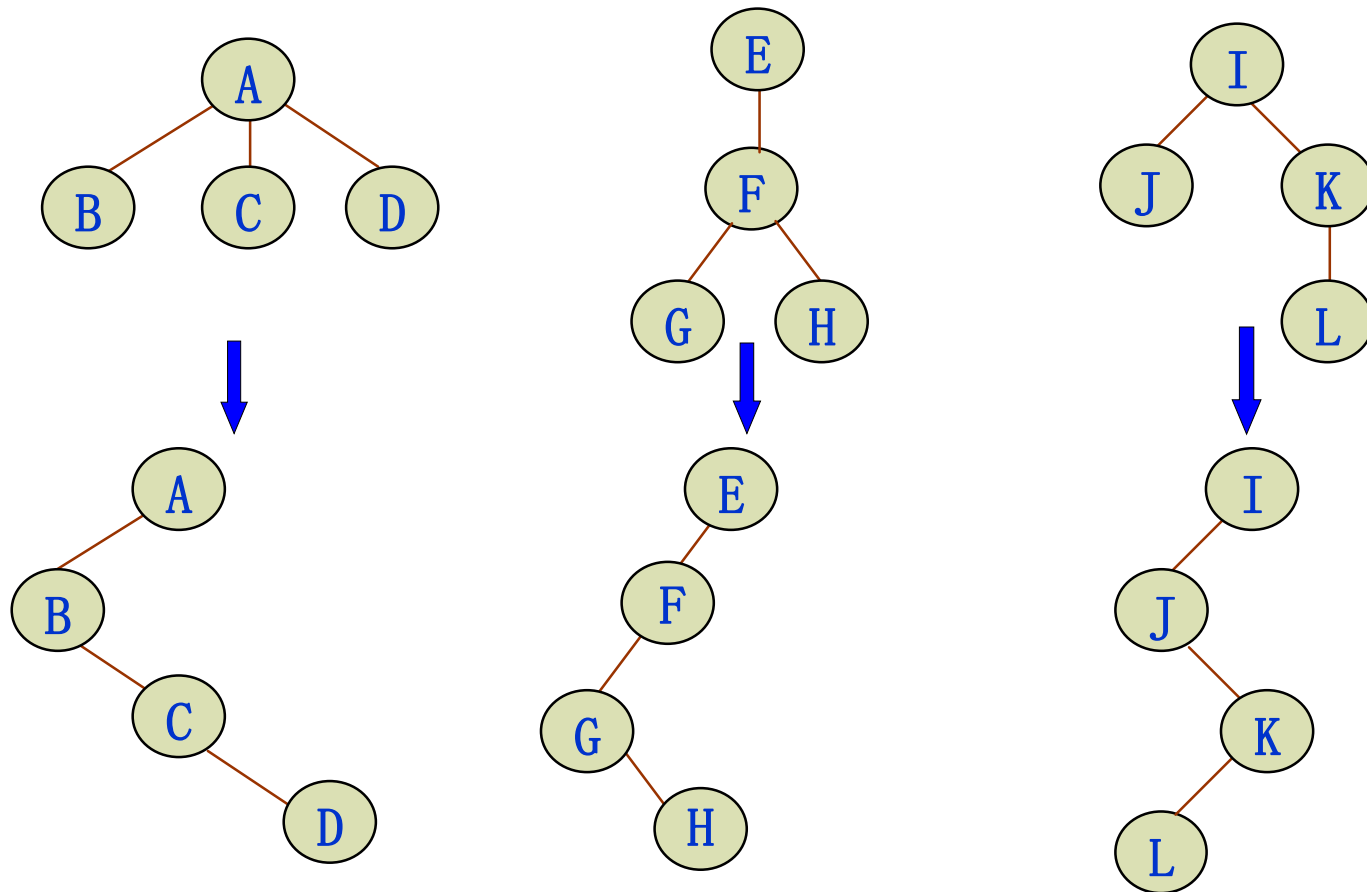


图示



5.5.2 树、森林与二叉树的转换

例 将树转换为二叉树



每棵树对应的二叉树



5.5.2 树、森林与二叉树的转换

- 森林转换为二叉树的方法

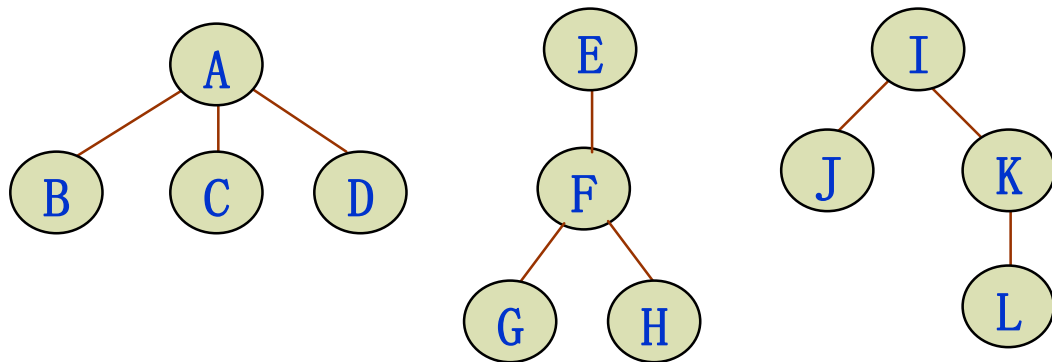
- (1) 将森林中的每一树转换二叉树;
- (2) 将各二叉树的根结点视为兄弟连在一起。



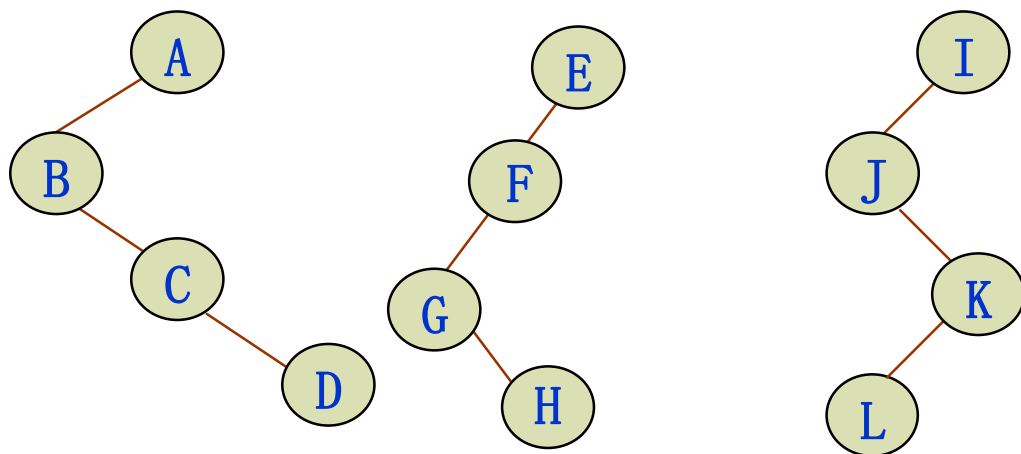


森林转换为二叉树的方法

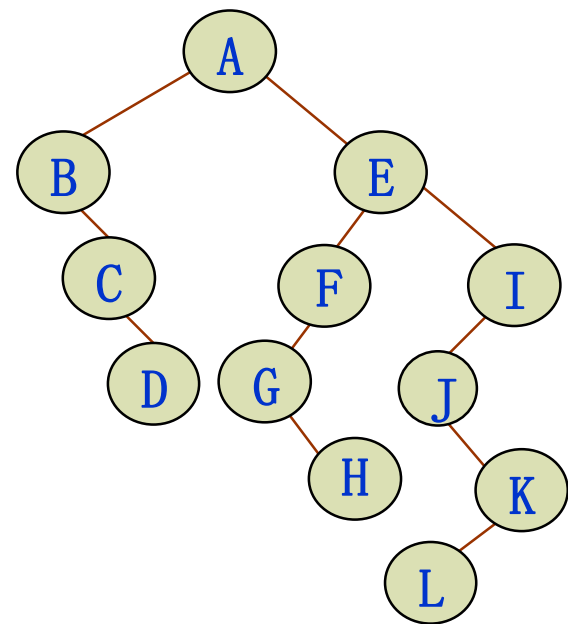
例 森林转换为二叉树



包含3棵树的森林



每棵树对应的二叉树



森林对应的二叉树



5.5.2 树、森林与二叉树的转换

• 森林和二叉树的对应关系

设森林 $F = (T_1, T_2, \dots, T_n)$;

$$T_1 = (\text{root}, t_{11}, t_{12}, \dots, t_{1m});$$

二叉树 $B = (\text{Node}(\text{root}), \text{LBT}, \text{RBT})$;

由森林转换成二叉树的转换规则为:

若 $F = \Phi$, 则 $B = \Phi$;

否则,

由 $\text{ROOT}(T_1)$ 对应得到 $\text{Node}(\text{root})$;

由 $(t_{11}, t_{12}, \dots, t_{1m})$ 对应得到 LBT ;

由 $(T_2, T_3, \dots, T_n)$ 对应得到 RBT 。



5.5.2 树、森林与二叉树的转换

- 森林和二叉树的对应关系

由二叉树转换为森林的转换规则为：

若 $B = \Phi$ ，则 $F = \Phi$ ；

否则，

由 $\text{Node}(\text{root})$ 对应得到 $\text{ROOT}(T_1)$ ；

由LBT 对应得到 $(t_{11}, t_{12}, \dots, t_{1m})$ ；

由RBT 对应得到 $(T_2, T_3, \dots, T_n)$ 。



5.5.2 树、森林与二叉树的转换

• 二叉树到树、森林的转换

- (1) 如果结点 X 是其双亲 Y 的左孩子，则把 x 的右孩子，右孩子的右孩子，...，都与 Y 用连线连起来；
- (2) 去掉所有双亲到右孩子的连线



图示



5.5.2 树、森林与二叉树的转换

- 树或森林与二叉树之间有一个自然的一一对应的关系。
- 任何一个森林或树可以唯一地对应到一棵二叉树；
- 任何一棵二叉树可以唯一地对应到一个森林或一棵树

**任何一棵与树对应的
二叉树, 其右子树必为空**



5.6 树和森林

5.5.1 树的存储结构

5.5.2 树、森林与二叉树的转换

5.5.3 树和森林的遍历

5.5.4 huffman树及其应用



5.5.3 树和森林的遍历

- 与二叉树的遍历类似，树的遍历也是从根结点出发，对树中各个结点访问一次且仅进行一次。
- 由于一个结点可以有两棵以上的子树，因此一般不讨论中根遍历。
- 对树进行遍历可有两条搜索路径：
 - (1) 从左到右
 - (2) 按层次从上到下。
- 树的按层次遍历类似于二叉树的按层次遍历；



5.5.3 树和森林的遍历

• 树的先根遍历

□ 先根顺序

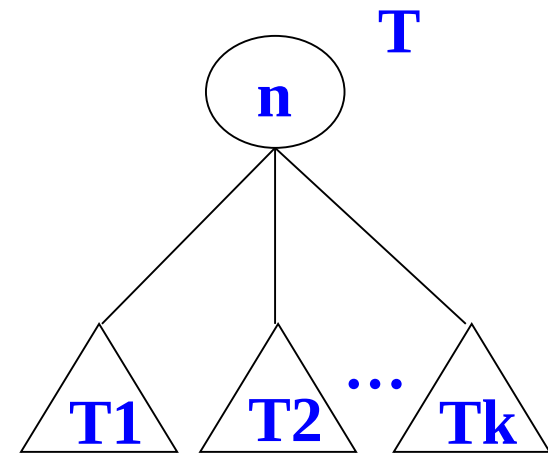
访问根结点；

先根顺序遍历 T_1 ；

先根顺序遍历 T_2 ；

...

先根顺序遍历 T_k ；





5.5.3 树和森林的遍历

• 树的先根遍历

树的先根遍历递归该算法如下：

```
void preordertre(CSnode * root) /* root 一般树的根  
    结点 */  
    { if(root!=NULL)  
        { visit(root->data); /* 访问根结点 */  
          preordertre( root-> firstchild);  
          preordertre( root-> nextsilbing);  
        }  
    }
```



5.5.3 树和森林的遍历

• 树的后根遍历

□ 后根顺序

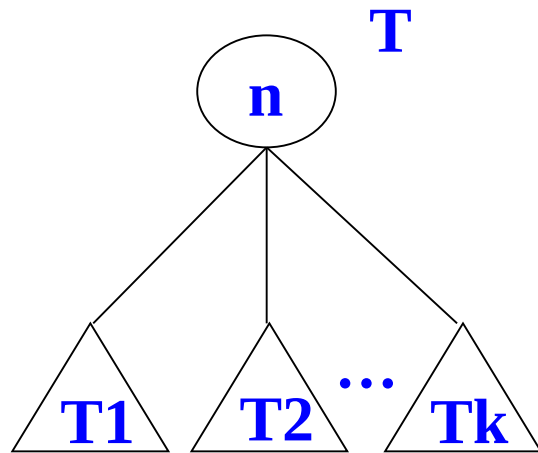
后根顺序遍历 T_1 ;

后根顺序遍历 T_2 ;

...

后根顺序遍历 T_k ;

访问根结点;





5.5.3 树和森林的遍历

- 树的后根遍历

树的后根遍历递归算法:

```
void postordertre(CSnode * root)
```

```
/* root 一般树的根结点 */
```

```
{ if (root!=NULL )
```

```
{   postordertre( root-> firstchild);
```

```
    postordertre( root-> nextsilbing);
```

```
    visit(root->data);
```

```
    }
```

```
}
```



5.5.3 树和森林的遍历

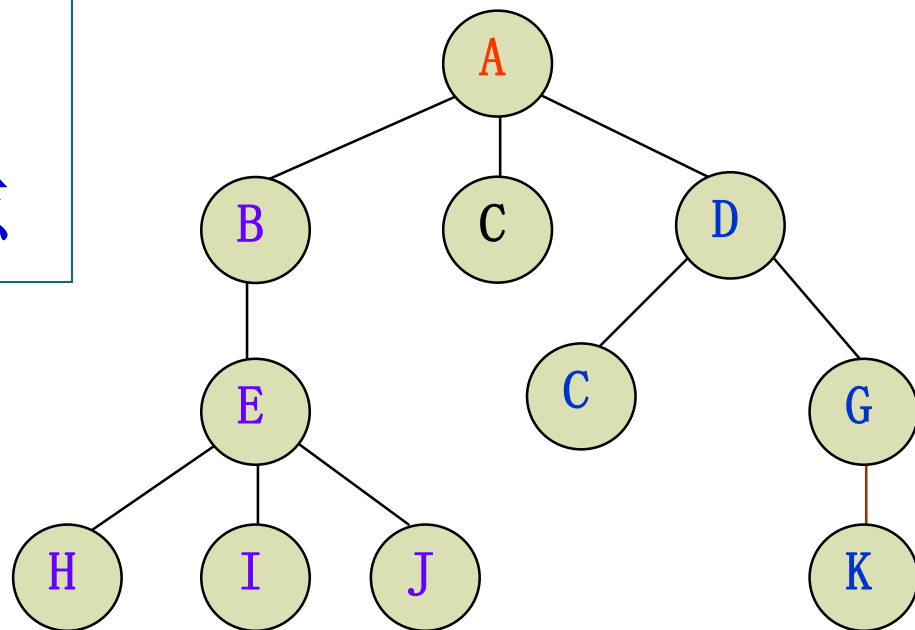
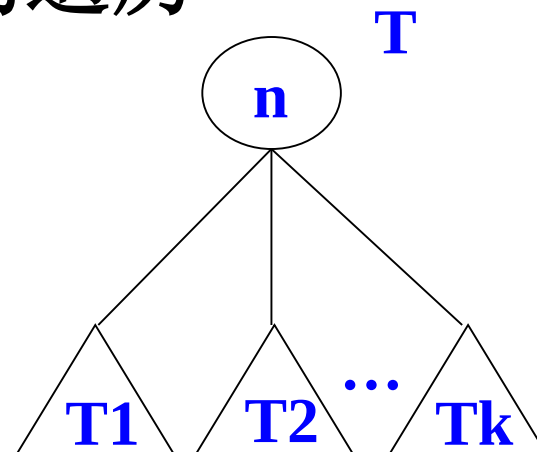
• 树的按层次遍历

□ 层次遍历

自上而下

自左到右

依次访问树中的每个结点





5.5.3 树和森林的遍历

• 树的按层次遍历

算法思想：

算法运用队列做辅助存储结构。其步骤为：

- (1) 首先将树根入队列；
- (2) 出队一个结点便立即访问之，然后将它的非空的第一个孩子结点进队，同时将它的所有非空兄弟结点逐一进队；
- (3) 重复 (2)，这样便实现了树按层遍历。



5.5.3 树和森林的遍历

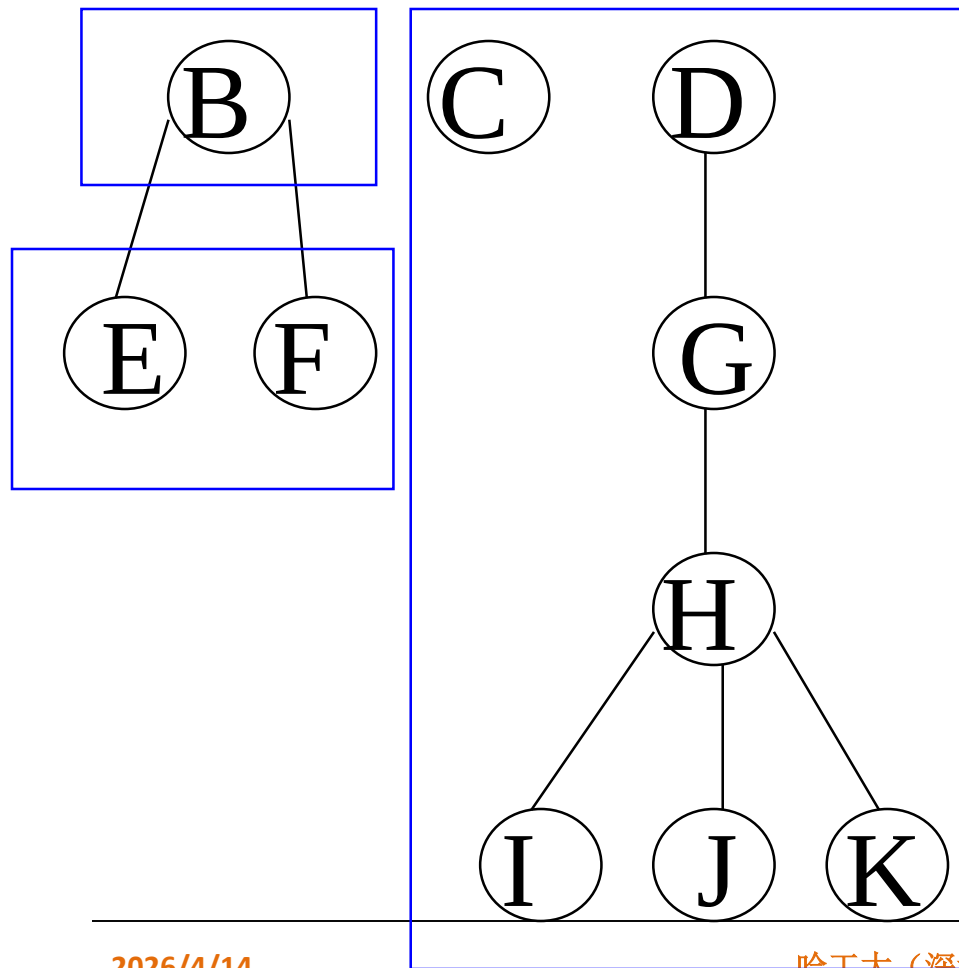
• 森林的组成部分

森林由三部分构成：

1. 森林中第一棵树的根结点；

2. 森林中第一棵树的子树森林；

3. 森林中其它树构成的森林。





5.5.3 树和森林的遍历

• 森林的先序遍历

□ 先序遍历

若森林不空，则

- 访问森林中**第一棵树的根结点**；
- 先序遍历森林中**第一棵树的子树森林**；
- 先序遍历森林中(除第一棵树之外)其余树构成的森林。

即：依次**从左至右**对森林中的每一棵树进行**先根**遍历。

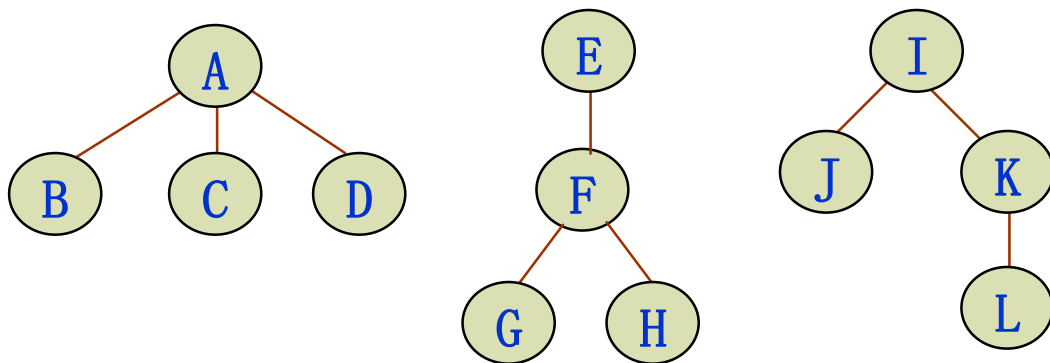
先序遍历森林



5.5.3 树和森林的遍历

• 森林的先序遍历

例



A B C D E F G H I J K L



5.5.3 树和森林的遍历

• 森林的后序（中序）遍历

不同教材的说法不一：中序或后序

□ （中）后序遍历

若森林不空，则

- 后序遍历森林中**第一棵树的子树森林**；
- 访问森林中**第一棵树的根结点**；
- 后序遍历森林中(除**第一棵树之外**)其
余树构成的森林。

即依次从**左至右**对森林中的
每一棵树进行**后根**遍历。

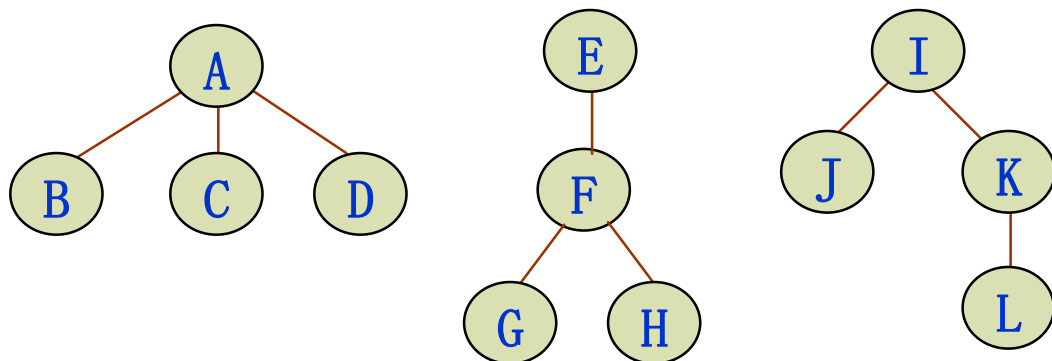
后序遍历森林



5.5.3 树和森林的遍历

- 森林的后序遍历

例 森林的遍历



B C D **A** G H F E J L K I



5.5.3 树和森林的遍历

- 森林和二叉树的对应关系

在森林转换成二叉树过程中：

森林

二叉树

第一棵树的根节点



二叉树的根

第一棵树的子树森林



左子树

剩余树森林



右子树；



5.5.3 树和森林的遍历

树的遍历和二叉树遍历的对应关系？

树

森林

二叉树

先根遍历

先序遍历

先序遍历

后根遍历

后序遍历

中序遍历

对树和森林的遍历可以调用二叉树对应的遍历算法



5.5.3 树和森林的遍历

- 求森林的深度的算法:

```
int TreeDepth(CSTree T) { //T是森林
    if(!T) return 0;
    else {
        h1 = TreeDepth( T->firstchild );
        //森林中第一棵树的子树森林的深度
        h2 = TreeDepth( T->firstbrother);
        //森林中其他子树构成的森林的深度, 与T在同一层
        return(max(h1+1, h2));
    }
} // TreeDepth
```



5.6 树和森林

5.5.1 树的存储结构

5.5.2 树、森林与二叉树的转换

5.5.3 树和森林的遍历

5.5.4 huffman树及其应用



5.5.4 huffman树及其应用 (连接第8章贪心算法)

- **应用：**在电报通信中，电文是以二进制按照一定的编码反射传送。
- **发送方：**按照预先规定的方法将要传送的字符换成0和1组成的序列----编码；
- **接收方：**由0和1组成的序列换成对应的字符----解码

如何编码能获得较高的传送效率？

Huffman成功解决了该问题！



5.5.4 huffman树及其应用

- **Huffman**教授简介

- **David Huffman**教授，美国人，1999年逝世。
- 在他的一生中，他对于有限状态自动机、开关电路、异步过程和信号设计有杰出的贡献。
- 他发明的**Huffman**编码能够使我们通常的数据传输数量减少到最小。
- 1950年，**Huffman**在MIT(麻省理工)的信息理论与编码研究生班学习。
Robert Fano教授让学生们自己决定是参加期末考试还是做一个大作业。而**Huffman**选择了后者，原因很简单，因为解决一个大作业可能比期末考试更容易通过。这个大作业促使了**Huffman**以后算法的诞生。



5.5.4 huffman树及其应用

Huffman树是一类带权路径长度最短的树

---哈夫曼树

---赫夫曼树

---最优二叉树

应用

- 哈夫曼编码;
- 通信与数据传送的二进制编码
- 堆结构
- 树形选择排序;
- 折半查找的判定树;
- 动态查找的二叉排序树;
-



5.5.4 huffman树及其应用

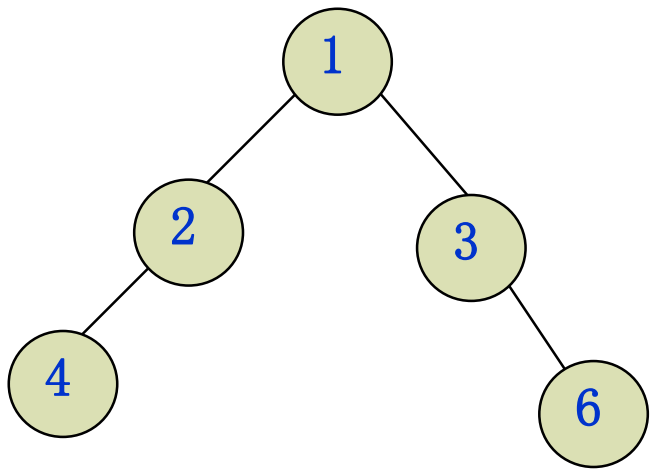
- Huffman树的相关概念

□ 结点的路径长度:

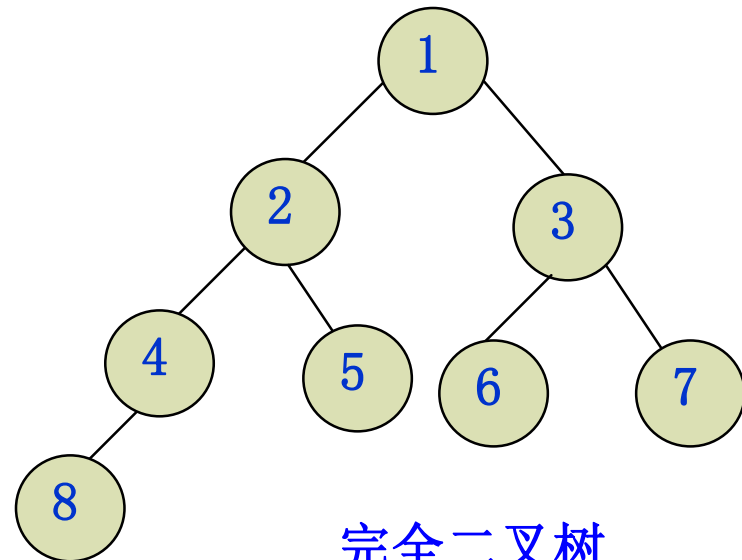
从根结点到该结点的路径上分支的数目

□ 树的路径长度:

树中每个结点的路径长度之和。



非完全二叉树



完全二叉树



5.5.4 huffman树及其应用

- Huffman树的相关概念

□ 结点的路径长度:

从根结点到该结点的路径上分支的数目

□ 树的路径长度:

树中每个结点的路径长度之和。

在结点数相同的条件下，
完全二叉树是路径最短的二叉树



5.5.4 huffman树及其应用

- Huffman树的相关概念

□ 树的带权路径长度：

树中所有叶子结点的带权路径长度之和

$$WPL(T) = \sum w_k l_k \text{ (对所有叶子结点)。}$$

□ “最优树”或Huffman树。

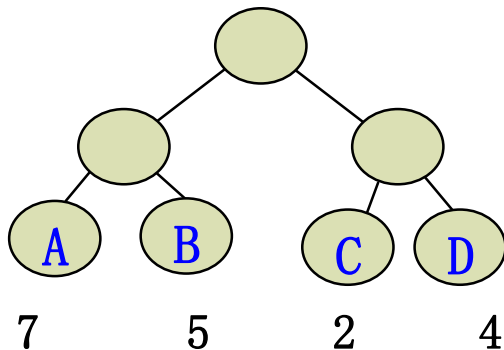
在所有含 n 个叶子结点、并带相同权值的 m 叉树中，必存在一棵其带权路径长度取最小值的树，称为“最优树”或Huffman树。

□ 最优二叉树或哈夫曼树（Huffman）：带权路径长度 WPL 最小的二叉树，称该二叉树为最优二叉树或哈夫曼树。

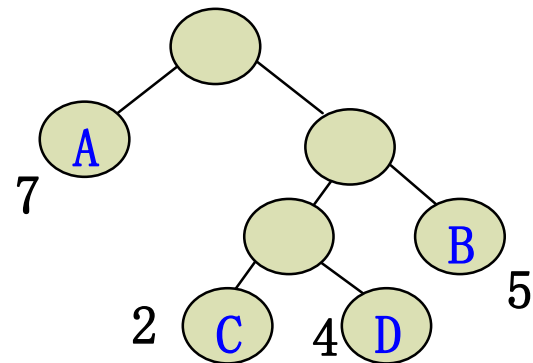


5.5.4 huffman树及其应用

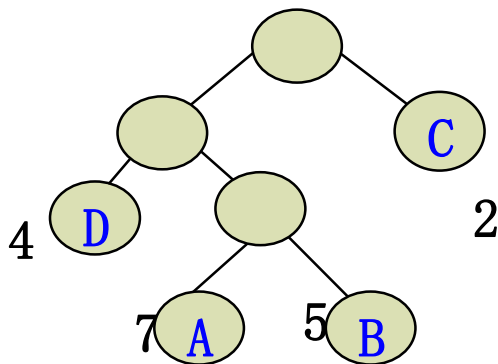
• 带权路径长度



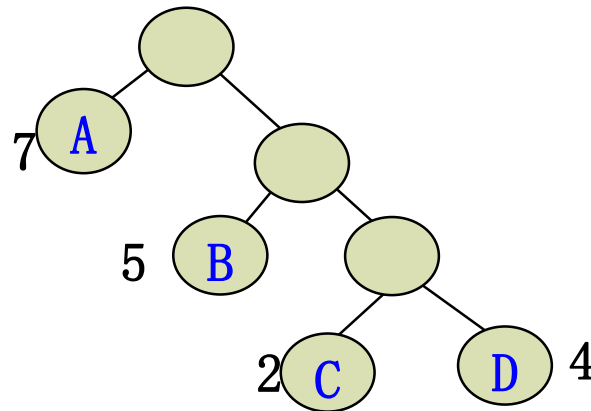
$$WPL = 7 \times 2 + 5 \times 2 + 2 \times 2 + 4 \times 2 = 36$$



$$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$$



$$WPL = 7 \times 3 + 5 \times 3 + 2 \times 1 + 4 \times 2 = 46$$



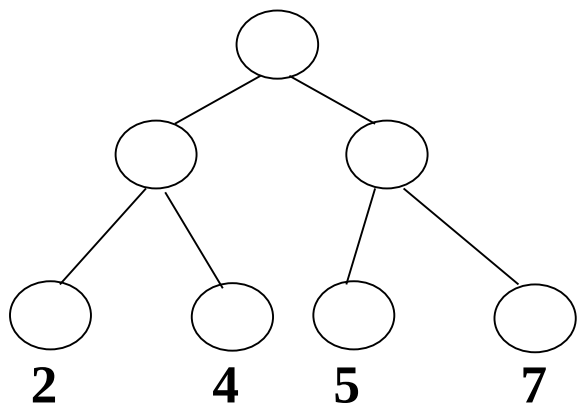
$$WPL = 7 \times 1 + 5 \times 2 + 2 \times 3 + 4 \times 3 = 35$$



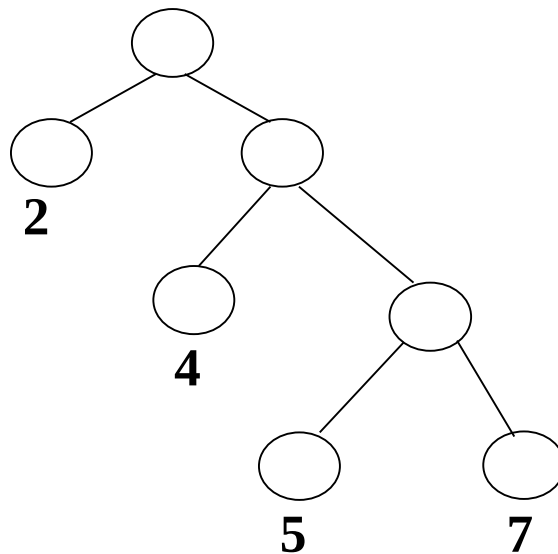
5.5.4 huffman树及其应用

- Huffman树的相关概念

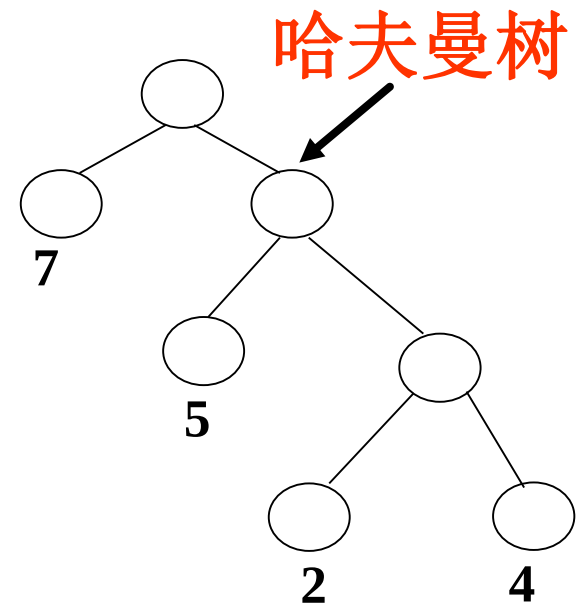
在哈夫曼树中，权值大的结点离根最近。



(a) WPL=36



(b) WPL=46



(c) WPL=35



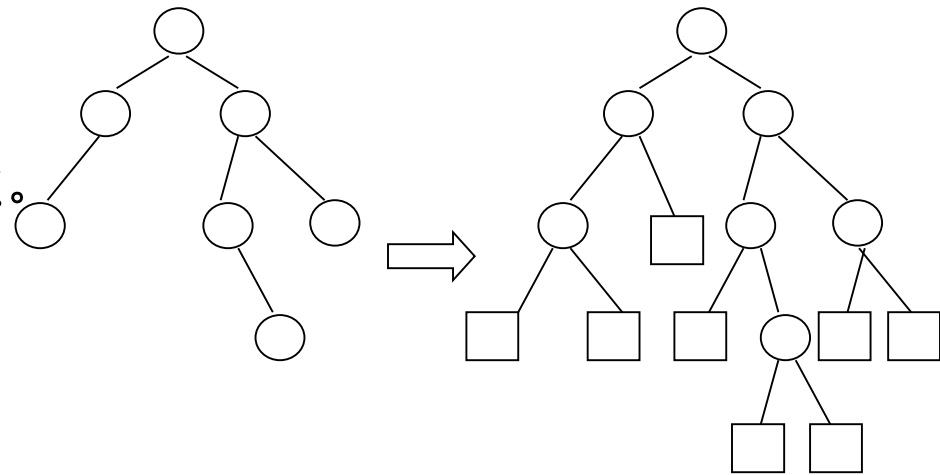
5.5.4 huffman树及其应用

• Huffman树的相关概念

将非满二叉树中，所有度不满2的节点扩充为2，便得到了扩充二叉树。扩充的节点称为外部结点，其余原来的节点称为内部结点。

增长树

{	内结点 ○
	外结点 □



如内结点数为 n ，则外结点 $S = n + 1$

为什么 $S = n + 1$?

$$n_0 = n_2 + 1$$

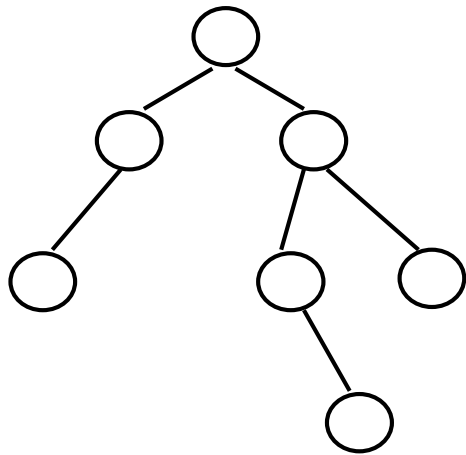


5.5.4 huffman树及其应用

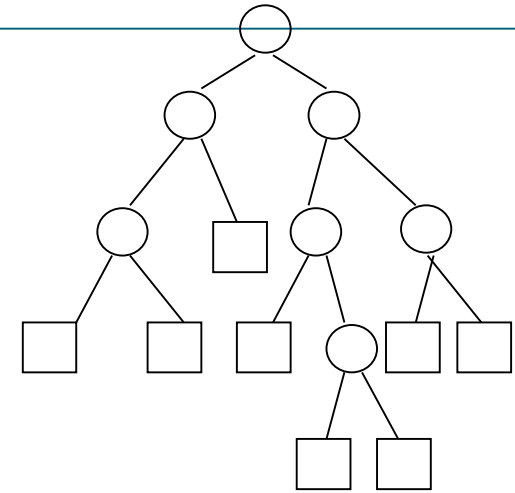
- Huffman树的相关概念

内结点路径长度I: 从根结点到每个内结点的路长的总和。

外结点路径长度E: 从根结点到每个外结点的路长的总和。



$$I = 2 \times 1 + 3 \times 2 + 1 \times 3 = 11$$



$$E = 1 \times 2 + 5 \times 3 + 2 \times 4 = 25$$

结论: 如内结点路径长度为I, 则外结点路径长度

$$E = I + 2 \times n$$



5.5.4 huffman树及其应用

- Huffman树的构造

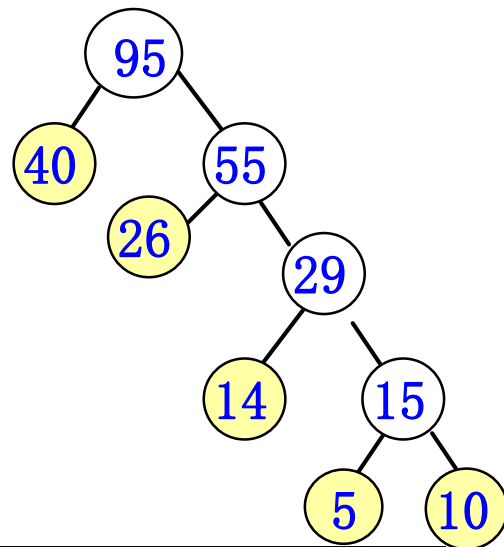
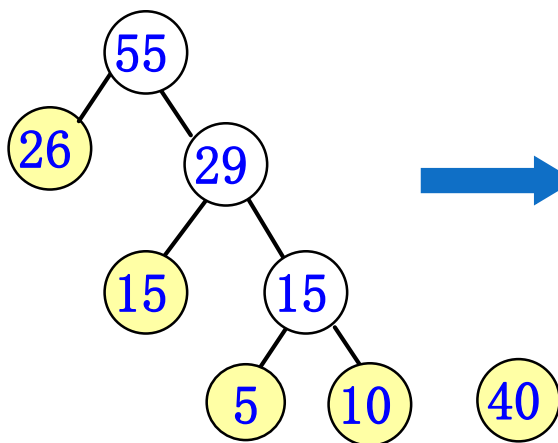
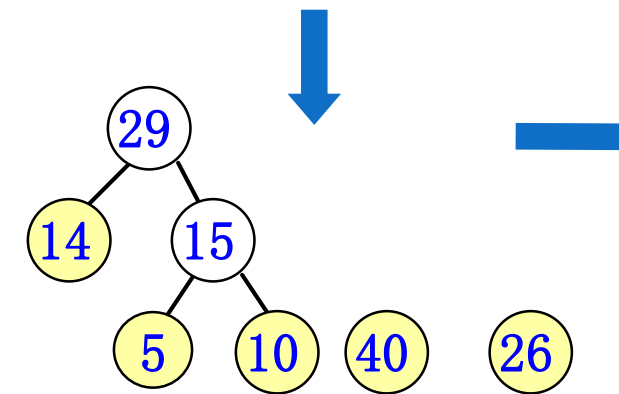
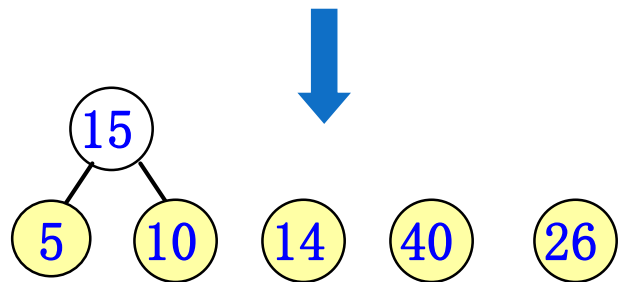
构造Huffman树的步骤:

1. 根据给定的 n 个权值，构造 n 棵只有一个根结点的二叉树， n 个权值分别是这些二叉树根结点的权。
2. 设 F 是由这 n 棵二叉树构成的集合，在 F 中选取两棵根结点权值最小的树作为左、右子树，构造一颗新的二叉树，置新二叉树根的权值=左、右子树根结点权值之和；
3. 从 F 中删除这两棵树，并将新树加入 F ；
4. 重复 2、3，直到 F 中只含一颗树为止； 这棵树便是Huffman树



5.5.4 huffman树及其应用

例 构造以 $W = (5, 14, 40, 26, 10)$ 为权的哈夫曼树。



要使二叉树WPL小,就须在构造树时,将权值大的结点靠近根。



5.5.4 huffman树及其应用

- Huffman树的构造

Huffman树中没有度为1的结点，这类树又
称严格的（strict）或正则的二叉树

n个结点构造的Huffman树最终有多少个结点？

Huffman树的构造过程说明：

n个结点需要进行 $n-1$ 次合并，每次合并都产生一个
新的结点，所以最终的Huffman树共有 $2n-1$ 个结点。



5.5.4 huffman树及其应用

- Huffman算法

Huffman 树结点结构

```
typedef struct
```

```
{
```

```
    float weight;
```

```
    int lch,rch,parent;
```

```
}hufmtree;
```

```
hufmtree tree[m];
```

```
CreatHuffmanTree (tree)
```

```
    hufmtree tree[ ]
```

```
    {        int i,j,p1,p2;
```

```
              float small1,small2,f;
```

```
              for (i=0; i<m; i++)
```

```
                  //初始化
```

```
                  {
```

```
                      tree[i].weight=0.0;
```

```
                      tree[i].lch=-1;
```

```
                      tree[i].rch=-1;
```

```
                      tree[i].parent=-1;
```

```
                  }
```

```
    }
```

图示



5.5.4 huffman树及其应用

- Huffman算法

```
int Huffmantree(HuffmanTree T)
{
    //输入权值
    float w;int i;
    for (i=0; i<n;i++)
        {scanf("%f",&w);
         tree[i].weight=w;}
    }
    for(i=n; i<m; i++)
        //把合并后的结点放入向量tree[n]~tree[m]
        {p1=0;p2=0; // 两个根结点在向量tree中的下标
         small1=maxval;small2=mavval;
         //maxval 是float类型的最大值
```



5.5.4 huffman树及其应用

- Huffman算法

```
for (j=0;j<i-1;j++)           //选择权值最小和次小的结点
    if(tree[j].parent==-1)
        if(tree[j].weight<small1)
        {
            small2=small1;//改变最小权，次小权及其位置
            small1=tree[j].weight;//找出最小的权值
            p2=p1; p1=j; }
        else if(tree[j].weight<small2)
            {small2=tree[j].weight;//改变次小权及位置
              p2=j;}
    }//选择结束
```



5.5.4 huffman树及其应用

- Huffman算法

```
tree[p1].parent=i+1;  
tree[p2].parent=i+1;  
//新生成的结点放在向量tree[i+1]  
  
tree[i].lch=p1;  
tree[i].rch=p2;  
tree[i].weight= tree[p1]. weight + tree[p1]. weight;  
}  
} //huffmantree
```



5.5.4 huffman树及其应用

- Huffman算法

Huffman 树结点结构

```
typedef struct
{
    float weight;
    int lch,rch;
    int tag;
}huffree;
hufftree tree[m];
```

tag=0 结点独立

Tag=1 结点已并入树中

	tag	lch	weight	rch
1	0	0	5	0
2	0	0	6	0
3	0	0	2	0
4	0	0	9	0
5	0	0	7	0
6				
7				
8				
9				



5.5.4 huffman树及其应用

- Huffman编码

Huffman树在通讯编码中的一个应用

利用哈夫曼树构造一组最优前缀编码。主要用途是实现数据压缩。在某些通讯场合，需将传送的文字转换成由二进制字符组成的字符串。

方法：

利用哈夫曼树构造一种不等长的二进制编码，并且构造所得的哈夫曼编码是一种最优前缀编码，使所传电文的总长度最短。



5.5.4 huffman树及其应用

- Huffman编码

- 前缀编码

任何一个字符的编码都不是同一字符集中另一个字符的编码的前缀。

例 前缀编码 A: 0 B: 110 C: 10 D: 111

发送方：将ABACCDA 转换成 0110010101110 发出

接收方： 0110010101110 。所得的译码是**唯一**的。



5.5.4 huffman树及其应用

- Huffman编码

- 不等长编码

即各个字符的编码长度不等。(0, 10, 110, 011,)

例 不等长编码 A: 0 B: 00 C: 1 D: 01

发送方: 将ABACCD A 转换成 000011010

接收方: 000011010 转换成 ?

不是前缀编码



5.5.4 huffman树及其应用

- Huffman编码

□等长编码

这类编码的二进制串的长度取决于电文中不同的字符个数，假设需传送的电文中只有四种字符，只需两位字符的串便可分辨。(01, 10, 11, 00)

例 要传输的原文为**ABACCDA**

等长编码 **A: 00 B: 01 C: 10 D: 11**

发送方: 将**ABACCDA** 转换成 **00010010101100**

接收方: 将 **00010010101100** 还原为 **ABACCDA**



5.5.4 huffman树及其应用

- Huffman编码

□不等长编码的好处

可以使传送电文的字符串的总长度尽可能地短。对出现频率高的字符采用尽可能短的编码，则传送电文的总长便可减少。



5.5.4 huffman树及其应用

- 利用二叉树设计前缀编码：

例 某通讯系统只使用8种字符a、b、c、d、e、f、g、h，其使用频率分别为0.05, 0.29, 0.07, 0.08, 0.14, 0.23, 0.03, 0.11，利用二叉树设计一种不等长编码：

- 1) 构造以 a、b、c、d、e、f、g、h为叶子结点的二叉树；
- 2) 将该二叉树所有左分枝标记0，所有右分枝标记1；
- 3) 从根到叶子结点路径上标记作为叶子结点所对应字符的编码；



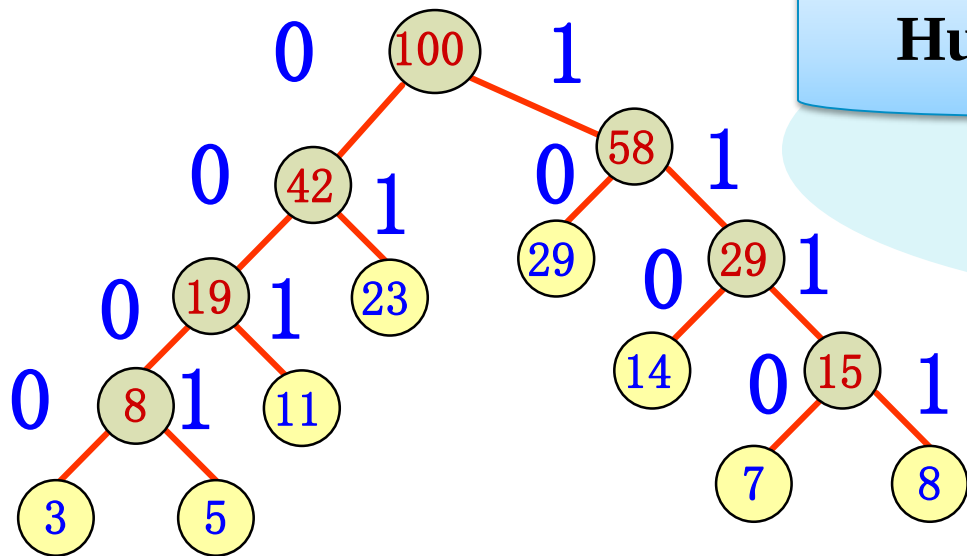
5.5.4 huffman树及其应用

- 利用二叉树设计前缀编码:

a、b、c、d、e、f、g、h,
5、29、7、8、14、23、3、11

Huffman编码从上到下编码

构造以字符使用频率
作为权值的Huffman树



a: 0001

b: 10

c: 1110

d: 1111

e: 110

f: 01

g: 0000

h: 001

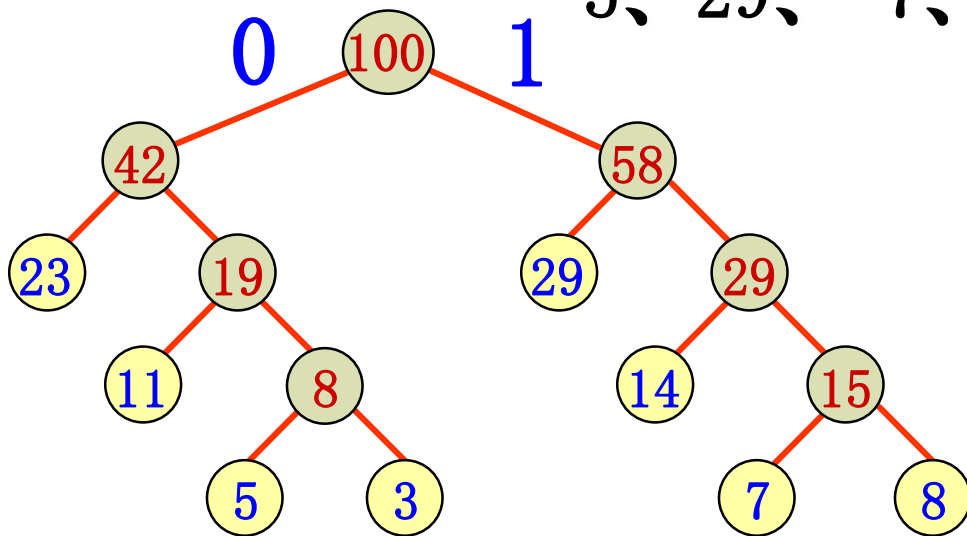
- 总编码长度等于Huffman树的带权路径长度WPL。
- Huffman编码是一种前缀编码。解码时不会混淆。



5.5.4 huffman树及其应用

- 利用二叉树设计前缀编码：

a、b、c、d、e、f、g、h,
5、29、7、8、14、23、3、11



a: 0110

b: 10

c: 1110

d: 1111

e: 110

f: 00

g: 0111

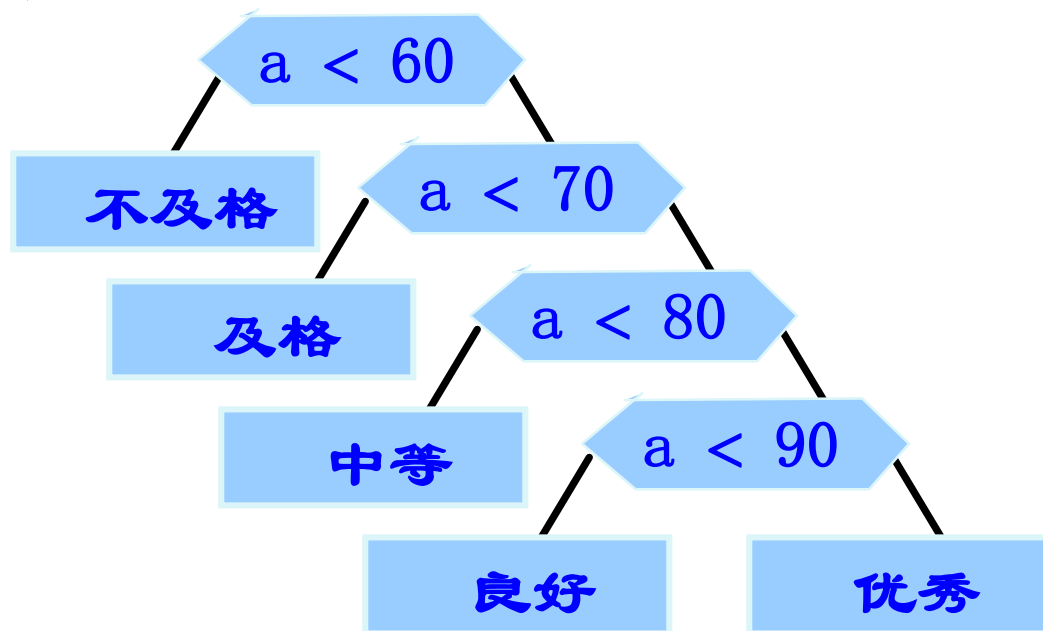
h: 010

不同的规则获得的编码是不同的



5.5.4 huffman树及其应用

例 编制一个将百分制转换成五分制的程序。可用二叉树描述判定过程。



在求得某些判定问题时，利用Huffman树获得最佳判定算法。



5.5.4 huffman树及其应用

例 设有10000个百分制分数要转换，设学生成绩在5个等级以上的分布如下：

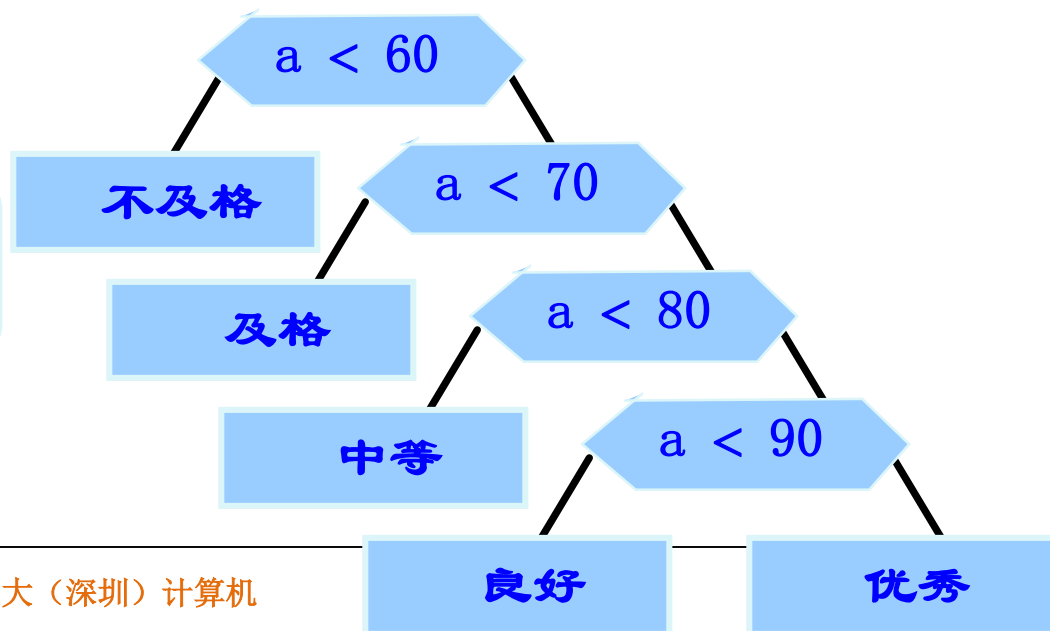
分数	0-59	60-69	70-79	80-89	90-100
比例数	0.05	0.15	0.40	0.30	0.10

其判定过程：

转换一个分数所需的比较次数= 从根到对应结点的路径长度

转换10000个分数所需的总比较次数= $10000 \times (0.05 \times 1 + 0.15 \times 2 + 0.4 \times 3 + 0.3 \times 4 + 0.1 \times 4)$

二叉树的
带权路径长度

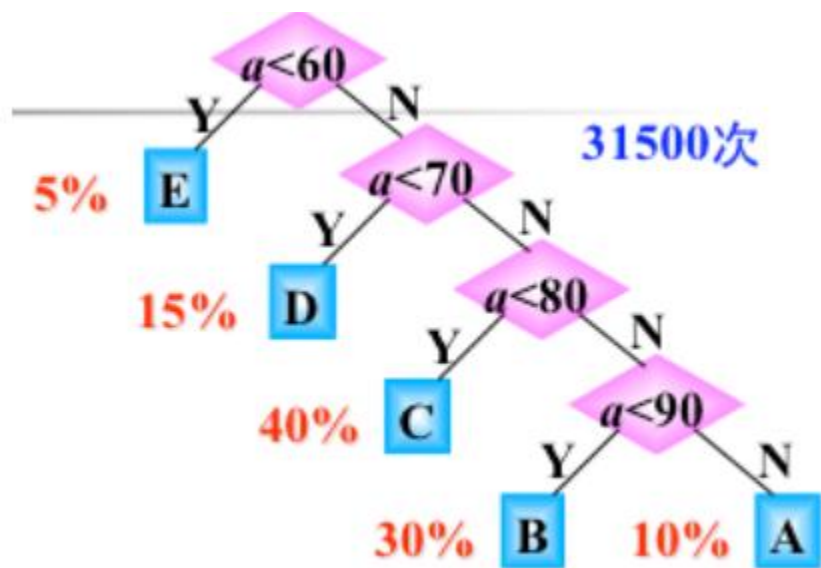




5.5.4 huffman树及其应用

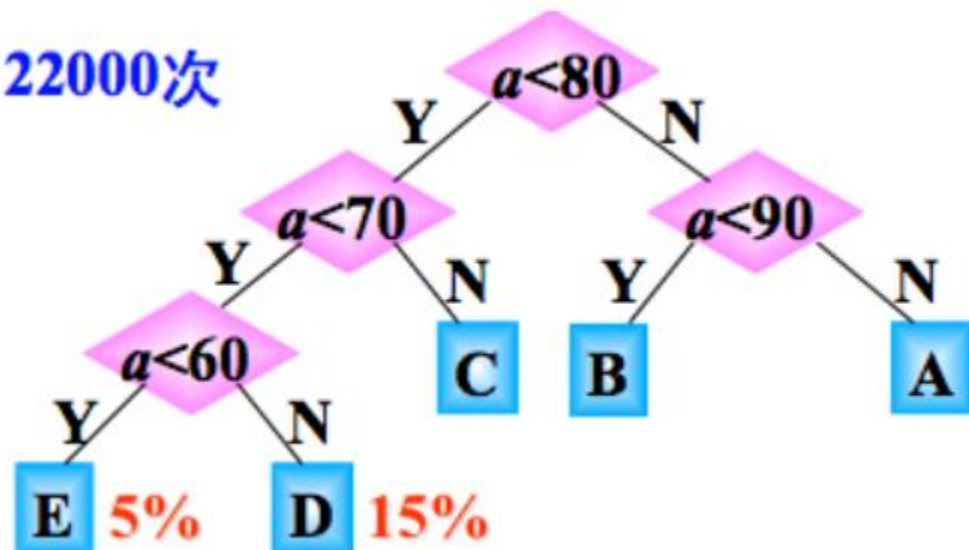
例 设有10000个百分制分数要转换，设学生成绩在5个等级以上的分布如下：

分数	0-59	60-69	70-79	80-89	90-100
比例数	0.05	0.15	0.40	0.30	0.10



次数为： $10000 (5\% + 2 \times 15\% + 3 \times 40\% + 4 \times 10\%) = 31500$ 次

22000次



此种形状的二叉树，需要的比较次数是： $10000 (3 \times 20\% + 2 \times 80\%) = 22000$ 次，显然：两种判别树的效率是不一样的。



本章小结

树和二叉树

树的ADT

逻辑结构
存储结构

二叉树

二叉树逻辑结构
二叉树存储结构
二叉树基本性质
二叉树的遍历
二叉树的应用
线索二叉树

顺序存储
二叉链表

先根遍历
中根遍历
后根遍历
层次遍历

树和森林

树的存储结构

树和森林的转换与二叉树的转换
树和森林的遍历

树的应用

Huffman树
判定过程



本章小结

- ✓ 树和二叉树两种数据类型的定义
- ✓ 二叉树的性质
- ✓ 二叉树的遍历
- ✓ 线索二叉树
- ✓ 二叉树的应用
- ✓ 树存储和遍历
- ✓ 树、森林与二叉树之间的转换
- ✓ 树的应用（重点是Huffman 树）



本章小结

- ✓ 熟练掌握二叉树的结构特性，了解相应的证明方法。
(完全二叉树、满二叉树、哈夫曼树)
- ✓ 熟悉二叉树的各种存储结构的特点及适用范围。
- ✓ 遍历二叉树是二叉树各种操作的基础。实现二叉树遍历的具体算法与所采用的存储结构有关。掌握各种遍历策略的递归算法，灵活运用遍历算法实现二叉树的其它操作。层次遍历是按另一种搜索策略进行的遍历。
- ✓ 理解二叉树线索化的实质是建立结点与其在相应序列中的前驱或后继之间的直接联系；
- ✓ 熟练掌握二叉树的线索化过程以及在中序线索化树上找给定结点的前驱和后继的方法；
- ✓ 二叉树的线索化过程是基于对二叉树进行遍历，而线索二叉树上的线索又为相应的遍历提供了方便。



本章小结

- ✓ 熟悉树的各种存储结构及其特点，掌握树和森林与二叉树的转换方法。
- ✓ 建立存储结构是进行其它操作的前提，因此应掌握 1 至 2 种建立二叉树和树的存储结构的方法。
- ✓ 学会编写实现树的各种操作的算法。
- ✓ 了解最优树的特性，掌握建立最优树和哈夫曼编码的方法。